

Kierunek: **Informatyka techniczna (ITE)**
Specjalność: **Grafika i systemy multimedialne (IGM)**

PRACA DYPLOMOWA
MAGISTERSKA

**Optymalizacja algorytmu Path
Tracingu w grafice komputerowej**

**Optimization of Path Tracing
Algorithm in computer graphics**

Adam Krizar

Opiekun pracy
dr inż. Marek Woda K3oWo4NDo3

Słowa kluczowe: path tracing, optymalizacja, CUDA, C++, grafika komputerowa

Streszczenie

Na rynku od kilku lat znajdują karty graficzne umożliwiające wykorzystanie path-tracing lub ray-tracingu w czasie rzeczywistym w grach komputerowych. Pomimo tego, programy wykorzystujące obiektywne algorytmy path-tracingu w dalszym ciągu potrzebują często wielu godzin na stworzenie pojedynczego renderu sceny. Głównym powodem tej sytuacji jest konieczność próbkowania każdego piksela generowanego obrazu wiele razy w celu uzyskania jak najdokładniejszej informacji. Jest to konieczne ponieważ w obiektywnym path-tracingu nie mamy gwarancji, że wysłany przez nas promień wybierze najbardziej optymalną ścieżkę. Jest możliwe, że promień nie przyniesie żadnej informacji w maksymalnym limicie odbić.

Praca ta proponuje zmodyfikowanie algorytmu path-tracingu aby wyeliminować taką sytuację. Poprzez dodanie sprawdzenia czy trafiony przez obiekt jest oświetlony przez źródło światła eliminujemy zmarnowane promienie, które nie wnoszą żadnej informacji do renderowanego obrazu. Wyemitowane promienie w dalszym ciągu są odbijane w sposób losowy na scenie co pozwala zachować nam zalety algorytmów obiektywnych.

Proponowane rozwiązanie wymaga mniejszej maksymalnej ilości odbić na testowanej scenie i oferuje obrazy końcowe o wyższej jakości dla takiej samej ilości próbek, kosztem nieco wydłużonego czasu renderu.

Słowa kluczowe: path tracing, optymalizacja, CUDA, C++, grafika komputerowa

Abstract

It has been few years since graphics cards that enable real time path-tracing or ray-tracing in computer games appeared on the market. Despite this, programs using unbiased path-tracing still often require many hours to finish a render of a single scene. Main reason for this is the necessity to probe each pixel of the final image many times in order to get most precise information. This happens because in an unbiased path-tracing it cannot be guaranteed that the ray will choose the most optimal path. As a result of that it is possible that given sample will not yield any useful information for the final image in a max reflections limit.

In this document I am proposing a modification to the path-tracing algorithm that will eliminate such issues. By adding additional check if hit object is lit by the light source we eliminate rays that did not bring any information to the final image. Emitted rays are still randomly reflected in the scene which lets us keep advantages of the classic unbiased algorithm.

This solution requires lesser max reflection limit and offers images of better quality for the same amount of samples, at the cost of slightly increased computation time.

Keywords: path tracing, optimization, CUDA, C++, computer graphics

Spis treści

1. Wstęp	6
1.1. Wprowadzenie	6
1.2. Cel pracy	6
1.3. Inne prace	7
1.4. Testowane modele	8
1.5. Teza	14
2. Aplikacja testowa	15
2.1. Smallpt	15
2.2. Opis aplikacji	16
2.3. Przykłady użycia	19
2.4. Wyjście programu	20
3. Środowisko testowe i metodologia	21
3.1. Środowisko	21
3.2. Metodologia	22
4. Rozwój aplikacji	28
4.1. Napotkane problemy	28
4.2. Wersje GPU	31
5. Testy aplikacji	39
5.1. Wersja CPU - punkt odniesienia	39
5.2. Wybór algorytmu	45
6. Optymalizacja	74
6.1. Model klasyczny	74
6.2. Model hybrydowy	83
6.3. Model rozszerzony	87
7. Porównanie modeli oświetlenia	99
7.1. Cornell Box	99
7.2. Lustra	105
7.3. Labirynt	111
8. Podsumowanie	120
8.1. Wnioski	120
8.2. Dodatkowe badania	121
8.3. Dalszy rozwój	121
Literatura	122
Spis rysunków	124
Spis tabel	127
Spis listingów	137
A. Instrukcja wdrożeniowa	138
A.1. Wymagania:	138
A.2. Kompilacja	139
A.3. Uruchomienie programu	139

A.4. Plik Json	139
B. Opis załączonej płyty CD/DVD	141

Skróty

B/I (brak informacji - oznaczenie pola, którego wartość nie mogła być obliczona)

K/C (koniec czasu - oznaczenie testu, który przekroczył limit czasu wykonania)

MiB (Mebibajt/Mebibajty)

Rozdział 1

Wstęp

1.1. Wprowadzenie

W przeciągu ostatnich lat na rynku pojawiły się karty graficzne, które umożliwiają tworzenie realistycznej grafiki z wykorzystaniem path-tracingu i jego odmian. Jest to jednak technologia w dalszym ciągu ograniczona. Wymaga ona drogich kart graficznych z wyższej półki. Dodatkowo w większości przypadków algorytmy te są wykorzystywane tylko w niektórych elementach sceny: cienie, odbicia lub załamanie światła w obiektach przezroczystych. Niewiele gier pozwala na wykorzystanie path-tracingu jako jedynego modelu oświetlenia. Gry, które to robią w większości przypadków są oparte o stare tytuły z prostą grafiką:

- Quake II - gra oryginalnie wydana w 1997 roku, zmieniony model oświetlenia został dodany do gry w 2019 roku,
- Portal - gra z 2007 roku, path-tracing został do niej dodany w 2022 roku - gra jest grywalna tylko na najszybszych kartach graficznych,
- Minecraft - tytuł z 2011 roku, wsparcie dla ray-tracingu zostało dodane w 2019 roku. Gra z prostą grafiką opartą o bloki. Wykorzystanie ray-tracingu w tym tytule wiąże się ze znacznym ograniczeniem maksymalnego zasięgu widzenia gracza.

W przypadku programów służących do tworzenia grafiki 3D - takich jak *Blender* - sytuacja wygląda jeszcze gorzej. Są one często używane do tworzenia jak najbardziej realistycznych statycznych scen, w przypadku, których nie jest możliwe wykorzystanie technik maskujących niedokładności jak w przypadku gier wideo. Programy tego typu korzystają z obiektywnych metod generowania grafiki. Ich celem jest odtworzenie świata rzeczywistego w jak najdokładniejszy sposób. Renderzy z wykorzystaniem tych technik w dalszym ciągu nie mogą być generowane w czasie rzeczywistym. W zależności od złożoności sceny oraz porządnej dokładności końcowego obrazu mogą one zająć od kilkudziesięciu sekund do kilkunastu, bądź nawet kilkudziesięciu godzin.

1.2. Cel pracy

Celem tej pracy jest zbadanie możliwości przyspieszanie procesu path-tracingu przy jednoczesnym zachowaniu jakości końcowego obrazu. Za punkt, w który daje największe możliwości rozwoju wybrano model oświetlenia sceny, a dokładnie jak szukane jest źródło światła po trafieniu w punkt na scenie. Obecnie największym problemem w algorytmie path-tracingu jest konieczność wielokrotnego próbkowania każdego piksela w celu uzyskania jak najlepszego efektu końcowego. Jeżeli znaleźć by sposób na ograniczenie potrzebnych próbek pozwoliło by to znacząco skrócić czas potrzebny na wygenerowanie obrazu. Praca ta skupia się na znalezieniu ta-

kiego rozwiązania.

Na rynku istnieje wiele programów implementujących obiektywny rendering: *Cycles*, *Mantra*, *Octane Renderer*, *Fstom Renderer* i wiele innych. Są to rozbudowane programy o zaawansowanych funkcjonalność biorące pod uwagę wiele czynników, które wykraczają poza zakres tej pracy. Celem autora nie jest stworzenie aplikacji, który będzie w stanie konkurować z tymi programami. Praca ta skupia się tylko i wyłącznie na porównaniu modeli oświetlenia w algorytmie path-tracingu.

1.3. Inne prace

W trakcie przygotowań do tej pracy, przeanalizowane zostały inne prace które powstały w dziedzinie generacji grafiki z wykorzystaniem algorytmu path-tracingu.

Pierwszą z nich była praca zatytułowana "*FIPT: Factorized Inverse Path Tracing for Efficient and Accurate Material-Lighting Estimation*"[14]. Skupia się na nowym podejściu do metody path-tracingu, która obiecuje przyspieszenie w porównaniu do tradycyjnego oraz oferuje lepsze rozwiązywanie niejasności w pomiędzy zachowaniem obiektów odbijających, a emitujących światło. Ze wszystkich badanych prac tematyka tej była najbardziej zbliżona do zagadnienia badanego w tej pracy. Skupia się ona jedna na innych elementach algorytmu path-tracingu w związku z czym nie stanowi ona bezpośredniego porównania dla badań przeprowadzonych w tej pracy.

Kolejną pracą jest "*Combinatorial bidirectional path-tracing for efficient hybrid CPU/GPU rendering*"[10]. Skupia się bardziej na aspekcie implementacji algorytmu niż na zmianach w samym działaniu jego działania. Praca ta posłużyła za inspirację przy implementacji programu wykorzystanego w tej pracy - jedna z testowanych wersji została napisana mając na celu maksymalizację wykorzystanie zarówno CPU jak i GPU w procesie generowania sceny.

"*Bi-Directional Path Tracing*"[7] jest jedną z wcześniejszych prac dotyczących path-tracingu. Została ona napisana w 1998 roku, czyli ma tyle samo lato co autor tego dokumentu. Pomimo tego pozostaje ona aktualna i była jednym z głównych źródeł informacji, które pomogły przy implementacji i zrozumieniu algorytmu path-tracingu.

Praca "*ReSTIR GI: Path Resampling for Real-Time Path Tracing*"[9] wspomina o podobnym problemie z bardzo małym udziałem metod śledzenia promieni w grafice komputerowej pomimo tego, że sprzęt do tego zdolny jest już obecny od jakiegoś czasu na rynku. Skupia się ona na redukcji błędów oświetlenia poprzez dodanie nowego algorytmu path-samplingu zoptymalizowanego pod kątem wysoce równoległej architektury GPU.

Jednym z ważniejszych dokumentów, na którym zostały oparte badania przeprowadzone w tej pracy jest "*Inverse Path Tracing for Joint Material and Lighting Estimation*" [2]. Wprowadza on koncept, odwrotnego path-tracingu oraz wykorzystania metod Monte Carlo w celu aproksymacji realistycznego oświetlenia sceny.

Warto także wspomnieć o wydanej w 1986 roku pracy *The rendering equation* [6], która jako jedna z pierwszych zajęła się analizą zagadnienia symulacji zjawisk optycznych w grafice komputerowej. Pomimo tego, że praca ta nie była bezpośrednio wykorzystana przeze mnie pojawiała się często w źródłach innych dokumentów wykorzystanych przy tworzeniu tej pracy. W związku z tym została ona uznana za wartą do dołączenia w źródłach tej pracy.

Ciekawą pracą jest także *Rendering synthetic objects into real scenes: Bridging traditional and*

image-based graphics with global illumination and high dynamic range photograph [5], mówiąca o technikach realistycznego renderowania obiektów na scenie poprzez symulację interakcji oświetlenia z geometrią sceny.

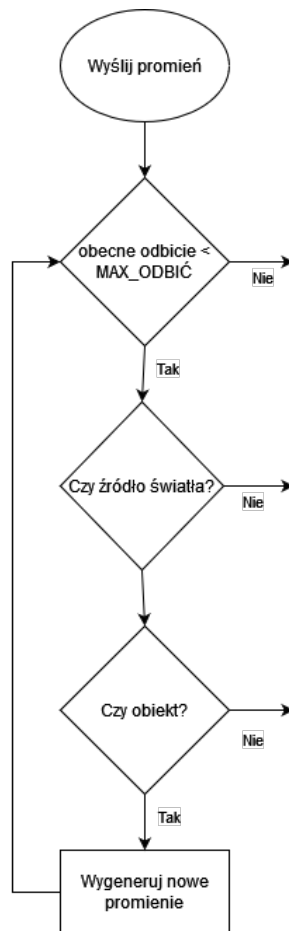
Bardzo ciekawe rozwiązanie jest proponowane przez prace wykorzystujące uczenie maszynowe do generowania oświetlenia na scenie. Przykładowo są to takie prace jak *Neural inverse rendering of an indoor scene from a single image* [11], *Learning indoor inverse rendering with 3d spatially-varying lighting* [13] lub *Dense vision transformers for single-image inverse rendering in indoor scenes* [15]. Rozwiązania tego typu nie nadają się jednak do wykorzystania jako obiektywne metody generowania grafiki. Uczenie maszynowe jest ograniczone przez dane wejściowe i nie było by sobie poradzić z sytuacją, której nie spotkało wcześniej. Ponadto oświetlenie w takim wypadku nie jest symulowane, tylko model uzupełnia oświetlenie zgodnie najbardziej pasującymi wzorcami.

Jednym z elementów, który zainspirował tematykę tej pracy było zainteresowanie działaniem renderera *cycles* dostępnego w programie Blender. Poszukiwanie informacji do tej pracy zaczęło się właśnie od artykułów dotyczących tego programu. Jednym z ciekawszych był *Comparison of time, size and quality of 3D object rendering using render engine eevee and cycles in blender* [1] opisujący porównanie pomiędzy dostępnymi rendererami. Na koniec warto jeszcze wspomnieć o książce *Learning Blender* [12], która służy jako dobre wprowadzenie w świat Blendera i wyjaśnia najważniejsze pojęcia.

Poza wymienionymi pracami istnieje wiele innych dokumentów, które poruszają tematykę związaną z *path-tracingiem*, wiele z nich skupia się na optymalizacji i poprawkach w działaniu tego algorytmu. Żadna z nich nie proponowała jednak badań dotyczących systemu oświetlenia podobnych do zaproponowanych w tej pracy. W związku z tym jest nadzieją autora, że praca ta wniesie nowe informacje w tej dziedzinie.

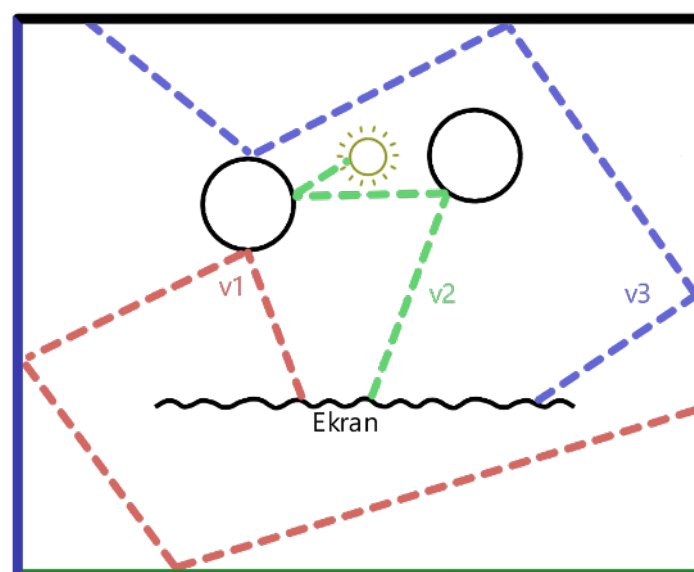
1.4. Testowane modele

Jak zostało wspomniane wcześniej, jednym z większych problemów algorytmu *path-tracingu* jest konieczność wielokrotnego próbkowania każdego piksela obrazu. Na rysunku 1.1 przedstawiono uproszczony schemat działania klasycznego algorytmu. Może on zakończyć się w trzech przypadkach, z których tylko jeden daje nam konkretne informacje o pikselu obrazu. W przypadkach braku trafienia w obiekt lub osiągnięcia maksymalnej ilości odbić zostanie zwrócony kolor czarny.



Rys. 1.1: Diagram przedstawiający uproszczony schemat algorytmu klasycznego.

Działanie wersji klasycznej zostało zilustrowanie na przykładzie na rysunku 1.2. Tylko promień v_2 wnosi nowe informacje do generowanego obrazu. Promień v_1 oraz v_3 kończą swoje odbicia po osiągnięciu maksymalnej głębokości (w tym przykładzie 4). Ponieważ nie natrafiły one na źródło światła zwracają one kolor, czarny pomimo tego, że punkty od których odbijały się te promienie, są oświetlone. Oznacza to w praktycie, że promień v_1 i v_3 zostały zmarnowane.

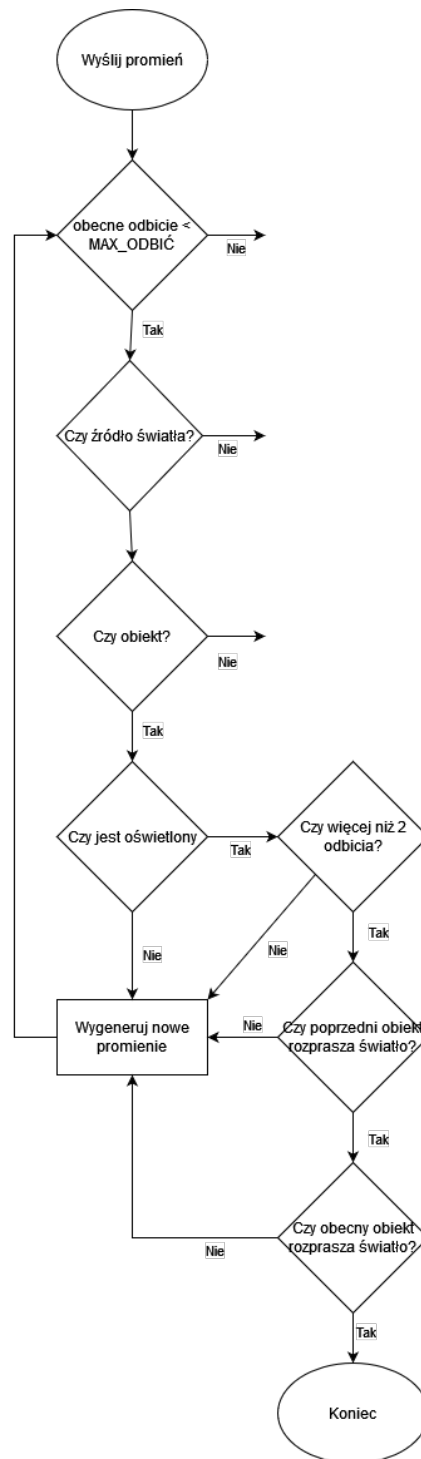


Rys. 1.2: Przykład działania modelu klasycznego.

Aby uniknąć takich sytuacji powstały dwa nowe modele oświetlenia, których celem jest wyeliminowanie jak największej ilości zmarnowanych promieni.

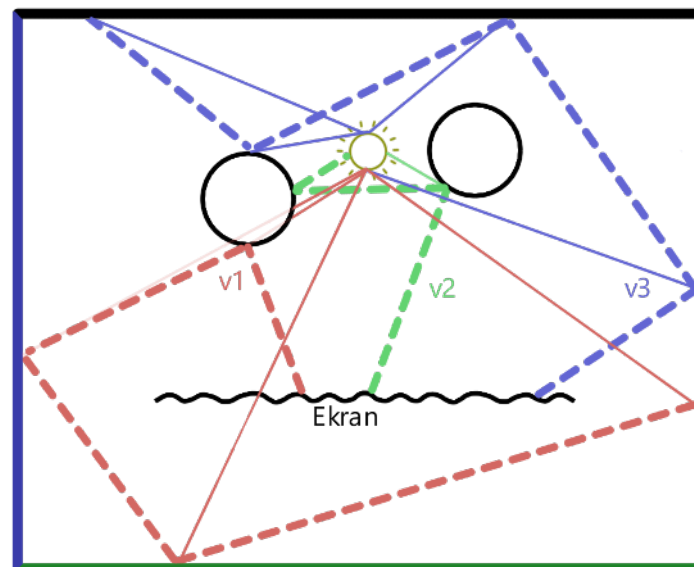
1.4.1. Model rozszerzony

Bardziej złożonym z nich jest tak zwany model rozszerzony. Dla każdego trafionego punktu na scenie dodaje on sprawdzanie czy dany element jest oświetlony przez źródło światła. W dalszym ciągu zachowuje on losowość odbić, która cechuje metody obiektywne. Idea tej wersji algorytmu została przedstawiona na diagramie 1.3. Model ten umożliwia zmniejszenie maksymalnej ilości próbek i maksymalnej głębokości odbić, koniecznych do wygenerowania obrazu. Nawet w sytuacji gdy algorytm osiąga, któryś z warunków końca (brak trafienia w obiekt, maksymalna ilość odbić) istnieje duża szansa, że oświetlenie zostało obliczone dla któregoś z wcześniejszych punktów. Dzięki temu informacja uzyskana z każdego odbicia jest więcej warta. W takim wypadku jeżeli badany promień zwróci kolor czarny możemy założyć, że dany piksel faktycznie znajduje w zaciemnionej części sceny.



Rys. 1.3: Diagram przedstawiający uproszczony schemat algorytmu rozszerzonego

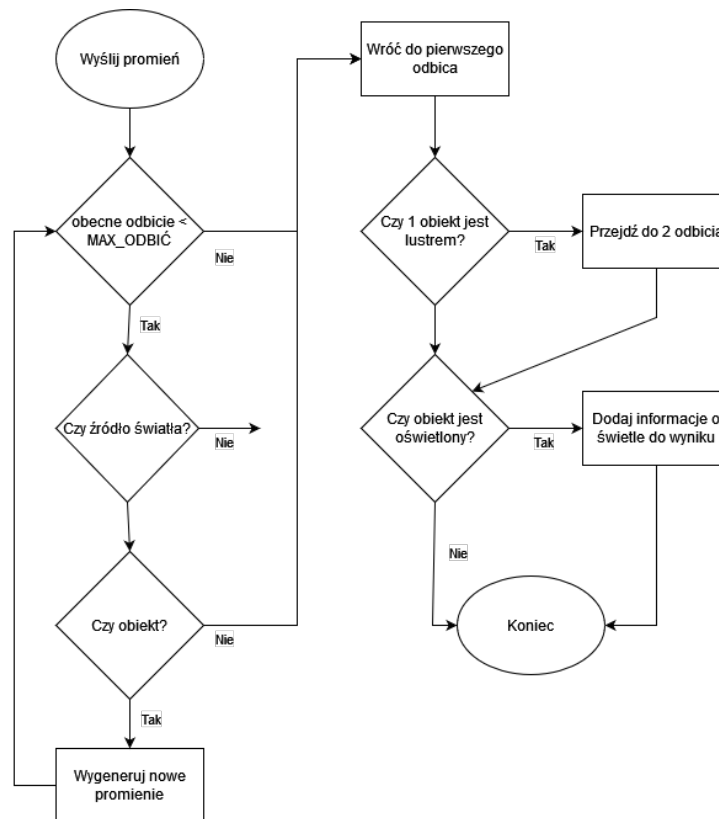
Na rysunku 1.4 przedstawiono wizualizację działania tej wersji algorytmu. Każdy z promieni wnosi dodatkową informację o kolorze końcowego obrazu. Promień $v1$ nie trafia w źródło światła, jednak jego trzecie i czwarte odbicie są oświetlone. W przypadku promienia $v3$ oświetlone są wszystkie punkty od których promień się odbił.



Rys. 1.4: Przykład działania modelu rozszerzonego.

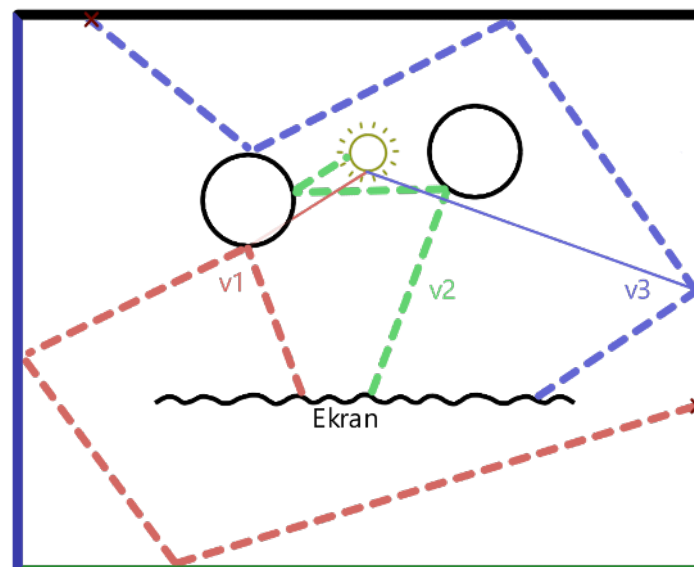
1.4.2. Model hybrydowy

W trakcie badań nad wersją rozszerzoną powstała dodatkowy model oświetlenia, którego celem było osiągnięcie części zalet modelu rozszerzonego przy niższym koszcie obliczeniowym. Model ten celował głównie w sceny z bardzo dużą ilością źródeł światła, które są słabą stroną poprzedniego modelu. Diagram przedstawiający działanie tego algorytmu został przedstawiony na rysunku 1.5. Ta wersja algorytmu, nie różni się niczym w działaniu od wersji klasycznej, w przypadku promieni, które znalazły źródło światła. Rozbudowana została natomiast obsługa nie trafionych promieni. Dla każdego z nich sprawdzane jest czy pierwszy, lub w przypadku odbicia drugi, obiekt jest oświetlony. Jeżeli tak ta informacja jest dodawana do obliczonego koloru piksela.



Rys. 1.5: Diagram przedstawiający uproszczony schemat algorytmu hybrydowego.

Na rysunku 1.6 przedstawiono działanie tej wersji algorytmu na przykładzie. Dla promienia v_2 , nie jest sprawdzane czy pierwsze odbicie jest oświetlone ponieważ ten promień trafił w źródło światła. Promień v_1 zwraca w tej wersji kolor czarny, ponieważ pierwsze odbicie znajduje się w zacienionym punkcie sceny. Natomiast promień v_3 zwraca dodatkowe oświetlenie.



Rys. 1.6: Przykład działania modelu hybrydowego.

1.5. Teza

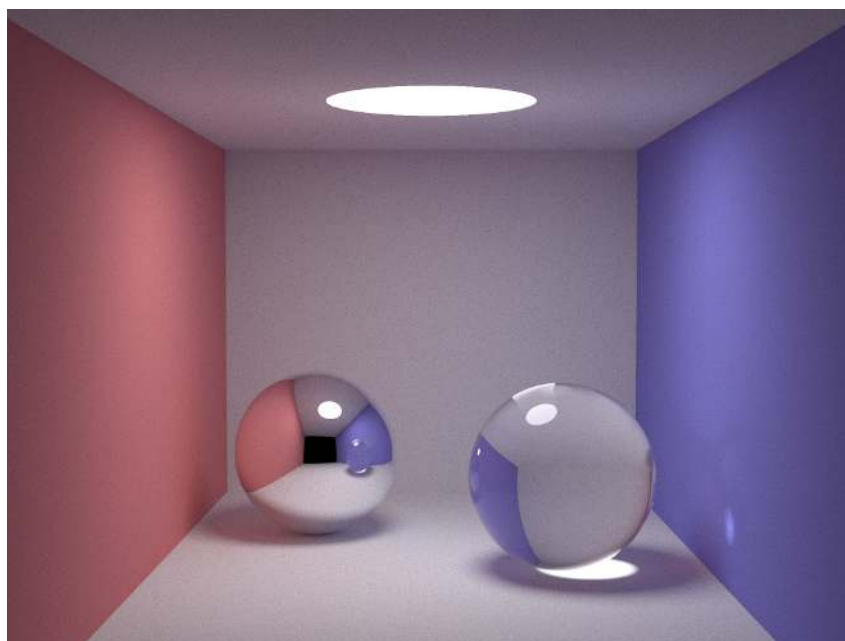
Nowy model obsługi odbić promienia wprowadzony w wersji rozszerzonej oferuje lepszą jakość wygenerowanego obrazu dla takiej samej ilości czasu w stosunku do klasycznej wersji algorytmu path-tracingu.

Rozdział 2

Aplikacja testowa

2.1. Smallpt

Rozwój aplikacji testowej został rozpoczęty od analizy przykładowej aplikacji smallpt [3] stworzonej przez Kevina Beason'a. Dodatkowo jako pomoc przy w zrozumieniu kodu tego programu wykorzystana została prezentacja [4] stworzona przez Dr. David'a Cline'a. Wyniki tej analizy zostały wykorzystane jako punkt startowy do implementacji mojej wersji algorytmu path-tracing'u. Na rysunku 2.1 możemy zobaczyć przykładową scenę stworzoną przez ten program.



Rys. 2.1: Scena Cornell Box wygenerowana przez smallpt, 5000 próbek, głębokość 10

Program ten pomimo bycia bardzo przyjaznym punktem startowym - działający path-tracer w 99 liniach kodu - posiada wiele ograniczeń, które rozbudowałem w mojej implementacji tego algorytmu. Poniżej, krótka lista problemów, które musiałem zaadresować w moim programie:

1. Zapisane na stałe zmienne, takie jak rozdzielczość sceny, czy maksymalna ilość odbić.
2. Zapisana na stałe w kodzie programu renderowana scena. Wprowadzenie jakichkolwiek zmian wymaga ponownej kompilacji programu.
3. Brak obsługi innych figur geometrycznych niż kula. Efekt widoczny na 2.1 został osiągnięty poprzez manipulację wielkością kul. Ogranicza to możliwe do stworzenia sceny.

4. Brak wykorzystania funkcji języka obiektowego takich jak klasy czy polimorfizm - Zmniejsza to czytelność kodu oraz utrudnia dalszy rozwój tej aplikacji.

Dodatkowo smallpt nie wykorzystuje żadnych funkcjonalności z nowszych standardów języka C++.

2.2. Opis aplikacji

Pełna nazwa mojej aplikacji to *Improved Path-Tracer*. Pod tą nazwą można znaleźć repozytorium aplikacji na platformie GitHub (<https://github.com/AdamStudies-PWR/ImprovedPathTracer>). Program ten też identyfikuje się skróconą nazwą *tracer* (pod tą nazwą kompiluje się plik wykonywalny). W dalszych częściach tej pracy użycie tej nazwy zawsze odnosi się do programu testowego.

2.2.1. Przechowywanie danych

Konstrukcję programu została rozpoczęta od implementacji mechanizmu pozwalającego na definicję scen w pliku Json. Umożliwia to szybkie testowanie zmian w kodzie bez konieczności utrzymywania różnych wersji oprogramowania dla każdej ze scen.

Listing 2.1: Przykładowy plik JSON sceny.

```

1    "height": 720,
2    "width": 1280,
3    "camera": {
4        "direction": {
5            "xx": 0.0,
6            "yy": 0.0,
7            "zz": -1.0
8        },
9        "orientation": {
10           "xx": -1.0,
11           "yy": 0.0,
12           "zz": 0.0
13       },
14       "position": {
15           "xx": 640.0,
16           "yy": 360.0,
17           "zz": 500.0
18       }
19   },
20   "objects": [
21       {
22           "type": "sphere",
23           "reflection": 0,
24           "radius": 45.0,
25           "position": {
26               "xx": 64.0,
27               "yy": 660.0,
28               "zz": 43.0
29           },
30           "color": {
31               "xx": 0.75,
32               "yy": 0.25,
33               "zz": 0.25
34           },
35           "emission": {
36               "xx": 20.0,
37               "yy": 5.0,
38               "zz": 5.0
39           }
40       }
41   ]
42 }

```

Na listingu 2.1 znajduje się przykładowy plik definiujący scenę dla programu *tracer*. Plik ten ma cztery główne pola:

- *height* - Pole definiujące wysokość sceny i rozdzielczość pionową końcowego obrazu,
- *width* - Pole definiujące szerokość sceny i rozdzielczość poziomą końcowego obrazu,
- *camera* - Pole definiujące "kamerę" kierunek patrzenia o położenie okna z którego widziana jest dana scena,
- *objects* - Lista obiektów z których zbudowana jest dana scena.

Obiekt *camera* jest zdefiniowany za pomocą następujących pól:

- *direction* - Określa zwrot wektora, który wskazuje kierunek patrzenia,
- *orientation* - Określa zwrot wektora, który jest prostopadły w stosunku do wektora "direction", i jest skierowany w prawo od środka kamery. Pomaga zorientować kamerę w poziomie,

- *position* - Określa punkt, który jest centrum generowanego obrazu stanowi on punkt zaczepienia dla wektorów "direction"i "orientation".

Wykorzystując iloczyn wektorowy pomiędzy wektorami "direction"i "orientation"można wyznaczyć trzeci wektor, który wskazuje w górę. Służy on do zorientowania kamery w pionie.

Ważne! Wektory "direction"i "orientation"muszą być wektorami normalnymi.

W przykładowym pliku JSON 2.1 na liście obiektów znajduje się obiekt typu "sphere"(Kula). Obiekt ten jest zdefiniowany za pomocą następujących pól:

- *type* - Identyfikator typu obiektu, w tym przypadku kula,
- *reflection* - Rodzaj obiektu. Program tracer obsługuje typy "Diffuse obiekt chropowaty, "Specular lustro i "Refractive szkło,
- *radius* - Promień kuli,
- *position* - Pozycja środka obiektu w przestrzeni 3D,
- *color* - Kolor obiektu. Program tracer przechowuje dane kolorów są przechowywane w formacie **Unit RGB** - czyli są przechowywane w zakresie od 0 do 1 (gdzie 0 oznacza brak danego koloru, a 1 oznacza jego maksymalne nasycenie),
- *emission* - Określa moc emisji dla danego obiektu (W przypadku gdy dany obiekt jest źródłem światła).

Oprócz kul program *tracer* wspiera także płaszczyzny.

Listing 2.2: JSON obiektu płaszczyzny.

```

1  {
2      "type": "plane",
3      "reflection": 0,
4      "position": {
5          "xx": 640.0,
6          "yy": 360.0,
7          "zz": 0.0
8      },
9      "north": {
10         "xx": 0.0,
11         "yy": 370.0,
12         "zz": 0.0
13     },
14     "east": {
15         "xx": 650.0,
16         "yy": 0.0,
17         "zz": 0.0
18     },
19     "color": {
20         "xx": 0.75,
21         "yy": 0.75,
22         "zz": 0.75
23     },
24     "emission": {
25         "xx": 0.0,
26         "yy": 0.0,
27         "zz": 0.0
28     }
29 }
```

Płaszczyzna posiada dwa dodatkowe pola, która nie wystąpiły w przypadku kuli:

- *north* - Zwrot wektora skierowanego w górę w płaszczyźnie. Jest on zaczepiony w punkcie "position". Długość tego wektora stanowi połowę długości "pionowego" boku płaszczyzny.
- *east* - Zwrot wektora skierowanego na prawo w stosunku do wektora "north". Jest on także zaczepiony w punkcie "position", a jego długość to połowa długości "poziomego" boku płaszczyzny.

Wektory te są wykorzystywane do wyliczenia punktów znajdujących się w rogach płaszczyzny oraz do obliczenia wektora normalnego płaszczyzny.

2.2.2. Generowanie obrazu

Wersja CPU

Wersja ta implementuje podstawową wersję algorytmu path-tracing'u. Służy ona jako punkt odniesienia do oceny osiągnięć głównej wersji testowej aplikacji (GPU). Wykorzystuje ona bibliotekę Open MP (<https://www.openmp.org/spec-html/5.0/openmp.html>) do wsparcia wielowątkowości.

Wersja GPU

Główna wersja aplikacji na rozwoju, której głównie skupiono się w trakcie tworzenia tej pracy. Do pracy z GPU wykorzystuje ona bibliotekę CUDA w związku z czym do uruchomienia tej wersji aplikacji konieczne jest posiadanie GPU firmy NVIDIA. Powstało 11 wersji tej aplikacji począwszy od naiwnego portu wersji CPU na kod CUDA, po wersje eksplorujące różne modele oświetlenia. Zostały one opisane dokładnie w rozdziale 4.

2.2.3. Zapisywanie obrazu

Funkcja generacji obrazu (dla obu wersji programu) zwraca wektor pikseli o długości równej iloczynowi rozdzielczości pionowej i poziomej końcowego obrazu. Funkcja zapisu obrazu najpierw konwertuje wartości kolorów z zakresu 0-1 do zakresu 0-255, a następnie wykorzystując bibliotekę ImageMagic (<https://imagemagick.org/index.php>) zapisuje scenę końcową w formacie PNG.

2.3. Przykłady użycia

Aby uruchomić program *tracer* należy użyć komendy podanej w listingu 2.3

Listing 2.3: Komenda do uruchomienia programu *tracer*

```
./tracer [-s/--samples]=X [-d/--depth]=Y Z
```

Program ten przyjmuje następujące argumenty:

- X - Argumenty opcjonalny. Podawany zawsze z flagą *-s* (krótka) lub *-samples* (długa). Parametr ten określa ilość próbek (ile razy promień zostanie wysłany) dla pojedynczego piksela. Wartość ta musi być z zakresu <4, 65535>. W przypadku pominięcia tego argumentu zostanie użyta domyślna wartość 40.
- Y - Argument opcjonalny. Podawany zawsze z flagą *-d* (krótka) lub *-depth* (długa). Parametr ten określa ile razy maksymalnie pojedynczy promień może się odbić. Wartość ta musi być z zakresu <3, 255>. W przypadku pominięcia tego argumentu zostanie użyta domyślna wartość 10.

- **Z** - Argument wymagany. Podawany bez flagi, zawsze musi być ostatni. Argument ten jest ścieżką do pliku Json definiującego generowaną scenę. Plik ten musi być zgodny z konwencją opisaną w sekcji 2.2.1.

. Kolejność podania argumentów **X** oraz **Y** nie znaczenia.

Aplikację *tracer* można także wywołać podając argument *-help* (listing: 2.4) w celu wydrukowania aktualnej instrukcji obsługi programu.

Listing 2.4: Komenda do wyświetlenia pomocy programu *tracer*

```
./tracer --help
```

2.4. Wyjście programu

Jako wynik działania programu tworzone są dwa pliki:

- *nameDXSY.png* - obraz przedstawiający końcową scenę, gdzie *name* oznacza nazwę sceny (nazwę wejściowego pliku *.json*), *X* oznacza maksymalną głębokość odbić użytą przy renderowaniu sceny, a *Y* ilość próbek,
- *benchmark.txt* - plik, w którym zapisywane są pomiary wykonane w trakcie działania programu.

Dane w pliku *benchmark.txt* zapisywane w formacie przedstawionym na listingu 2.5

Listing 2.5: Dane zapisywane w pliku *benchmark.txt*

```
nameDXSY;HH:MM:SS.milisekundy
```

- *nameDXSY* - Identyfikator renderu tworzony zgodnie z konwencją tworzenia nazwy dla pliku *.png*,
- *HH:MM:SS.milisekundy* - Czas trwania renderu.

Gdy plik *benchmark.txt* istnieje, wynik nowego renderu zostanie do niego dopisany. W przeciwnym wypadku program *tracer* utworzy nowy plik.

Rozdział 3

Środowisko testowe i metodologia

3.1. Środowisko

3.1.1. Sprzęt

Testy zostały podzielone na dwóch platformach sprzętowych: budżetowa oraz wydajna. Podział ten ma na celu porównanie zachowania aplikacji na platformnych osiągnięciach.

CPU	AMD Ryzen 7 5800X3D (8 rdzeni, 16 wątków) @4.50 GHz
GPU	NVIDIA GeForce RTX 3070 (8 GB VRAM)
RAM	32GB DDR4 - 3600 MHz

Tab. 3.1: Specyfikacje platformy testowej I (wydajna)

CPU	Intel Core i5-11300H (4 rdzenie, 8 wątków) @4.40 GHz
GPU	NVIDIA GeForce GTX 1650 Mobile (Max-Q) (4 GB VRAM)
RAM	16GB DDR4 - 3200 MHz

Tab. 3.2: Specyfikacje platformy testowej I
I (budżetowa)

W przypadku II platformy testowej wszystkie testy zostały przeprowadzone na zasilaniu sieciowym.

3.1.2. Oprogramowanie

Języki programowania

Program *tracer* został napisany w języku C++ z wykorzystaniem standardu C20 oraz frameworku CUDA w wersji 12.4.

System operacyjny

Wszystkie testy zostały przeprowadzane na systemie Kubuntu 24.04 o następujących wersjach głównych komponentów systemu:

KDE Plasma Version	5.27.11
KDE Frameworks Version	5.115.0
Qt Version	5.15.13
Kernel Version	6.8.0-31-generic (64-bit)
Graphics Platform	Wayland

Tab. 3.3: Wersje oprogramowania systemu operacyjnego

Kompilatory

Do kompilacji testowanych programów użyte zostały następujące wersje kompilatorów i narzędzi wspomagających:

CMake	3.28.3
g++	(Ubuntu 13.2.0-23ubuntu4) 13.2.0
nvcc	12.4, V12.4.131 (Build cuda_12.4.r12.4compiler.34097967_0)

Tab. 3.4: Wersje kompilatorów i oprogramowania wspomagającego

3.2. Metodologia

Aby uniknąć niechcianych wycieków pamięci środowisko testowe jest czyszczone i resetowane pomiędzy każdą grupą testów. W celu zabezpieczenia się przed niechcianym wykorzystaniem zmian zapisanych w cache'u z innej wersji testowej aplikacji folder *build*, w którym przechowywane są pliki tymczasowe, jest usuwany przy każdej zmianie testowanej wersji aplikacji.

Maksymalny czas na wykonanie renderu to 24 godziny. Jeżeli test nie zakończy się przed limitem tego czasu zostanie on przerywany oraz oznaczany jako K/C. Wszystkie następne testy, dla tej samej głębokości oraz sceny, nie są już testowane oraz są także oznaczane jako K/C. Przykładowo jeżeli render dla sceny *Labirynt* dla głębokości 10 oraz ilości próbek 1000 przekroczył limit czasu, testy dla ilości próbek 2000, 5000 i 10000 zostaną także oznaczane jako K/C. Zmiana parametru maksymalnej głębokości bądź testowanej sceny znosi to ograniczenie.

3.2.1. Automatyzacja testów

Wszystkie testy zostały wykonane za pomocą skryptu *test_automation.py*. Uruchamia on program dla wszystkich kombinacji parametrów testowych, pilnuje nie przekroczenia limitów czasowych oraz zapisuje wyniki do pliku tekstowego. Użycie go bez argumentów skutkuje przetestowaniem następujących parametrów:

Scena	scenes/maze.json, scenes/mirrors.json, scenes/spheres.json
Głębokość	10
Ilość próbek	40, 80, 200, 400, 1000, 2000, 5000, 10000

Tab. 3.5: Domyślne parametry testowe

Oprócz tego możliwe jest uruchomienie programu podając własne argumenty.

Listing 3.1: Uruchomienie skryptu *test_automation.py* z własnymi argumentami.

```
python3 test_automation.py -o -s=X -d=Y -p=PATH
```

Argumenty w listingu 3.1 oznaczają:

- `-o` - Flaga pozwalająca na uruchomienie pojedynczego testu. Bez niej pozostałe argumenty zostaną zignorowane,
- `-s` - Zmienna określająca ilość próbek dla której uruchomiony zostanie program *tracer*,
- `-d` - Zmienna określająca maksymalną ilość odbić,
- `-p` - Zmienna określająca ścieżkę do sceny, którą chcemy przetestować.

Uwaga! Skrypt *test_automation.py* nie sprawdza poprawności wprowadzonych argumentów.

Program ten dopisuje dane do pliku *benchmark.txt* tworzonego przez program *tracer*. Przykład tego pliku został przedstawiony listingu 3.2. Kolejną są to argumenty:

- Identyfikator test (Taki sam jak nazwa odpowiadającego mu pliku *.png*),
- Czas trwania renderu,
- Zużyta pamięć systemowa,
- Zużyta pamięć GPU (Wartość ta jest dopisywana tylko w przypadku aplikacji używających biblioteki CUDA). W przeciwnym wypadku wartość nie jest brana pod uwagę.

Listing 3.2: Plik *benchmark.txt*

```
mazeD10S10000;21:01:28.848;135.64;96.0
mazeD10S20000;42:09:12.99;135.52;96.0
```

Uwaga: Uruchomienie testów poprzez skrypt testowy skutkuje usunięciem poprzednich wyników (pliku *benchmark.txt*).

3.2.2. Pomiar czasu

W program *tracer* została wbudowana funkcja pomiaru czasu z wykorzystaniem biblioteki standardowej *chrono*.

Listing 3.3: Funkcja pomiaru czasu

```
1  const std::vector<Vec3> measure(const std::string& id, std::function<Vec3<
    ↪ std::vector<Vec3>()> testable)
2  {
3      std::cout << "Begining render..." << std::endl;
4      auto start = high_resolution_clock::now(); // Marker czasu w ↪
    ↪ momencie rozpoczęcia testu.
5      const auto image = testable(); // Testowana funkcjonalność.
6      auto stop = high_resolution_clock::now(); // Marker czasu w ↪
    ↪ momencie zakończenia testu.
7      std::cout << " - Done" << std::endl;
8      const auto time = getTimeString(duration_cast<milliseconds>(stop -↪
    ↪ start).count()); // Obliczenie długości trwania czasu oraz ↪
    ↪ konwersja do formatu czytelnego dla ludzi.
9      std::cout << "Render took: " << time << std::endl;
10     saveBenchmark(id, time); // Zapisanie wyniku pomiaru do pliku.
11     return image;
12 }
```

Pomiary czasu wykonywane zostały tylko dla funkcji render z pominięciem funkcji wczytujących konfigurację sceny oraz zapisu wygenerowanego obrazu. Pozwala nam to wyeliminować niechciane części programu (zewnętrzne biblioteki) nad, które nie są kontrolowane przez autora. Czas mierzony jest z dokładnością do milisekund w formacie *HH:MM:SS.milisekundy*. Wynik pomiaru czasu jest drukowany na standardowym wyjściu (*cout*) oraz zapisywany do pliku *benchmark.txt*.

3.2.3. Pomiar pamięci

Pomiary pamięci wykonywane są automatycznie przez skrypt *test_automation.py* w trakcie trwania każdego testu. Pomiar zużycia pamięci GPU wykonywany jest tylko w przypadku wersji programu wykorzystujących kartę graficzną.

Listing 3.4: Przygotowanie i egzekucja testu w skrypcie *test_automation.py*

```

1 def run_test(scene, depth, sample):
2     manager = Manager()
3     result = manager.dict() # Zmienna do komunikacji z subprocessem, w
    ↪ którym uruchamiany jest test.
4     gpu_memory = subprocess.Popen(NVIDIA_SMI_CMD, shell=False, stdout=
    ↪ subprocess.PIPE) # Uruchomienie pomiaru zużycia pamięci GPU
    ↪ (Nie występuje w wersji tylko CPU)
5     p = Process(target=test, args=(scene, depth, sample, result)) #
    ↪ Przygotowanie procesu na którym wykonywany jest test
6     p.start() # Uruchomienie testu.
7     p.join() # Oczekiwanie na zakończenie testu (Timeout albo sukces)
    ↪
8     gpu_memory.kill() # Zakończenie pomiaru pamięci GPU.
9
10    if not result[0]: # Sprawdzenie czy wystąpił timeout.
11        write_timeout(scene, sample, depth)
12        return True
13
14    mib_used = result[1] # Pamięć zużyta przez CPU
15    print("CPU Memory used: " + mib_used + " MiB")
16    with open(BENCHMARK_FILE, 'a') as file:
17        file.write(mib_used + ";\n")
18
19    output = gpu_memory.stdout.read()
20    peak_gpu = get_gpu_usage(output) # Pamięć zużyta przez GPU.
21    print("GPU Memory used: " + peak_gpu + " MiB\n")
22    with open(BENCHMARK_FILE, 'a') as file:
23        file.write(peak_gpu + "\n")
24    return False

```

Pamięć systemowa

Aby uzyskać jak najdokładniejszy wynik pomiaru skrypt *test_automation.py* izoluje każde uruchomienie programu *tracer* do własnego subprocessu. Następnie wykorzystując bibliotekę *resource* (listing 3.5) sprawdzane jest maksymalne zużycie pamięci procesu dziecka (czyli tylko dla procesu testowanej instancji).

Listing 3.5: Pomiar zużycia pamięci systemowej

```

1  proc = subprocess.Popen([EXECUTABLE + " -d="+str(depth) + " -s="+str(↵
    ↵ sample) + " " + scene], shell=True) # Przygotowanie procesu ↵
    ↵ programu tracer
2  try:
3      proc.wait(timeout=TIMEOUT) # Oczekiwanie na zakończenie działania↵
    ↵ procesu (lub timeout).
4  except subprocess.TimeoutExpired: # Obsługa timeout'u.
5      print("\nTimeout! Skipping further execution for scene/depth ↵
    ↵ combination.\n")
6      kill_dangling_process()
7      result[0] = False
8
9  maxrss = (resource.getrusage(resource.RUSAGE_CHILDREN).ru_maxrss) # ↵
    ↵ Sprawdzenie maksymalnego zużycia pamięci przez subprocess.
10 result[1] = to_mebibytes(maxrss) # Konwersja do bardziej czytelnego ↵
    ↵ formatu.

```

Pamięć GPU

Pomiar pamięci użytej przez GPU wykonywany jest z wykorzystaniem programu *nvidia-smi*. Jest on uruchamiany z parametrami przedstawionymi na listingu 3.6. Pomiar zużycia pamięci GPU zapisywany jest do bufora co 500 milisekund. Po zakończeniu pracy testowanej instancji programu *tracer* dane zapisane w buforze są przetwarzane i jako wynik zapisywane jest maksymalne odnotowane zużycie pamięci GPU.

Listing 3.6: Parametry dla *nvidia-smi*

```

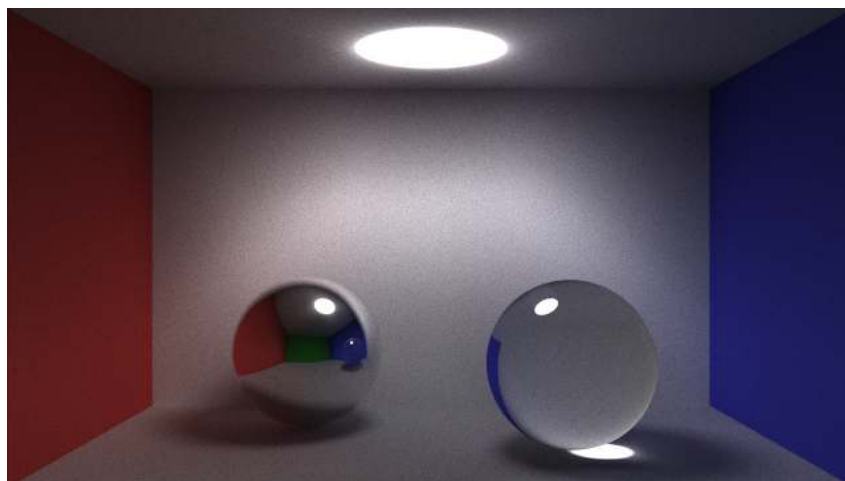
1  NVIDIA_SMI_CMD = ["nvidia-smi", "--query-compute-apps=name,used_memory↵
    ↵ ", "--format=csv", "-lms=500"]

```

3.2.4. Sceny testowe

W celu przetestowania programu *tracer* zostały stworzone trzy sceny testowe, których celem jest zbadanie działania algorytmu w różnych warunkach.

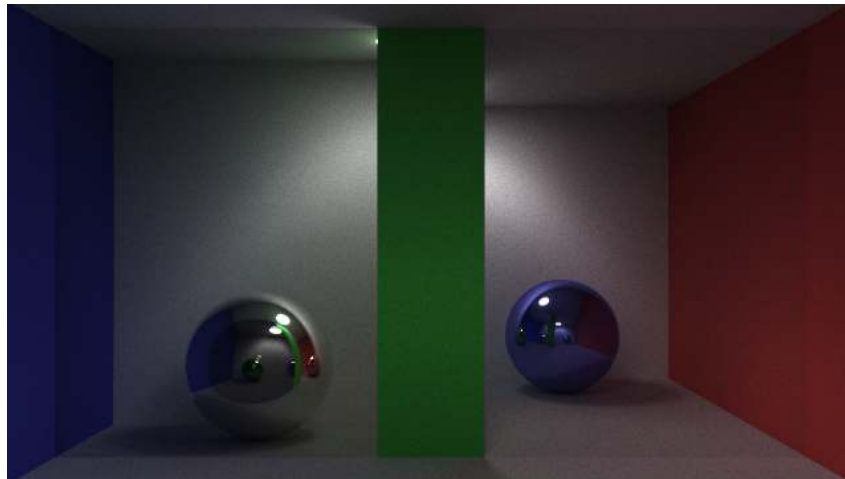
Cornell Box

Rys. 3.1: Scena Cornell Box wygenerowana przez *tracer(cuda-3)*, 5000 próbek, głębokość 10

Przedstawiona na rysunku 3.1 jest scena typu "Cornell Box". Jest to standardowa scena wykorzystywana przy badaniu dokładności oprogramowania renderującego grafikę. W tym konkretnym przypadku scena ta została zbudowana z dwóch kul i sześciu płaszczyzn. Na tej scenie znajdują się wszystkie elementy obsługiwane przez program *tracer*:

- Obiekty typu *diffuse* -rozpraszające światło,
- Obiekty typu *specular* - lustra,
- Obiekty typu *refractive* -symulujące szkło.

Lustra



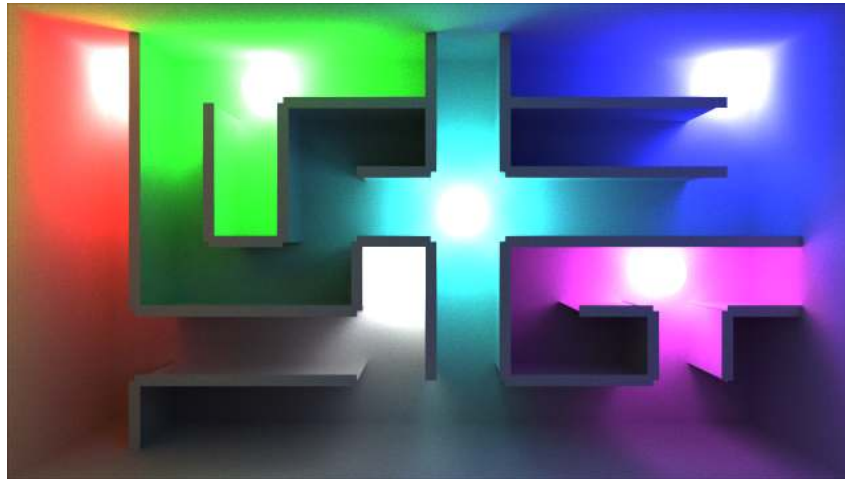
Rys. 3.2: Scena Lustra wygenerowana przez *tracer(cuda-3)*, 5000 próbek, głębokość 10

Na pierwszy rzut oka scena "Lustra" jest bardzo podobna do opisanej wcześniej sceny "Cornell Box". Została ona jednak w znaczny sposób rozbudowana. Główną cechą tej sceny jest podział na dwa komory, tylną i przednią. Kamera (z której widzimy wygenerowany obraz) została umieszczona w tylnej komorze i jest skierowana w kierunku ściany dzielącej tą scenę na dwie części. Ściana dzieląca tą scenę na dwie komory także sama jest podzielona na trzy różne części. Od lewej strony najpierw znajduje się płaszczyzna typu *refractive* (szkło), która pozwala nam zajrzeć do sąsiedniej komory. Następnie na środku sceny widzimy zielony pas, który blokuje nam bezpośredni widok na źródło światła. Ostatnim elementem jest płaszczyzna typu *specular* (lustro) - Widzimy w niej obraz, który znajduje się za kamerą.

Wszystkie te elementy pozwalają nam zbadać zachowanie programu *tracer* w następujących sytuacjach:

- Brak bezpośrednio widocznego na kamerze źródła światła,
- Oświetlenie obiektów po wielokrotnym odbiciu w obiektach obijających światło,
- Załamanie światła w szybie.

Labirynt



Rys. 3.3: Scena Labirynt wygenerowana przez *tracer(cuda-3)*, 5000 próbek, głębokość 10

Ostatnia scena testowa została zbudowana w celu maksymalnego obciążenia programu *tracer*. Na scenie znajduje się 58 różnych obiektów, w tym 5 źródeł światła. Za kamerą znajduje się ściana typu *specular* - Jej celem jest zapobiegnięcie uciekania odbitych promieni w "próżnię". Kolejnym elementem występującym na tej scenie jest mieszanie się oświetlenia o różnych kolorach. Możemy to zaobserwować na w lewym górnym rogu na rysunku 3.3, gdzie czerwone i zielone światło mieszają oświetlając elementy sceny żółtym światłem.

Ostatnim ważnym elementem tej sceny są "zakamarki labiryntu", czyli elementy, które nie są bezpośrednio oświetlone przez źródło światła. Na tej scenie możemy zaobserwować jak badane modele wpływają na wygląd tych elementów sceny.

Rozdział 4

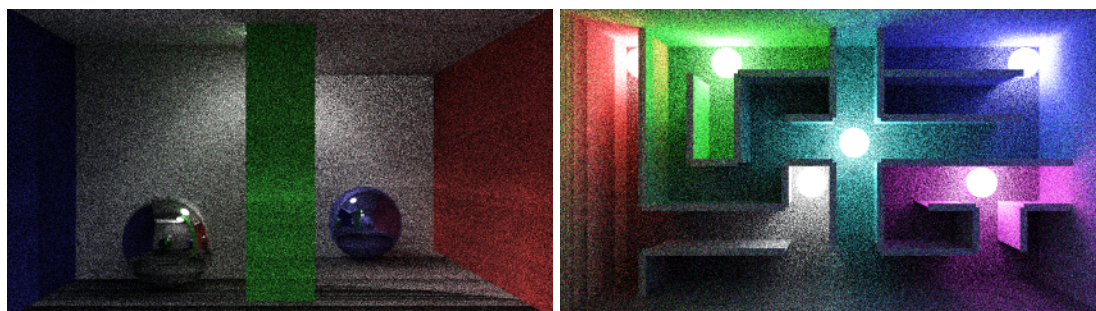
Rozwój aplikacji

4.1. Napotkane problemy

W trakcie rozwoju aplikacji testowej autor natknął się na kilka problemów, które zostały za wartość opisania w tej pracy.

4.1.1. Znikające światło

Problem ten objawia się ciemnymi pasmami pojawiającymi się na losowych elementach sceny. Był on szczególnie widoczny na scenie "Lustra" oraz "Labirynt" jak przedstawione na rysunku 4.1



Rys. 4.1: Problem znikającego światła widoczny na scenie "Labirynt" i "Lustra" (40 próbek, głębokość 10).

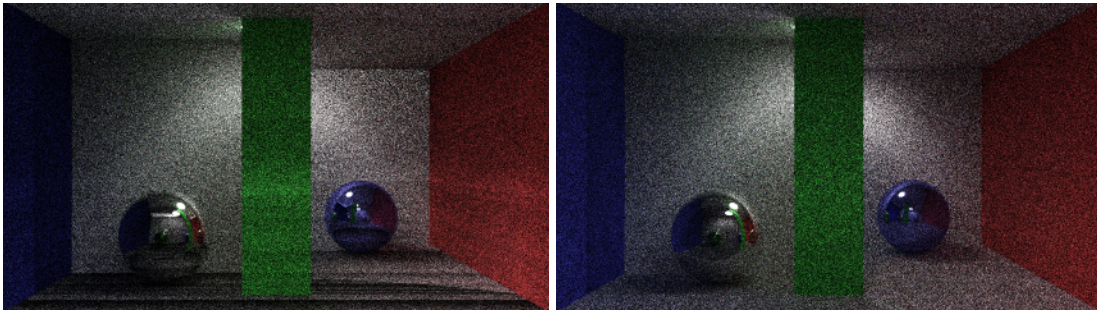
Problem ten także występuje na scenie "Cornell Box" jest jednak znacznie bardziej subtelny - został on przez to początkowo pominięty.

Po dokładnym zbadaniu tego zjawiska okazało się, że problem występuje w kodzie płaszczyzny. Problematyczna linijka została przedstawiona na listingu 4.1. System detekcji kolizji promienia z płaszczyzną sprawdza odległość od punktu początkowego promienia (poprzedniego punktu trafienia) do każdej z płaszczyzn znajdujących się na scenie - w tym potencjalnie od płaszczyzny od której promień się odbija. Założenie w takim wypadku było, że dana kolizja zostanie odrzucona jako, że obliczona odległość będzie wynosiła 0. Niestety jednak ze względu na naturę obliczeń zmiennoprzecinkowych czasami odległość ta była minimalnie większa od 0. W takim wypadku promień utykał w miejscu cały czas obliczając tą samą kolizję - nie wnosząc tym samym żadnej informacji do renderowanej sceny tworząc ciemne smugi widoczne na przykładowych obrazach. Zmiana problematycznej linijki poprzez wprowadzenie marginesu (odrzuć każdą kolizję, której odległość jest mniejsza od $1e-4$ (0.0001)) rozwiązało problem.

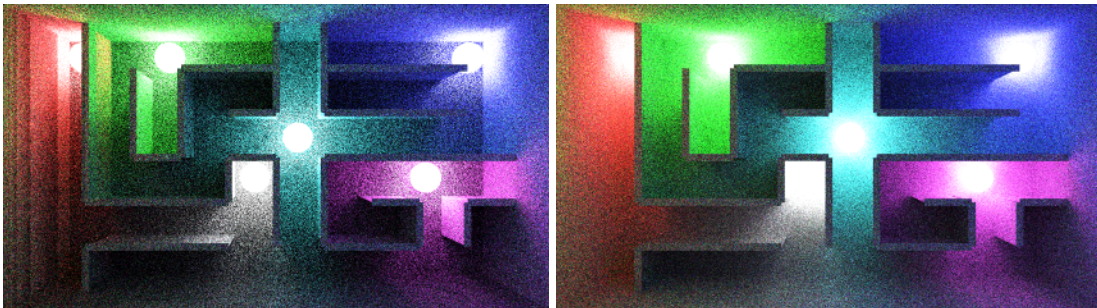
Listing 4.1: Linijka powodująca błąd w systemie oświetlenia płaszczyzny

```
1 if (distance <= 0.0) return 0.0;
```

Porównanie obu scen, na których występował problem zostało przedstawione na rysunkach 4.2 oraz 4.3.



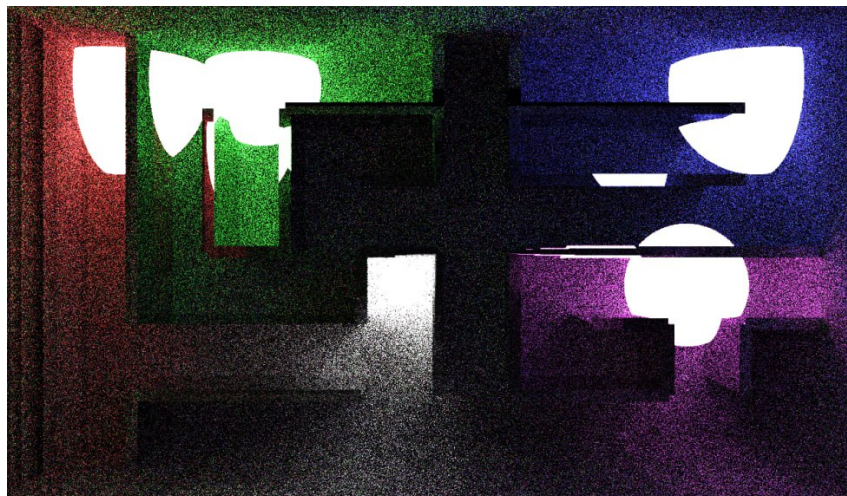
Rys. 4.2: Scena "Lustra" przed i po rozwiązaniu problemu (40 próbek, głębokość 10).



Rys. 4.3: Scena "Labirynt" przed i po rozwiązaniu problemu (40 próbek, głębokość 10).

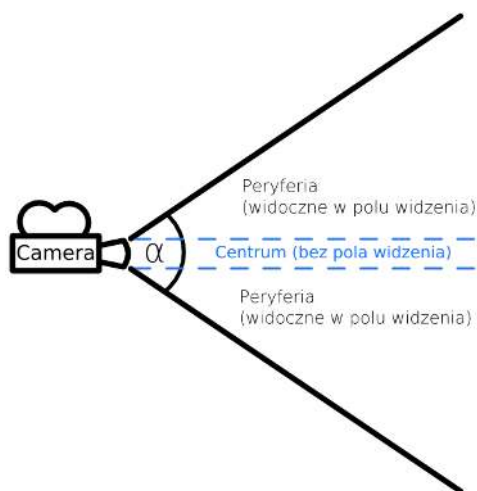
4.1.2. Pole widzenia

Kolejny problem został wykryty podczas tworzenia sceny "Labirynt". Objawiał się on niestabilnym rozmiarem renderowanych obiektów typu kula. Problem ten został przedstawiony na rysunku 4.4.



Rys. 4.4: Wczesna wersja sceny labirynt, widoczny problem z wielkością renderowanych kul (10 próbek, głębokość 4).

Problem ten zablokował rozwój aplikacji na dłuższy czas. Znalazienie jego źródła okazało się nie trywialne. Początkowo podejrzewany był problem z tym jak tworzony jest obiekt typu kula, bądź błąd w module odpowiedzialnym za wczytywanie danych sceny z pliku Json. Problem okazał się jednak z systemem emitującym promień dla każdego piksela kamery, a dokładnie z elementem odpowiedzialnym za symulowanie pola widzenia (rysunek 4.5).



Rys. 4.5: Diagram przedstawiający koncept pola widzenia.

Pole widzenia jest konieczne do nadania "głębi" generowanemu obrazowi. W przeciwnym wypadku końcowy obraz byłby tylko płaskim rzutem sceny na płaszczyznę - efekt końcowy wyglądał by nienaturalnie.

Aby uzyskać efekt pola widzenia generowany promień spojrzenia (kierunek promienia emitowanego dla danego piksela obrazu) jest zakrzywiany na zewnątrz (od środka obrazu). Kod odpowiedzialny za tą funkcję został przedstawiony na listingu 4.2

Listing 4.2: Linijka powodująca błąd w systemie emisji promieni.

```
1  const auto gaze = direction + vecX*stepX*FOV_SCALE + vecZ*stepZ*↵
    ↪ FOV_SCALE;
```

Zmienne wykorzystane w tym kodzie kolejno oznaczają:

- *direction* - kierunek spojrzenia kamery - prostopadły do płaszczyzny generowanego obrazu,
- *vecX* - odległość pomiędzy pojedynczymi pikselami w osi X (poziomo),
- *stepX* - ilość kroków, które należy wykonać od centrum kamery (środkowy punkt obrazu) do pozycji piksela, z którego emitowany jest promień (w osi X),
- *vecZ* - odległość pomiędzy pojedynczymi pikselami w osi Z (pionowo),
- *stepZ* - ilość kroków, które należy wykonać od centrum kamery (środkowy punkt obrazu) do pozycji piksela, z którego emitowany jest promień (w osi Z),
- *FOV_SCALE* - stała wynosząca 0.0009 (odchylenie promienia spojrzenia jest minimalne).

Co nie zostało wzięte pod uwagę tworząc ten kod, jest fakt, że nowo powstały wektor spojrzenia *gaze* zmienia także swoją długość. Jest to zmiana minimalna, nie mająca znaczenia dla obiektów znajdujących się w centrum obrazu. Jednak jednym z elementów, który biorę pod uwagę przy obliczaniu pola widzenia jest odległość danego piksela od centrum kamery - im bliżej krawędzi tym efekt jest silniejszy.

Dla obiektów typu kula długość promienia spojrzenia jest wykorzystywana do obliczenia odległości od punktu początkowego danego wektora do obiektu znajdującego się na scenie. Ponieważ długość promienia nie była stała (dłuższa im dalej od centrum obrazu). Tworzyło to iluzję,

w której kule znajdowały się bliżej kamery niż miało to miejsce w rzeczywistości. Objawiało się to zjawiskiem widocznym na rysunku 4.4. Im kula znajduje się dalej od centrum obrazu tym wydaje się ona większa.

Aby naprawić ten problem wystarczyło znormalizować promień spojrzenia (gwarantuje to, że jego długość zawsze wynosi 1).

Listing 4.3: Poprawiony kod obliczający zakrzywienie promienia spojrzenia.

```
1  const auto gaze = (direction + vecX*stepX*FOV_SCALE + vecZ*stepZ*↵
    ↵ FOV_SCALE).norm();
```

4.2. Wersje GPU

4.2.1. Wersja CUDA-1

Pierwsza implementacja na GPU wykorzystuje jeden kernel o maksymalnej ilości wątków wynoszącej 484 (blok rozmiaru 22 na 22). Jest to spowodowane ograniczeniami pamięci. Przy bardziej skomplikowanych scenach dokończenie renderu nie było możliwe ze względu na zbyt duże zużycie pamięci. Wynika to z faktu naiwnego przeniesienia kodu z wersji CPU - każdy wątek działał na własnej kopii tablicy obiektów znajdujących się na scenie. Generowany obraz podzielony jest na równe prostokąty x na y pikseli, które są przydzielane do każdego z wątków.

W przypadku biblioteki CUDA wykorzystanie rekurencji powoduje nie zdefiniowane zachowania. Kompilator nie jest w stanie określić złożoności kodu (ile razy dana funkcja zostanie wywołana) w związku z czym nie może on też za-alokować odpowiedniej ilości pamięci potrzebnej na wykonanie programu (ilość ta musi być znana w czasie kompilacji). Aby uniknąć tego problemu główna pętla programu została rozbita na trzy osobne funkcje. Dwie pierwsze obsługują odpowiednio odbicie pierwsze i drugie. Podział ten wynika z faktu, że odbicia te mają największy wpływ na końcowy stan piksela - wydzielenie ich do osobnych funkcji pozwala na łatwiejszą kontrolę zachowania dla danego odbicia (Przykładowo możemy generować więcej niż jeden nowy promień przy odbiciu). Pozostałe odbicia, do maksymalnej głębokości zadanej przez użytkownika, obsługiwane są w pętli w trzeciej funkcji. Poza tymi zmianami wersja ta jest identyczna do implementacji w wersji CPU.

4.2.2. Wersja CUDA-2

Druga wersja aplikacji skupia się na lepszym zarządzaniu pamięcią GPU. Elementy wspólne dla wszystkich wątków są kopiowane do pamięci współdzielonej co zmniejsza zużycie pamięci na pojedynczy wątek. Dodatkowo ilość wątków nie jest zależna od maksymalnej ilości bloków. Dzięki temu możemy uruchomić maksymalnie 1024 bloków po 256 wątków (262144 wątki). Jest to ilość, która znacząco przekracza ilość wątków CUDA dostępnych na najpotężniejszych konsumenckich kartach graficznych dostępnych na rynku (RTX 4090, 18176 rdzeni CUDA, dane z dnia 2024-06-18).

Dodatkowo wprowadzono wykorzystanie atrybutu `__shared__` (Listing: 4.4), który oznacza pamięć, współdzieloną dla każdego bloku wątków. Pamięć ta daje najszybszy dostęp do danych dla wątków znajdujących się w danym bloku, wymaga jednak stworzenia osobnej kopii danych. W naszym przypadku oznacza to, że każdy z 256 wątków ma dostęp do tych samych danych. Niestety korzystanie z tego typu pamięci wymaga znania maksymalnej możliwej wielkości tablicy w czasie kompilacji. Wymusiło to wprowadzenie maksymalnego limitu obiektów znajdujących się na scenie, który w tej wersji wynosi 100.

Listing 4.4: Tworzenie współdzielonego obiektu *tracer*

```

1  __shared__ AObject* sharedObjects[MAX_OBJECT_COUNT];
2  const auto id = threadIdx.x;
3  if (id < imageProperties->objectCount_)
4  {
5      auto assignedObjects = imageProperties->objectCount_/BLOCK_SIZE;
6      const auto overflow = imageProperties->objectCount_ % BLOCK_SIZE;
7      const auto startingPoint = id * assignedObjects + ((id < overflow)↵↵
8          ↵↵ ? id : overflow);
9      assignedObjects = assignedObjects + ((id < overflow) ? 1 : 0);
10     const auto target = startingPoint + assignedObjects;
11     for (auto ii = startingPoint; ii < target; ii++)
12     {
13         sharedObjects[ii] = objects[ii];
14     }
15 }
16 __syncthreads();

```

4.2.3. Wersja CUDA-3

Wersja ta jest bardzo podobna do wersji CUDA-2. Zrezygnowano jednak w niej z wykorzystania atrybutu `__shared__`, zamiast tego umieszczając dane w pamięci GPU dostępnej dla wszystkich wątków. Nie ma konieczności kopiowania danych osobno dla każdego bloku. Ogranicza to zużycie pamięci oraz unika czekania na skopiowanie danych do współdzielonego bloku, kosztem korzystania z minimalnie wolniejszej pamięci.

4.2.4. Wersja CUDA-3.1

Wersja ta stanowi rozwinięcie wersji CUDA-3. Sceny testowe są zbudowane w większości z płaszczyzn w związku z czym ta wersja skupia się głównie na optymalizacji kodu obsługującego obiekty tego typu. Po pierwsze zmianie uległ algorytm znajdujący odległość od punktu początkowego promienia do płaszczyzny. W wersji oryginalnej odległość ta była obliczana z równania zbudowanego na podstawie założenia, że promień trafia płaszczyznę gdy wektor poprowadzony od punktu trafienia do środka płaszczyzny jest prostopadły do normalnej płaszczyzny. W takim wypadku iloczyn skalarny tych wektorów jest równy 0. Na tej podstawie wyprowadzone zostało równanie:

$$d = \frac{n_x \cdot v_x + n_y \cdot v_y + n_z \cdot v_z}{r_x \cdot n_x + r_y \cdot n_y + r_z \cdot n_z} \quad (4.1)$$

4.1: Równanie wykorzystane do obliczenia odległości od punktu startowego promienia do płaszczyzny.

Gdzie:

- d - odległość jak promień musi przebyć żeby trafić w płaszczyznę,
- $n(x, y, z)$ - wektor normalny płaszczyzny,
- $v(x, y, z)$ - wektor od punktu początkowego promienia do środka płaszczyzny,
- $r(x, y, z)$ - wektor wskazujący kierunek padania promienia (kierunek spojrzenia).

Do tej pory w implementacji potrzebne było 5 operacji do obliczenia powyższego równania. Zostały one przedstawione na listingu 4.5

Listing 4.5: Wyznaczenie odległości do płaszczyzny przed optymalizacją.

```

1  const auto refPoint = position_ - ray.origin_; // Wyznaczenie wektora ↵
    ↪ od początku promienia do środka płaszczyzny.
2  const auto top = planeVector_.xx_ * refPoint.xx_ + planeVector_.yy_ * ↵
    ↪ refPoint.yy_ + planeVector_.zz_ * refPoint.zz_; // Obliczenie ↵
    ↪ dzielnej równania.
3  const auto bottom = planeVector_.xx_ * ray.direction_.xx_ + ↵
    ↪ planeVector_.yy_ * ray.direction_.yy_ + planeVector_.zz_ * ray.↵
    ↪ direction_.zz_; // Obliczenie dzielnika równania.
4
5  if (bottom == 0.0) return 0.0; // Sprawdzenie czy nie dzielimy przez 0 ↵
    ↪ (Wektor nie trafia w płaszczyznę).
6  auto distance = top/bottom; // Obliczenie odległości do płaszczyzny

```

W nowej wersji odległość ta jest wyznaczana z równania płaszczyzny. Równanie to może być wyznaczone raz przy tworzeniu płaszczyzny, a następnie wykorzystywane przy każdym obliczaniu kolizji z płaszczyzną.

$$d = \frac{r_x \cdot A + r_y \cdot B + r_z \cdot C}{-(D + A \cdot o_x + B \cdot o_y + C \cdot o_z)} \quad (4.2)$$

4.2: Równanie wykorzystane do obliczenia odległości od punktu startowego promienia do płaszczyzny po optymalizacji.

Gdzie:

- d - odległość jak promień musi przebyć żeby trafić w płaszczyznę,
- A, B, C, D - składniki równania płaszczyzny $A \cdot x + B \cdot y + C \cdot z + D = 0$,
- $r(x, y, z)$ - wektor wskazujący kierunek padania promienia (kierunek spojrzenia).
- $o(x, y, z)$ - początek promienia.

Implementacja tego algorytmu w kodzie została przedstawiona na listingu: 4.6

Listing 4.6: Wyznaczenie odległości do płaszczyzny po optymalizacji.

```

1  const auto bottom = ray.direction_.xx_ * equation_.A_ + ray.↵
    ↪ direction_.yy_ * equation_.B_ + ray.direction_.zz_ * equation_.↵
    ↪ .C_; // Wyznaczenie dzielnej równania.
2  if (bottom == 0.0) return 0.0; // Sprawdzenie czy nie dzielimy przez 0 ↵
    ↪ (Wektor nie trafia w płaszczyznę).
3
4  const auto top = -(equation_.D_ + equation_.A_ * ray.origin_.xx_ + ↵
    ↪ equation_.B_ * ray.origin_.yy_ + equation_.C_ * ray.origin_.↵
    ↪ zz_); // Obliczenie dzielnika równania.
5  auto distance = top/bottom; // Obliczenie odległości do płaszczyzny

```

Dzięki obliczeniu parametrów równania płaszczyzny wcześniej (Przy wczytywaniu obiektu) możemy zaoszczędzić jedną operację w porównaniu do oryginalnej wersji.

Kolejna optymalizacja zmieniła działanie kodu odpowiedzialnego za sprawdzenie czy punkt, znaleziony na płaszczyźnie, znajduje się wewnątrz prostokąta ograniczającego daną płaszczyznę. Oryginalnie sprawdzenie odbywało się poprzez obliczenie odległości pomiędzy punktem na płaszczyźnie, a bokiem prostokąta. Przykładowo: jeżeli odległość punktu od prawego boku i lewego boku jest równa odległości pomiędzy tymi bokami punkt znajduje się na płaszczyźnie, natomiast jeżeli odległość ta jest większa dany punkt znajduje się poza interesującym nas obszarem. Funkcja odpowiedzialna za obliczenie odległości pomiędzy punktem, a bokiem

prostokąta została przedstawiona na listingu 4.7. Algorytm ten działa poprzez wyznaczenie punktu, który tworzy wektor prostopadły w stosunku do prostej ograniczającej dany bok prostokąta. Punkt ten jest wyznaczany z zależności, że iloczyn skalarny pomiędzy wektorem leżącym na danym boku, a wektorem utworzonym pomiędzy punktem trafienia w płaszczyznę, a rzutem ortogonalnym tego punktu na prostą ograniczającą dany bok, jest równy 0. Możemy w tym celu wykorzystać przedstawione wcześniej równanie 4.1.

Listing 4.7: Funkcja obliczająca odległość punktu do boku prostokąta przed optymalizacją.

```

1  // origin - Punkt zaczepienia wektora równoległego do prostej ↵
   ↵ ograniczającej dany bok prostokąta.
2  // border - Wektor leżący równoległy do prostej ograniczającej dany ↵
   ↵ bok prostokąta.
3  // impact - Punkt kolizji z płaszczyzną.
4  __device__ double distanceToBorder(const Vec3& origin, const Vec3& ↵
   ↵ border, const Vec3& impact)
5  {
6      const auto refPoint = impact - origin; // Wektor pomiędzy punktem↵
   ↵ trafienia w płaszczyznę, a punktem zaczepienia wektora okre↵
   ↵ ślającego bok prostokąta.
7      const auto top = border.xx_ * refPoint.xx_ + border.yy_ * refPoint↵
   ↵ .yy_ + border.zz_ * refPoint.zz_; // Obliczenie dzielnika r↵
   ↵ ównania.
8      const auto bottom = pow(border.xx_, 2) + pow(border.yy_, 2) + pow(↵
   ↵ border.zz_, 2); // Wyznaczenie dzielnej równania.
9
10     if (bottom == 0.0) return 0.0; // Punkt trafienia znajduje się na↵
   ↵ boku, odległość 0.
11
12     const auto distance = top / bottom; // Obliczenie odległość od ↵
   ↵ punktu zaczepienia wektora boku do punktu, który tworzy ↵
   ↵ prostopadły wektor pomiędzy punktem trafienia, a bokiem.
13     return (origin + border * distance).distance(impact); // ↵
   ↵ Obliczenie odległości pomiędzy punktem trafienia, a ↵
   ↵ prostopadłym rzutem punktu na wektorze boku.
14 }
```

Nowa wersja została napisana na podstawie równania opisanego na stronie Wolfram MathWorld [8]. Wykorzystanie tej metody pozwoliło na pominięcie obliczania rzutu ortogonalnego na bok prostokąta i znaczne zmniejszenie ilości operacji potrzebnych do obliczenia odległości od punktu trafienia do danego boku. Nowy kod został przedstawiony na listingu 4.8

Listing 4.8: Funkcja obliczająca odległość punktu do boku prostokąta po optymalizacji.

```

1 // start - Początek wektora ograniczającego dany bok, jeden z rogów ↵
  ↵ prostokąta.
2 // vec - Wektor od jednego rogu prostokąta do drugiego.
3 // impact - Punkt kolizji z płaszczyzną.
4 // length - Długość boku.
5 __device__ inline double distanceToBorder(const Vec3& start, Vec3 vec, ↵
  ↵ const Vec3& impact, const double& length)
6 {
7     Vec3 toImpact = start - impact; // Wektor od punktu kolizji z pł↵
  ↵ aszczyzną do punktu znajdującego się w rogu prostokąta. (W ↵
  ↵ tym punkcie zaczepiony jest wektor reprezentujący bok ↵
  ↵ prostokąta).
8     const auto top = (vec % toImpact).length(); // Obliczenie długoś↵
  ↵ ci wektora powstałego po obliczeniu iloczynu wektorowego ↵
  ↵ pomiędzy wektorem obliczonym liniijkę wyżej, a wektorem leż↵
  ↵ acym na boku prostokąta.
9     return top / length; // Obliczenie długości pomiędzy punktem, ↵
  ↵ bokiem prostokąta.
10 }

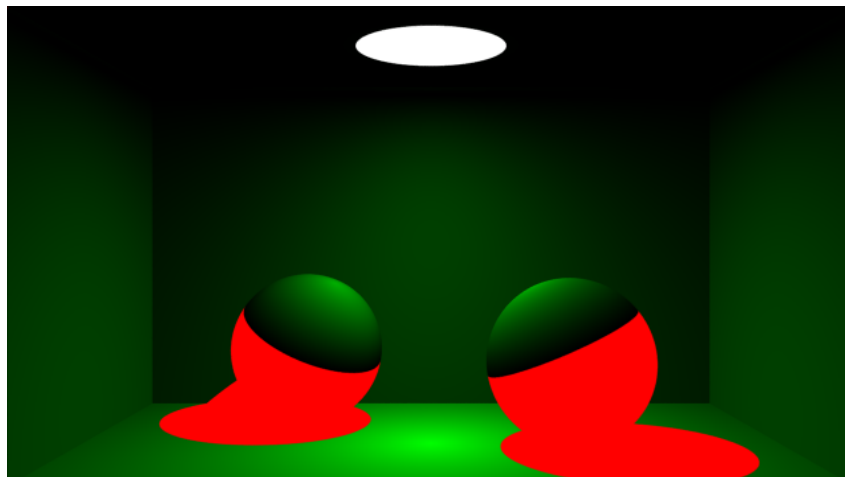
```

4.2.5. Wersja CUDA-3.2

W tej wersji dodano podział pomiędzy źródłami światła, a zwykłymi obiektami (w kodzie określonymi jako *props*). Ułatwia to wprowadzenie różnych zachowań dla obiektów emitujących światło. W tym wypadku wprowadzono ograniczenie, które przerywa śledzenie promienia po trafieniu w źródło światła. Jest to jedyna różnica względem wersji *CUDA-3.1*.

4.2.6. Wersja CUDA-3.3

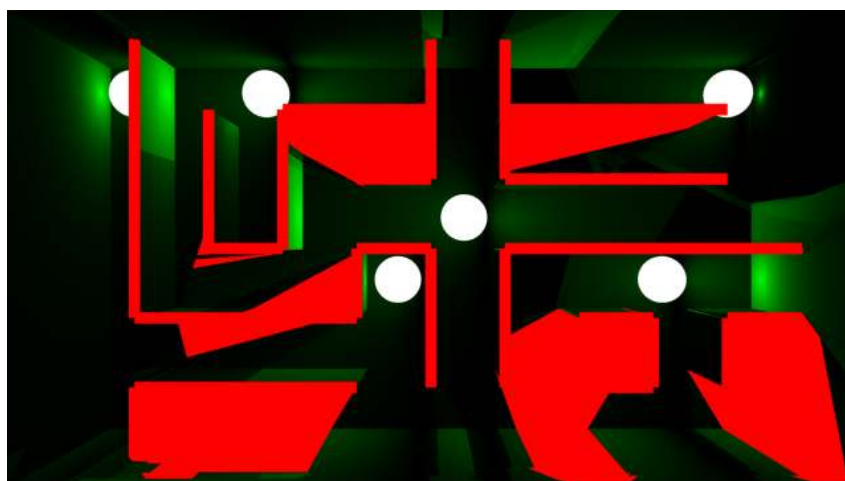
Wersja ta implementuje model hybrydowy oświetlenia zgodnie z opisem przedstawionym w sekcji 1.4.2. Jako za podstawę tej wykorzystano wersję *CUDA-3.2*. Głównym elementem modelu hybrydowego jest funkcja *findLight*, która sprawdza czy dany punkt jest oświetlony i zwraca informacje o natężeniu oświetlenia na podstawie konta pomiędzy źródłem światła, trafionym obiektem. Na rysunkach 4.6, 4.7 i 4.8 przedstawiono wynik działania tej funkcji przedstawiony w formie mapy kolorów. Białym kolorem oznaczono źródło światła, na czerwono elementy sceny, które nie są oświetlone, a kolorem zielonym elementy oświetlone. W przypadku przykładu dla sceny "Labirynt" widzimy ograniczenie tego modelu, gdzie algorytm szukania światła nie bierze pod uwagę przezroczystości obiektu.



Rys. 4.6: Stopień oświetlenia sceny "Cornell Box" przez źródło światła.



Rys. 4.7: Stopień oświetlenia sceny "Lustra" przez źródło światła.



Rys. 4.8: Stopień oświetlenia sceny "Cornell Box" przez źródło światła.

4.2.7. Wersja CUDA-3.4

Wersja ta skupia się na optymalizacji systemu szukania oświetlenia wprowadzonego w wersji *CUDA-3.4*. Dla wszystkich rekwizytów (*props*) znajdujących się na scenie szukany jest centralny

punkt. Dla tego punktu następnie skrajne punkty źródła światła są sortowane od najbliższych do takich znajdujących się najdalej. Przykładowo dla źródła światła typu kula jest to 6 punktów znajdujących się najdalej względem osi sceny. Idea według, której tworzone są te ekstrema została przedstawiona na listingu 4.9

Listing 4.9: Funkcja obliczająca odległość punktu do boku prostokąta po optymalizacji.

```

1  __device__ Sphere(double radius, Vec3 position, Vec3 emission, Vec3 ↵
    ↵ color, EReflectionType reflection)
2      : AObject(position, emission, color, reflection, SPHERE_EXTREMS)
3      , radius_(radius)
4  {
5      extremes_ = (Vec3*)malloc(sizeof(Vec3) * SPHERE_EXTREMS);
6      extremes_[0] = position_ + Vec3(1, 0, 0) * radius_;
7      extremes_[1] = position_ - Vec3(1, 0, 0) * radius_;
8      extremes_[2] = position_ + Vec3(0, 1, 0) * radius_;
9      extremes_[3] = position_ - Vec3(0, 1, 0) * radius_;
10     extremes_[4] = position_ + Vec3(0, 0, 1) * radius_;
11     extremes_[5] = position_ - Vec3(0, 0, 1) * radius_;
12 }
```

Motywacją stojącą za wprowadzeniem tej zmiany jest usprawnienie działania funkcji, która sprawdza czy dany punkt jest oświetlony przez źródło światła. Sortowanie ekstremów względem źródła światła ma sprawić, że dla jak największej ilości obiektów sprawdzenie kolizji z źródłem światła zaczyna się od ekstremum znajdującego się najbliżej (Największa szansa, że nie jest zablokowane przez inne obiekty).

4.2.8. Wersja CUDA-3.5

Wersja ta implementuje rozszerzony model oświetlenia zgodnie z założeniami przedstawionymi w sekcji 1.4.1. W wersji CUDA-3.5 domyślnie maksymalna liczba odbić została ustawiona na 10. Dodatkowo dla odbić powyżej dwóch pierwszych wprowadzono warunek wczesnego zakończenia szukania światła jeżeli obecnie trafiony oraz poprzedni element sceny są obiektami rozpraszającymi światło. Pozwala to na ograniczenie maksymalnej koniecznej do obliczenia głębokości odbić.

4.2.9. Wersja CUDA-3.6

W tej wersji wprowadzono drobną zmianę w warunku zakończenia odbić dla obiektów typu *diffuse* w stosunku do wersji *CUDA-3.5*. Dodano dodatkowe sprawdzenie do warunku wcześniejszego zakończenia odbić. Odbicie, na którym kończy się działanie aplikacji nie może znajdować się w cieniu (musi być oświetlone przez źródło światła). Zmiana ta została przedstawiona na listingu 4.10 Motywacją do jej stworzenia było poprawienie jakości obrazu w częściach sceny, która nie są bezpośrednio oświetlone przez źródło światła.

Listing 4.10: Warunek wcześniejszego przerwania algorytmu śledzenia promienia dla obiektów rozpraszających światło (Wersja *CUDA-3.6*).

```

1  if ((previosType == Diffuse) and (object->getReflectionType() == ↵
    ↵ Diffuse) and not (pixel == Vec3()))
```

4.2.10. Wersja CUDA-4

Czwarta wersja aplikacji eksperymentuje z inną metodą podziału pracy pomiędzy GPU, a CPU. W tej wersji aplikacji, karta graficzna jest odpowiedzialna za obliczanie próbek dla danego

punktu na obrazie. CPU liczy średnią z wyników obliczonych przez GPU oraz obsługuje główną pętlę programu. Dzięki wykorzystaniu wielowątkowości na CPU program ten odpala wiele kerneli CUDA na raz. W tym podziale pojedynczy kernel CUDA wykonuje mniej pracy.

Celem tej wersji było zwiększenie wykorzystania CPU, które we wcześniejszych wersjach oprogramowania pozostaje niewykorzystany w trakcie gdy GPU wykonuje obliczenia). Jako, że ilość pracy wykonywana przez CPU się nie zmienia (rozdzielczość obrazu testowego pozostaje bez zmian) wyniki tego algorytmu powinny dobrze skalować się wraz ze wzrostem ilości próbek.

4.2.11. Wersja CUDA-5

Wersja ta stanowi wariację na temat zmian wprowadzony w wersji *CUDA-4* przesuując znowu więcej pracy w kierunku GPU. W tej wersji kernel uruchomiony na karcie graficznej obsługuje pojedynczy wiersz obrazu. Dzięki temu musimy wykonać mniej transferów pamięci pomiędzy system, a pamięcią GPU. Główną motywacją za powstaniem tej wersji było zbadanie, który podział pracy pomiędzy GPU, a CPU będzie się spisywał lepiej.

Rozdział 5

Testy aplikacji

5.1. Wersja CPU - punkt odniesienia

Jako punkt odniesienia do dalszych testów stworzona została wersja testowa aplikacji działająca na CPU, która została przetestowana na I i II platformie testowej.

5.1.1. Platforma I

Wyniki czasowe dla I platformy zostały przedstawione w tabelach 5.1 oraz 5.2.

Scena	Ilość próbek			
	40	80	200	400
Cornell Box	00:00:34.133	00:01:13.430	00:02:55.697	00:05:43.358
Lustra	00:01:02.837	00:02:06.880	00:04:51.809	00:09:29.411
Labirynt	00:01:18.306	00:02:36.515	00:06:30.393	00:13:12.198

Tab. 5.1: Czas potrzebny na wygenerowanie obrazu, wersja CPU, platforma I, głębokość 10, próbki 40 - 400

Dla testu dla 40 próbek scena "Labirynt" oraz scena "Lustra" są do siebie zbliżone. Różnica między nimi wynosi tylko 15.469 sekundy (24.62% więcej czasu). Wraz z wzrostem ilości próbek różnica ta wzrasta. Dla 400 próbek wynosi ona już 3 minuty i 42.787 sekundy (39.13% więcej czasu). Jest to różnica większa o 14.51%.

Scena "Cornell Box" jak najprostsza z testowanych jest szybsza o 45.68% w stosunku do sceny "Lustra" dla 40 próbek. Przy 400 próbkach różnica ta maleje do 39.70%.

Scena	Ilość próbek			
	1000	2000	5000	10000
Cornell Box	00:14:50.694	00:29:02.57	01:12:05.94	02:26:13.64
Lustra	00:23:59.38	00:47:04.66	02:01:29.94	03:59:41.31
Labirynt	00:33:08.37	01:04:28.29	02:43:52.17	05:26:06.18

Tab. 5.2: Czas potrzebny na wygenerowanie obrazu, wersja CPU, platforma I, głębokość 10, próbki 1000 - 10000

Analizując wyniki z przedziału 1000 - 1000 widzimy stabilizację różnic pomiędzy testowanymi scenami. Dla sceny "Cornell Box" od testu dla 200 próbek różnica procentowa w czasie w stosunku do sceny "Lustra" utrzymuje się na poziomie około 40%. Dla sceny "Labi-

rynt"maksymalna różnica w stosunku do sceny "Lustra"wyniosła 39.13%. Miało to miejsce przy teście dla 400 próbek. Dla kolejnych testów różnica ta spada i utrzymuje się na poziomie około 36%. Zmiany w czasie renderu sceny (przyjmując scenę "Lustra"jako 100%) zostały przedstawione w tabeli 5.3

Scena	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
Cornell Box	54.32%	57.87%	60.21%	60.30%	61.88%	61.69%	59.34%	61.01%
Lustra	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
Labirynt	124.62%	123.36%	133.78%	139.13%	138.14%	136.95%	134.87%	136.05%

Tab. 5.3: Czas renderu względem sceny "Lustra", platforma I, głębokość 10.

Aplikacja zużywa średnio 57.32 MiB pamięci. Głównym czynnikiem zużywającym pamięć w aplikacji jest tablica przechowująca obliczone piksele obrazu. Jest ona zawsze taka sama niezależnie od testowanej sceny. Różnice w ilości obiektów są zbyt małe żeby wpłynąć w znaczący sposób na końcowy wynik. Dokładne dane dotyczące zużycia pamięci zostały przedstawione w tabeli 5.4.

Scena	Ilość próbek								Średnie
	40	80	200	400	1000	2000	5000	10000	
Cornell Box	57.14	57.45	57.45	57.20	57.15	57.15	57.02	57.02	57.22
Lustra	57.27	57.39	57.32	57.00	57.64	57.60	57.37	57.27	57.37
Labirynt	57.14	57.14	57.02	57.52	57.55	57.34	57.40	57.14	57.30

Tab. 5.4: Zużycie pamięci systemowej, wersja CPU, platforma I, dane w MiB

5.1.2. Platforma II

Wyniki czasowe dla platformy II zostały przedstawione w tabelach 5.5 oraz 5.6

Scena	Ilość próbek			
	40	80	200	400
Cornell Box	00:00:54.78	00:01:49.99	00:04:37.79	00:09:12.61
Lustra	00:01:34.12	00:03:10.40	00:07:53.94	00:15:52.27
Labirynt	00:02:28.90	00:05:04.54	00:12:49.29	00:25:35.79

Tab. 5.5: Czas potrzebny na wygenerowanie obrazu, wersja CPU, platforma II, głębokość 10, próbki 40 - 400

Analizując wyniki z przedziału 40 - 400 możemy zaobserwować, że różnica pomiędzy długością renderów jest stała i oscyluje w okolicach 42% szybciej dla sceny "Cornell Box" oraz 60% wolniej dla sceny "Labirynt"(przyjmując czasy dla sceny "Lustra"jako 100%). Różnica pomiędzy sceną "Cornell Box"i "Lustra"jest zbliżona do różnicy, którą obserwowaliśmy dla I platformy. Dla 40 próbek jest ona o 4% większa, jednak już dla 200 próbek widzimy odwrócenie się tej zależności gdzie różnica jest teraz mniejsza o 2%. W przypadku sceny "Labirynt"różnica ta jest znacznie większa - rendery dla tej sceny są o około 60% wolniejsze od sceny "Lustra". W stosunku do platformy I różnica ta jest większa o 35% dla 40 próbek ale już tylko o 20% dla 400 próbek.

Scena	Ilość próbek			
	1000	2000	5000	10000
Cornell Box	00:23:05.74	00:46:17.23	01:55:07.57	04:21:15.40
Lustra	00:39:24.28	01:19:29.66	03:18:39.52	07:24:46.30
Labirynt	01:04:02.63	02:07:38.37	05:21:00.25	11:36:22.17

Tab. 5.6: Czas potrzebny na wygenerowanie obrazu, wersja CPU platforma II, głębokość 10, próbki 1000 - 10000

Dla danych z przedziału 1000 - 10000 widzimy kontynuację trendów, które zaobserwowaliśmy poprzednio. Różnica pomiędzy sceną "Cornell Box", a sceną "Lustra" utrzymuje się na poziomie 42%. Scena "Labirynt" jest w dalszym ciągu wolniejsza o około 60% w porównaniu do sceny "Lustra". Różnice te zostały szczegółowo przedstawione w tabeli 5.7. Prawdopodobnym powodem dla, którego w przypadku platformy II obserwujemy stałą procentową różnicę pomiędzy scenami jest różnica w chłodzeniu pomiędzy oboma komputerami testowymi. Chłodzenie wykorzystane w przypadku platformy I posiada większą masę, przez co pozwala na dłuższą pracę CPU w trybie "turbo". Efekt ten jest pomijalny w przypadku dłuższych renderów, natomiast wpływa na wynik testów dla małej ilości próbek. Dzieje się tak ponieważ w ich przypadku okres zwiększonej wydajności CPU stanowi istotny czas długości całego testu.

Scena	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
Cornell Box	58.20%	57.76%	58.61%	58.03%	58.61%	58.23%	57.95%	58.74%
Lustra	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
Labirynt	158.20%	159.94%	162.32%	161.28%	162.53%	160.56%	161.59%	156.57%

Tab. 5.7: Czas renderu względem sceny "Lustra", platforma II, głębokość 10.

Zużycie pamięci jest minimalne mniejsze od platformy I. Jest to jednak bardzo mała różnica, wynosząca mniej niż 0.5 MiB pomiędzy obiema platformami. 5.8.

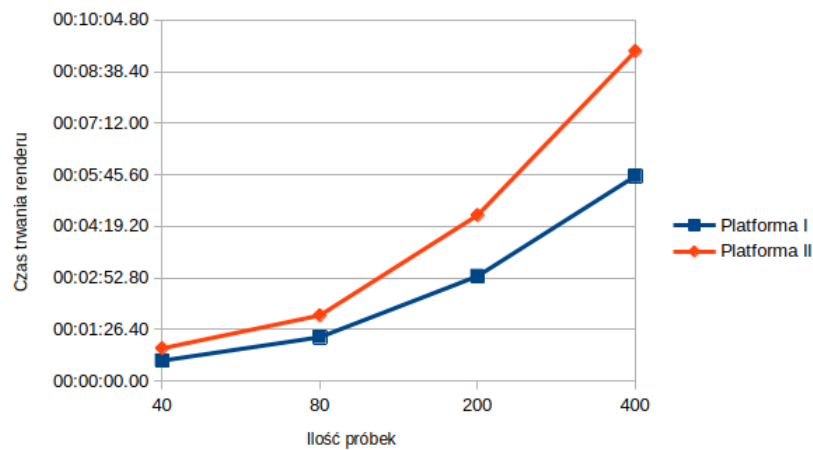
Scena	Ilość próbek								Średnie
	40	80	200	400	1000	2000	5000	10000	
Cornell Box	56.90	56.91	56.91	56.89	56.90	56.92	56.91	56.64	56.87
Lustra	56.79	56.92	56.67	56.89	56.78	57.02	56.89	56.77	56.85
Labirynt	56.89	56.77	56.89	56.77	56.89	56.89	56.77	56.77	56.84

Tab. 5.8: Zużycie pamięci systemowej, wersja CPU, platforma II, dane w MiB

5.1.3. Porównanie

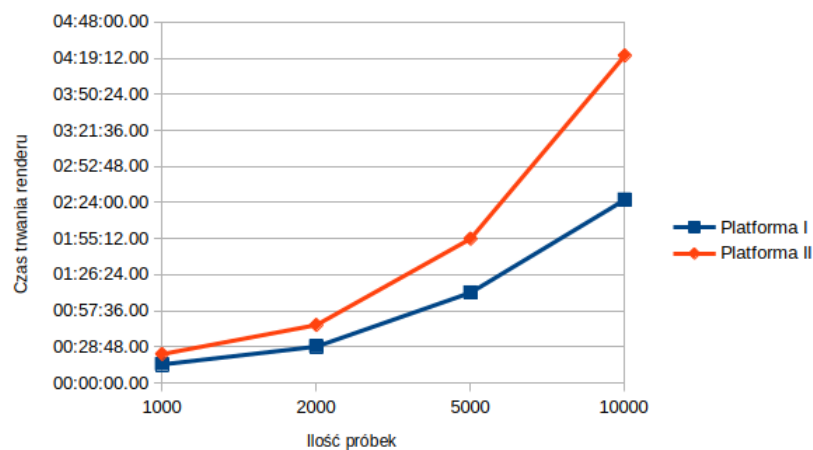
Cornell Box

W przedziale 40 - 400 Platforma I jest około o 60% wolniejsza niż platforma II.



Rys. 5.1: Czas renderu sceny "Cornell Box" pomiędzy platformą I i II, głębokość 10, próbki 40 - 400.

W przedziale 1000 - 1000 różnica pomiędzy obiema wersjami utrzymuje się dalej na poziomie 60% oprócz testu dla 10000 próbek gdzie różnica ta wzrosła 78.66%. Dane szczegółowe zostały przedstawione w tabeli 5.9. Średnio scena "Cornell Box" jest wolniejsza o 60.33% na platformie II.



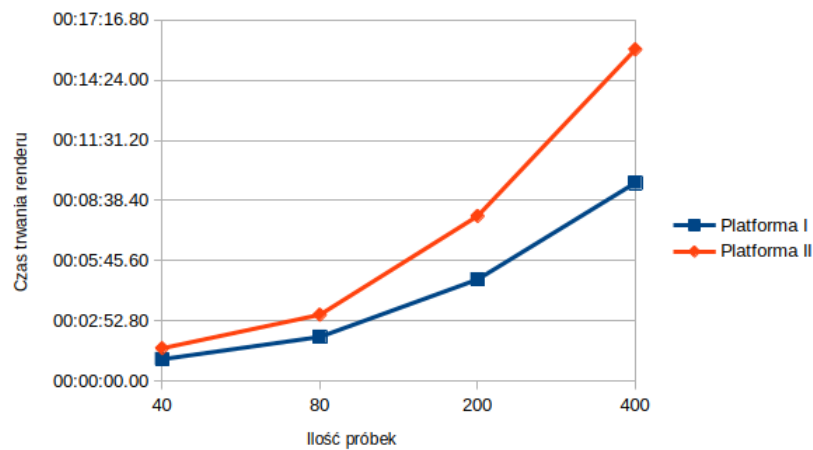
Rys. 5.2: Czas renderu sceny "Cornell Box" pomiędzy platformą I i II, głębokość 10, próbki 40 - 400.

Platforma	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
I	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
II	160.49%	149.79%	158.11%	160.94%	155.58%	159.38%	159.68%	178.66%

Tab. 5.9: Porównanie czasu renderu sceny "Cornell Box" pomiędzy platformami, głębokość 10.

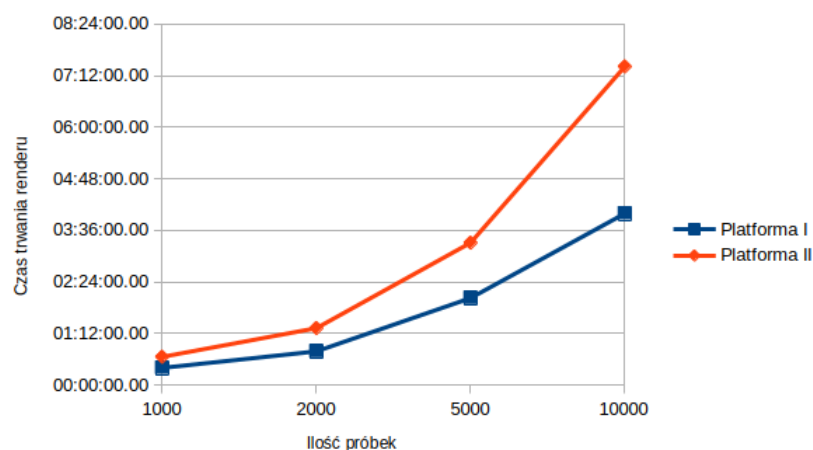
Lustra

Dla tej sceny w przedziale 40 - 400, widzimy ciut inne zachowanie niż miało to miejsce w przypadku sceny "Cornell Box". Platforma II jest wolniejsza o około 50% dla próbek 40 i 80. Wartość wzrasta kolejno dla test dla 200 próbek do 62.41% i dla 400 do 67.24%.



Rys. 5.3: Czas renderu sceny "Lustra" pomiędzy platformą I i II, głębokość 10, próbki 40 - 400.

W przedziale 1000 - 10000 widzimy kolejne wahania w różnicy pomiędzy oboma platformami. Dla próbek od 1000 do 2000 wartość ta oscyluje przedziale 66% $\pm$ 2.5%. Podobnie jak w przypadku sceny "Cornell Box" widzimy zwiększenie się tej różnicy dla testu dla 10000 próbek. W tym przypadku platforma II jest wolniejsza o 85.56%. Jest to wartość większa o około 20% w porównaniu do wcześniejszych wyników. Szczegółowe dane znajdują się w tabeli 5.10. Średnio scena "Lustra" jest wolniejsza o 63.96% w porównaniu do platformy I.



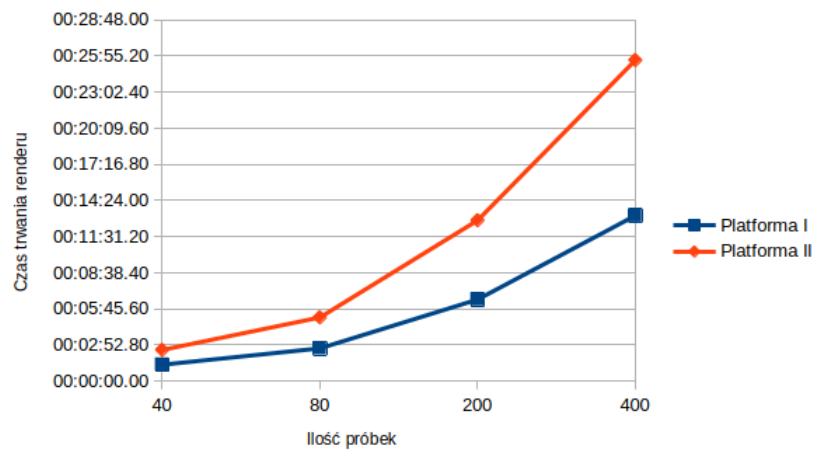
Rys. 5.4: Czas renderu sceny "Lustra" pomiędzy platformą I i II, głębokość 10, próbki 1000 - 10000.

Platforma	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
I	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
II	149.78%	150.06%	162.41%	167.24%	164.26%	168.86%	163.51%	185.56%

Tab. 5.10: Porównanie czasu renderu sceny "Lustra" pomiędzy platformami, głębokość 10.

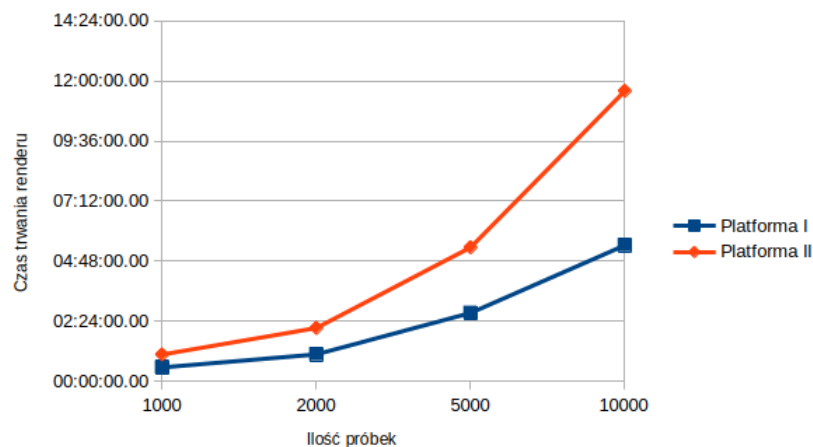
Labirynt

W przypadku sceny labirynt różnica pomiędzy oboma platformami jest znacznie większa. W przedziale 40 - 400 już dla 40 próbek platforma II jest wolniejsza 90.15%. Dla kolejnych próbek nie możemy wskazać trendu wzrostowego tak jak miało to miejsce w przypadku poprzednich scen - wartości wahają się od około 94% do 98%.



Rys. 5.5: Czas renderu sceny "Labirynt" pomiędzy platformą I i II, głębokość 10, próbki 40 - 400.

W przedziale 1000 - 10000 widzimy podobne zachowanie dla próbek 1000, 2000 oraz 5000. Podobnie jak w pozostałych przypadkach dla 10000 próbek widzimy znaczne spowolnienie platformy II - w tym przypadku aż o 113.54%. Oznacza to, że render ten był ponad dwukrotnie dłuższy od tego wykonanego na I platformie. Średnie opóźnienie dla platformy II w stosunku do platformy I dla sceny "Labirynt" wyniosło 97.04%.



Rys. 5.6: Czas renderu sceny "Labirynt" pomiędzy platformą I i II, głębokość 10, próbki 1000 - 10000.

Platforma	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
I	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
II	190.15%	194.58%	197.06%	193.86%	193.26%	197.98%	195.89%	213.54%

Tab. 5.11: Porównanie czasu renderu sceny "Labirynt" pomiędzy platformami, głębokość 10.

Pamięć

Zużycie pamięci na platformie II jest średnio niższe o 0.45 MiB. Jest to niewielka różnica, która nie wpływa w znaczący sposób na wnioski badań przeprowadzonych w tej pracy.

5.2. Wybór algorytmu

Pierwszą fazą implementacji programu *tracer* było znalezienie najlepszego sposobu na przeniesienie algorytmu path-tracingu z wersji CPU na kod CUDA. Powstało w tym celu kilka wersji, które eksperymentowały z różnymi metodami zrównoleglenia pracy na GPU. Są to wersje o dużych numerach bez części dziesiętnych. Przykładowo CUDA-X, ale nie CUDA-X.Y. Zmiany pomiędzy poszczególnymi wersjami zostały dokładnie opisane w rozdziale 4.

5.2.1. CUDA-1

Cornell Box

Platforma I

Pierwsza "naiwna" implementacja algorytmu path-tracingu na GPU osiąga znacznie gorsze wyniki w porównaniu do wersji CPU. W przedziale 40 - 400 (tabela 5.12) wersja CUDA-1 jest wolniejsza o 1719.75%.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:00:34.133	00:01:13.430	00:02:55.697	00:05:43.358
CUDA-1	00:10:39.461	00:21:12.804	00:53:15.242	01:46:04.491

Tab. 5.12: Czas potrzebny na wygenerowanie sceny Cornell Box, platforma I, głębokość 10, próbki 40 - 400.

Dla danych w przedziale 1000 - 10000 (tabela 5.13) opóźnienie utrzymuje się na poziomie 1720.50% w stosunku do wersji CPU. Średnio wersja CUDA-1 jest wolniejsza o 1720.07%. Test dla 10000 próbek przekroczył limit maksymalnej długości testu.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:14:50.694	00:29:02.565	01:12:05.939	02:26:13.639
CUDA-1	04:25:27.748	08:53:30.385	22:03:56.158	K/C

Tab. 5.13: Czas potrzebny na wygenerowanie sceny Cornell Box, platforma I, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
CUDA-1	1873.44%	1733.36%	1818.61%	1853.60%	1788.24%	1836.97%	1836.28%	K/C

Tab. 5.14: Porównanie czasu renderu sceny "Cornell Box" pomiędzy wersją CPU, a CUDA-1, platforma I, głębokość 10.

Oprócz tego dla wersji CUDA-1 widzimy średnio większe o 73.97 MiB zużycie pamięci systemowej w stosunku do wersji CPU. Wynika to z konieczności tworzenia lokalnych kopii obiektów, które umożliwiają transfer danych pomiędzy pamięcią systemową, a pamięcią GPU.

Wersja	Ilość próbek								Średnie
	40	80	200	400	1000	2000	5000	10000	
CPU	57.14	57.45	57.45	57.20	57.15	57.15	57.02	57.02	57.20
CUDA-1	131.20	131.32	131.20	131.06	131.19	131.19	131.20	K/C	131.19

Tab. 5.15: Zużycie pamięci systemowej dla sceny "Cornell Box", platforma I, głębokość 10, dane w MiB.

Platforma II

W przypadku II platformy testowej różnica pomiędzy wersją CPU, CUDA-1 jest znacznie mniejsza. W przedziale 40 - 400 (5.16) widzimy, że wersja ta jest wolniejsza o średnio 1315.93% od wersji CPU.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:00:54.779	00:01:49.986	00:04:37.793	00:09:12.610
CUDA-1	00:13:02.515	00:25:51.401	01:05:30.390	02:09:50.819

Tab. 5.16: Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 40 - 400

W przedziale 1000 - 10000 (tabela 5.17) dla przypadków, które zmieściły się w limicie czasowym widzimy podobną różnicę jak w poprzednim przedziale. W przypadku platformy II, aż dwa z testów nie zostały ukończone ze względu na przekroczenie limitu czasu.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	00:23:05.738	00:46:17.228	01:55:07.566	04:21:15.404
CUDA-1	05:25:04.348	10:44:49.838	K/C	K/C

Tab. 5.17: Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 1000 - 10000

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
CUDA-1	1428.47%	1410.49%	1414.88%	1409.82%	1407.50%	1393.11%	K/C	K/C

Tab. 5.18: Porównanie czasu renderu sceny "Cornell Box" pomiędzy wersją CPU, a CUDA-1, platforma I, głębokość 10.

Różnica w zużyciu pamięci pomiędzy obiema wersjami wynosi 78.84 MiB. Jest to wartość większa o 4.87 MiB niż miało to miejsce w przypadku platformy I.

Wersja	Ilość próbek								Średnie
	40	80	200	400	1000	2000	5000	10000	
CPU	56.90	56.91	56.91	56.89	56.9	56.92	56.91	56.64	56.87
CUDA-1	135.55	135.56	135.80	135.69	135.81	135.84	K/C	K/C	135.71

Tab. 5.19: Zużycie pamięci systemowej dla sceny "Cornell Box", platforma II, głębokość 10, dane w MiB.

Lustra**Platforma I**

W przypadku sceny "Lustra" różnica w stosunku do wersji CUDA-1 jest mniejsza i wynosi "tylko" średnio 1546.41%. W dalszym ciągu jest to nie akceptowalna różnica, która wyklucza wersję CUDA-1 z dalszego procesu rozwoju aplikacji.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:01:02.837	00:02:06.880	00:04:51.809	00:09:29.411
CUDA-1	00:16:17.443	00:32:28.124	01:20:57.789	02:42:06.125

Tab. 5.20: Czas potrzebny na wygenerowanie sceny "Lustra", platforma II, głębokość 10, próbki 40 - 400.

Wyróżniają się testy dla ilości próbek 40 oraz 80. Średnie opóźnienie względem wersji CPU dla nich wynosi tylko 1445.46%. Jest wartość mniejsza o 100.95% w stosunku do średniej dla tej sceny. Na razie nie wiemy czy jest to zjawisko unikalne dla tej sceny czy coś powtarzalnego. Testy dla 5000 i 10000 próbek przekroczyły maksymalny czas egzekucji testu.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	00:23:59.375	00:47:04.660	02:01:29.943	03:59:41.314
CUDA-1	06:45:47.870	13:31:47.230	K/C	K/C

Tab. 5.21: Czas potrzebny na wygenerowanie sceny "Lustra", platforma II, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
CUDA-1	1555.52%	1535.41%	1664.72%	1708.10%	1691.56%	1724.36%	K/C	K/C

Tab. 5.22: Porównanie czasu renderu sceny "Lustra" pomiędzy wersją CPU, a CUDA-1, platforma I, głębokość 10.

Różnica w zużyciu pamięci pomiędzy obiema wersjami jest podobna jak w przypadku sceny "Cornell Box" i wynosi średnio 73.78 MiB. Szczegóły zostały przedstawione w tabeli 5.23.

Wersja	Ilość próbek								Średnie
	40	80	200	400	1000	2000	5000	10000	
CPU	57.27	57.39	57.32	57.00	57.64	57.60	57.37	57.27	57.36
CUDA-1	131.07	131.14	131.19	131.06	131.06	131.31	K/C	K/C	131.14

Tab. 5.23: Zużycie pamięci systemowej dla sceny "Labirynt", platforma II, głębokość 10, dane w MiB.

Platforma II

W przypadku drugiej sceny znowu obserwujemy mniejszą różnicę pomiędzy wersją CPU, a CUDA-1 niż miało to miejsce w przypadku platformy I. W tym przypadku testy dla 40 i 80 próbek nie odbiegają od średniego opóźnienia względem wersji CPU.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:01:34.121	00:03:10.404	00:07:53.938	00:15:52.267
CUDA-1	00:20:45.698	00:41:15.666	01:43:26.973	03:26:43.312

Tab. 5.24: Czas potrzebny na wygenerowanie sceny "Lustra", platforma II, głębokość 10, próbki 40 - 400.

Podobnie jak w przypadku platformy I test dla 5000 i 10000 próbek zostały przerwane ze względu na przekroczenie limitu czasu.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	00:39:24.280	01:19:29.657	03:18:39.520	07:24:46.301
CUDA-1	08:36:38.518	17:15:26.334	K/C	K/C

Tab. 5.25: Czas potrzebny na wygenerowanie sceny "Lustra", platforma II, głębokość 10, próbki 40 - 400.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
CUDA-1	1323.52%	1300.24%	1309.65%	1302.50%	1311.12%	1302.53%	K/C	K/C

Tab. 5.26: Porównanie czasu renderu sceny "Lustra" pomiędzy wersją CPU, a CUDA-1, platforma II, głębokość 10.

Wersja	Ilość próbek								Średnie
	40	80	200	400	1000	2000	5000	10000	
CPU	56.79	56.92	56.67	56.89	56.78	57.02	56.89	56.77	56.84
CUDA-1	135.61	135.81	135.56	135.62	135.69	135.93	K/C	K/C	135.70

Tab. 5.27: Zużycie pamięci systemowej dla sceny "Lustra", platforma II, głębokość 10, dane w MiB.

Labirynt

Platforma I

W przypadku sceny "Labirynt" wersja CUDA-1 jest średnio wolniejsza o 3081.88% razy od wersji CPU.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:01:18.306	00:02:36.515	00:06:30.393	00:13:12.198
CUDA-1	00:41:39.731	01:23:11.490	03:29:09.113	06:58:15.730

Tab. 5.28: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 40 - 400.

Dla tej sceny, aż trzy testy nie zostały ukończone ze względu na przekroczenie limitu czasu.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	00:33:08.371	01:04:28.290	02:43:52.173	05:26:06.177
CUDA-1	17:22:26.770	K/C	K/C	K/C

Tab. 5.29: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
CUDA-1	3192.26%	3189.14%	3214.48%	3167.86%	3145.63%	K/C	K/C	K/C

Tab. 5.30: Porównanie czasu renderu sceny "Labirynt" pomiędzy wersją CPU, a CUDA-1, platforma I, głębokość 10.

Podobnie jak w wersji CPU większa scena nie wpływa znacząco na zużyty pamięć systemową.

Wersja	Ilość próbek								Średnie
	40	80	200	400	1000	2000	5000	10000	
CPU	57.14	57.14	57.02	57.52	57.55	57.34	57.40	57.14	57.28
CUDA-1	131.20	131.57	131.32	131.45	131.07	K/C	K/C	K/C	131.32

Tab. 5.31: Zużycie pamięci systemowej dla sceny "Labirynt", platforma I, głębokość 10, dane w MiB.

Platforma II

W przypadku platformy II scena "Labirynt" jest wolniejsza tylko o 1890.60% razy. Jest to wartość znacznie mniejsza niż w przypadku platformy I. Wynik ten jest w dalszym ciągu nieakceptowalny.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:02:28.903	00:05:04.538	00:12:49.285	00:25:35.789
CUDA-1	07:27:58.923	14:55:50.876	04:12:36.830	08:27:45.656

Tab. 5.32: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 40 - 400.

Podobnie jak w przypadku platformy I w przypadku tej sceny aż trzy testy nie zostały zakończone ze względu na przekroczenie limitu czasu.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	01:04:02.628	02:07:38.369	05:21:00.246	11:36:22.172
CUDA-1	21:01:28.848	K/C	K/C	K/C

Tab. 5.33: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
CUDA-1	2037.21%	1992.14%	1970.24%	1983.71%	1969.71%	K/C	K/C	K/C

Tab. 5.34: Porównanie czasu renderu sceny "Labirynt" pomiędzy wersją CPU, a CUDA-1, platforma II, głębokość 10.

Wersja	Ilość próbek								Średnie
	40	80	200	400	1000	2000	5000	10000	
CPU	56.89	56.77	56.89	56.77	56.89	56.89	56.77	56.77	56.83
CUDA-1	135.59	135.49	135.65	135.86	135.64	K/C	K/C	K/C	135.65

Tab. 5.35: Zużycie pamięci systemowej dla sceny "Labirynt", platforma I, głębokość 10, dane w MiB.

Pamięć GPU

Dla wszystkich scen widzimy dokładnie takie same zużycie pamięci GPU. Kompilator CUDA decyduje o alokacji pamięci przez dany kernel w czasie kompilacji. W związku z czym niezależnie od testowanej sceny, czy ilości próbek, zużycie pamięci GPU pozostaje takie samo. Jedyna różnica wynika z wykorzystanej platformy testowej. Ponieważ karta graficzna w platformie I pozwala na uruchomienie większej ilości wątków na raz wiąże się to z większą alokacją pamięci.

Platforma	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
I	96.00	96.00	96.00	96.00	96.00	96.00	96.00	96.00
II	226.00	226.00	226.00	226.00	226.00	226.00	226.00	226.00

Tab. 5.36: Zużycie pamięci GPU, wersja CUDA-1, głębokość 10, dane w MiB.

Dla wszystkich wersji algorytmu, które wykorzystują tylko jeden kernel analiza tych danych zostanie pominięta - chyba, że w wynikach testów zostanie znaleziona jakaś wyróżniająca się anomalia.

Podsumowanie

Wersja CUDA-1 jest zbyt wolna aby być realistycznie wykorzystana do dalszego rozwoju aplikacji *tracer*. W obecnej formie nie nadaje się ona do renderowania bardziej skomplikowanych sceny czy obrazów z większą ilości próbek.

Z przeprowadzonych testów wynika, że zużycie pamięci pomiędzy wersjami pozostaje waha o maksymalnie 4% od wyniku osiągniętego przez wersje CUDA-1. Dla platformy I odchylenie to wyniosło średnio 3.41%, a dla platformy II 2.20%. Schematy zużycia pozostają takie same pomiędzy testowanymi wersjami. Utrzymują one zużycie pamięci na tym samym poziomie niezależnie od testowanej sceny czy ilości próbek. Różnice nie przekraczają kilku Mebibajtów pomiędzy wersjami. Zdaniem autora jest, że różnice te nie wnoszą istotnych informacji do przedstawionych porównań. W związku z tym dane dotyczą zużycia pamięci systemowej zostały pominięte dla kolejnych wersji. Wyjątkiem tutaj są wersje CUDA-4 oraz CUDA-5, które ze względu na zmienioną architekturę znacznie odbiegają od średniej.

5.2.2. CUDA-2

Cornell Box

Platforma I

Wersja CUDA-2 stanowi znaczny krok naprzód w stosunku do wersji CUDA-1. Optymalizacja zużycia pamięci oraz zmiana architektury krenela, która pozwala na uruchomienie większej ilości wątków na raz, zmniejszyły w znaczny sposób różnicę do wersji CPU. W przypadku sceny "Cornell Box" wersja CUDA-1 jest średnio wolniejsza o 48.98%.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:00:34.133	00:01:13.430	00:02:55.697	00:05:43.358
CUDA-2	00:00:52.370	00:01:44.601	00:04:20.714	00:08:41.299

Tab. 5.37: Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma I, głębokość 10, próbki 40 - 400.

Wprowadzone zmiany sprawiły, że wszystkie testy dla sceny "Cornell Box" kończą się w limicie czasu.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	00:14:50.694	00:29:02.565	01:12:05.939	02:26:13.639
CUDA-2	00:21:46.488	00:43:32.127	01:48:16.161	03:37:54.187

Tab. 5.38: Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma I, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
CUDA-2	153.43%	142.45%	148.39%	151.82%	146.68%	149.90%	150.17%	149.02%

Tab. 5.39: Porównanie czasu renderu sceny "Cornell Box" pomiędzy wersją CPU, a CUDA-2, platforma I, głębokość 10.

Platforma II

Dla platformy II także widzimy znaczną poprawę w stosunku do wersji CUDA-1. Jest to różnica większa niż w przypadku platformy I. Wersja CUDA-2 jest wolniejsza o 238.31% od wersji CPU.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:00:54.779	00:01:49.986	00:04:37.793	00:09:12.610
CUDA-2	00:03:09.279	00:06:18.100	00:15:45.800	00:31:28.771

Tab. 5.40: Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 40 - 400.

Tutaj także wszystkie testy zostały zakończone w limicie czasu.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	00:23:05.738	00:46:17.228	01:55:07.566	04:21:15.404
CUDA-2	01:18:39.535	02:37:15.590	06:33:22.794	13:39:27.251

Tab. 5.41: Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
CUDA-2	345.53%	343.76%	340.47%	341.79%	340.58%	339.75%	341.69%	313.66%

Tab. 5.42: Porównanie czasu renderu sceny "Cornell Box" pomiędzy wersją CPU, a CUDA-II, platforma II, głębokość 10.

Lustra

Platforma I

W przypadku sceny "Lustra" wersja CUDA-2 jest wolniejsza 70.68% w stosunku do wersji CPU.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:01:02.837	00:02:06.880	00:04:51.809	00:09:29.411
CUDA-2	00:01:41.583	00:03:21.587	00:08:21.801	00:16:42.346

Tab. 5.43: Czas potrzebny na wygenerowanie sceny "Lustra", platforma I, głębokość 10, próbki 40 - 400.

Podobnie jak w przypadku sceny "Cornell Box" tutaj też wszystkie rendery zostały zakończone w limicie czasu.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	00:23:59.375	00:47:04.660	02:01:29.943	03:59:41.314
CUDA-2	00:41:43.377	01:23:26.870	03:28:26.800	06:57:32.682

Tab. 5.44: Czas potrzebny na wygenerowanie sceny "Lustra", platforma I, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
CUDA-2	161.66%	158.88%	171.96%	176.03%	173.92%	177.26%	171.56%	174.20%

Tab. 5.45: Porównanie czasu renderu sceny "Lustra" pomiędzy wersją CPU, a CUDA-2, platforma I, głębokość 10.

Platforma II

Ponownie dla sceny "Lustra" różnica w stosunku do wersji CPU jest większa niż dla platformy I. W tym przypadku wersja CUDA-2 jest średnio wolniejsza 284.58%.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:01:34.121	00:03:10.404	00:07:53.938	00:15:52.267
CUDA-2	00:06:05.580	00:12:10.881	00:30:26.874	01:00:51.422

Tab. 5.46: Czas potrzebny na wygenerowanie sceny "Lustra", platforma II, głębokość 10, próbki 40 - 400.

Niestety dla tej sceny znowu trafiamy na test, który nie mieści się w granicach czasowych.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	00:39:24.280	01:19:29.657	03:18:39.520	07:24:46.301
CUDA-2	02:32:03.859	05:03:58.456	12:40:05.755	K/C

Tab. 5.47: Czas potrzebny na wygenerowanie sceny "Lustra", platforma II, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
CUDA-2	388.42%	383.87%	385.47%	383.44%	385.90%	382.38%	382.61%	K/C

Tab. 5.48: Porównanie czasu renderu sceny "Lustra" pomiędzy wersją CPU, a CUDA-2, platforma Z, głębokość 10.

Labirynt

Platforma I

Tak samo jak w przypadku wersji CUDA-1 rendery dla sceny "Labirynt" działają wolniej w stosunku do wersji CPU, niż pozostałe sceny. Średnio wersja CUDA-2 jest wolniejsza o 272.79% od wersji bazowej.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:01:18.306	00:02:36.515	00:06:30.393	00:13:12.198
CUDA-2	00:04:53.252	00:09:44.814	00:24:21.272	00:48:40.390

Tab. 5.49: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 40 - 400.

Wraz z zakończeniem testów dla tej sceny widzimy, że dla platformy I wszystkie testy zostały zakończone przed przekroczeniem limitu czasowego.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	00:33:08.371	01:04:28.290	02:43:52.173	05:26:06.177
CUDA-2	02:01:41.981	04:03:13.221	10:09:28.306	20:22:22.442

Tab. 5.50: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
CUDA-2	374.49%	373.65%	374.31%	368.64%	367.23%	377.25%	371.92%	374.84%

Tab. 5.51: Porównanie czasu renderu sceny "Labirynt" pomiędzy wersją CPU, a CUDA-2, platforma I, głębokość 10.

Platforma II

W przypadku sceny "Labirynt" wersja CUDA-2 jest w dalszym ciągu nieakceptowanie wolniejsza od wersji CPU. Różnica wynosi 600.94% pomiędzy obiema wersjami.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:02:28.903	00:05:04.538	00:12:49.285	00:25:35.789
CUDA-2	00:17:56.650	00:35:50.743	01:29:36.914	02:59:11.616

Tab. 5.52: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 40 - 400.

Skutkuje to dwoma testami, których czas egzekucji przekroczył limit czasu (5000 i 10000 próbek).

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	01:04:02.628	02:07:38.369	05:21:00.246	11:36:22.172
CUDA-2	02:59:11.616	07:27:58.923	K/C	K/C

Tab. 5.53: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
CUDA-2	723.07%	706.23%	698.95%	700.07%	699.49%	701.86%	K/C	K/C

Tab. 5.54: Porównanie czasu renderu sceny "X" pomiędzy wersją CPU, a CUDA-Y, platforma Z, głębokość 10.

Podsumowanie

Wersja CUDA-2 jest znacznym krokiem naprzód w porównaniu do wersji CUDA-1. Jednakże pozostaje ona dalej wolniejsza od wersji bazowej na obu platformach. Dodatkowo w przypadku platformy II w dalszym ciągu niektóre testy przekraczają limit czasu wykonania. Pomimo tego wersja ta jest dobrym kandydatem do dalszego rozwoju.

5.2.3. CUDA-3

Cornell Box

Platforma I

Wersja CUDA-3 jest niewielką zmianą w stosunku do wersji CUDA-2 sprawdzającą głównie, różnice w modelu przechowywania danych wewnątrz kernela. Wyniki, które widzimy tutaj są bardzo podobne do tych z wersji CUDA-2. Obecnie badana wersja jest około 1.5 raza wolniejsza od wersji CPU.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:00:34.133	00:01:13.430	00:02:55.697	00:05:43.358
CUDA-3	00:00:52.636	00:01:45.500	00:04:22.476	00:08:41.226

Tab. 5.55: Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma I, głębokość 10, próbki 40 - 400.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	00:14:50.694	00:29:02.565	01:12:05.939	02:26:13.639
CUDA-3	00:21:39.547	00:43:27.453	01:48:41.669	03:37:35.395

Tab. 5.56: Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma I, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
CUDA-3	154.21%	143.67%	149.39%	151.80%	145.90%	149.63%	150.76%	148.80%

Tab. 5.57: Porównanie czasu renderu sceny "Cornell Box" pomiędzy wersją CPU, a CUDA-3, platforma I, głębokość 10.

Platforma II

Dla platformy II, widzimy bardzo podobne zachowanie do platformy I gdzie wyniki dla CUDA-3 są bardzo zbliżone do wyników z wersji CUDA-2. Powtarza się nawet zmniejszenie rozbieżności pomiędzy wersją CPU, a GPU dla 10000 próbek.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:00:54.779	00:01:49.986	00:04:37.793	00:09:12.610
CUDA-3	00:03:09.198	00:06:18.107	00:16:02.336	00:32:32.790

Tab. 5.58: Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 40 - 400.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	00:23:05.738	00:46:17.228	01:55:07.566	04:21:15.404
CUDA-3	01:21:13.481	02:42:30.416	06:44:59.332	13:42:09.941

Tab. 5.59: Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
CUDA-3	345.38%	343.76%	346.43%	353.38%	351.69%	351.08%	351.78%	314.70%

Tab. 5.60: Porównanie czasu renderu sceny "Cornell Box" pomiędzy wersją CPU, a CUDA-3, platforma II, głębokość 10.

Lustra

Platforma I

Wyniki dla sceny "Lustra" w dalszym ciągu pozostają bardzo zbliżone do testów CUDA-2. Początkowo wersja CUDA-3 jest wolniejsza 1.6 raza od wersji CPU. Obserwujemy następnie wzrost tej wartości do 1.77 dla 2000 próbek - takie samo zachowanie miało miejsce w poprzedniej wersji. Następnie dla wersji dla 5000 i 1000 próbek wartość ta spada do 1.71 raza. Powtarzalność tego zachowania pomiędzy wersją CUDA-2 i CUDA-3 sugeruje, że nie są to przypadkowe zmiany.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:01:02.837	00:02:06.880	00:04:51.809	00:09:29.411
CUDA-3	00:01:40.661	00:03:21.570	00:08:22.866	00:16:42.838

Tab. 5.61: Czas potrzebny na wygenerowanie sceny "Lustra", platforma I, głębokość 10, próbki 40 - 400.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	00:23:59.375	00:47:04.660	02:01:29.943	03:59:41.314
CUDA-3	00:41:50.270	01:23:37.177	03:28:51.583	06:57:32.922

Tab. 5.62: Czas potrzebny na wygenerowanie sceny "Lustra", platforma I, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
CUDA-3	160.19%	158.87%	172.33%	176.12%	174.40%	177.62%	171.90%	174.20%

Tab. 5.63: Porównanie czasu renderu sceny "Lustra" pomiędzy wersją CPU, a CUDA-3, platforma I, głębokość 10.

Platforma II

Wraz z wzrostem próbek możemy zaobserwować zmniejszenie się dystansu pomiędzy wersją CPU, a CUDA-3. Początkowo jest ona wolniejsza 3.88 raza, a dla 5000 próbek różnica ta spadła do 3.83 raza. Test dla 10000 próbek przekroczył czasowy limit egzekucji. Jest to analogiczna sytuacja do tej jaka miała miejsce w przypadku wersji CUDA-2.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:01:34.121	00:03:10.404	00:07:53.938	00:15:52.267
CUDA-3	00:06:05.610	00:12:10.897	00:30:26.874	01:00:54.461

Tab. 5.64: Czas potrzebny na wygenerowanie sceny "Lustra", platforma II, głębokość 10, próbki 40 - 400.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	00:39:24.280	01:19:29.657	03:18:39.520	07:24:46.301
CUDA-3	02:32:15.984	05:04:29.933	12:41:09.248	K/C

Tab. 5.65: Czas potrzebny na wygenerowanie sceny "Lustra", platforma II, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
CUDA-3	388.45%	383.87%	385.47%	383.76%	386.42%	383.04%	383.15%	K/C

Tab. 5.66: Porównanie czasu renderu sceny "Lustra" pomiędzy wersją CPU, a CUDA-3, platforma II, głębokość 10.

Labirynt

Platforma I

Podobnie jak w przypadku wersji CUDA-2, wersja CUDA-3 jest 3.7 raza wolniejsza od wersji podstawowej dla badanej sceny.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:01:18.306	00:02:36.515	00:06:30.393	00:13:12.198
CUDA-3	00:04:54.541	00:09:44.430	00:24:30.808	00:48:58.991

Tab. 5.67: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 40 - 400.

Najdłuższy test zajął 5 sekund dłużej niż w przypadku wersji CUDA-2. Przy teście trwającym ponad 20 godzin wartość ta jest pomijalna.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	00:33:08.371	01:04:28.290	02:43:52.173	05:26:06.177
CUDA-3	02:01:39.500	04:03:15.625	10:08:02.735	20:22:27.578

Tab. 5.68: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
CUDA-3	376.14%	373.40%	376.75%	370.99%	367.11%	377.31%	371.05%	374.87%

Tab. 5.69: Porównanie czasu renderu sceny "Labirynt" pomiędzy wersją CPU, a CUDA-3, platforma I, głębokość 10.

Platforma II

Ponownie obserwujemy podobne zachowanie jakie miało miejsce w przypadku wersji CUDA-2 włącznie ze zwiększeniem się różnicy pomiędzy wersją CPU i GPU. Testy dla 5000 i 10000 próbek przekroczyły limit czasu wykonania.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:02:28.903	00:05:04.538	00:12:49.285	00:25:35.789
CUDA-3	00:17:55.417	00:35:50.704	01:29:36.907	02:59:13.776

Tab. 5.70: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 40 - 400.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	01:04:02.628	02:07:38.369	05:21:00.246	11:36:22.172
CUDA-3	07:27:58.512	14:56:04.405	K/C	K/C

Tab. 5.71: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
CUDA-3	722.24%	706.21%	698.94%	700.21%	699.48%	702.03%	K/C	K/C

Tab. 5.72: Porównanie czasu renderu sceny "Labirynt" pomiędzy wersją CPU, a CUDA-3, platforma II, głębokość 10.

Podsumowanie

Wersja ta zachowuje się bardzo podobnie do wersji CUDA-2. Jedną z kluczowych różnic jest brak limitu obiektów na scenie (W przypadku wersji CUDA-2 można maksymalnie wczytać 100 obiektów.). Wersja ta także jest rozważana jako baza do dalszego rozwoju.

5.2.4. CUDA-4

Wersja ta została stworzona w celu zmaksymalizowania wykorzystania GPU i CPU jednocześnie. Ponieważ każdy kernel GPU oblicza tylko próbki dla pojedynczego piksela celem tej wersji jest uzyskanie lepszego skalowania niż w wersjach poprzednich.

Cornell Box

Platforma I

Początkowo wyniki wersji CUDA-4 są gorsze niż wersja CUDA-1. Dla 40 próbek wersja CUDA-4 jest wolniejsza o 8291.33%. Wraz z zwiększeniem się ilości próbek wynik ten ulega poprawie. Dla 10000 próbek wersja CUDA-4 jest wolniejsza już tylko o 204.80%. Jest to maszywna różnica pomiędzy skrajnymi testami.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:00:34.133	00:01:13.430	00:02:55.697	00:05:43.358
CUDA-4	00:52:51.409	01:06:31.547	02:17:57.260	04:17:06.143

Tab. 5.73: Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma I, głębokość 10, próbki 40 - 400.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	00:14:50.694	00:29:02.565	01:12:05.939	02:26:13.639
CUDA-4	05:24:36.457	05:38:54.447	06:22:12.381	07:25:41.659

Tab. 5.74: Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma I, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100%	100%	100%	100%	100%	100%	100%	100%
CUDA-4	9291.33%	5435.85%	4711.10%	4492.73%	2186.66%	1166.93%	530.11%	304.80%

Tab. 5.75: Porównanie czasu renderu sceny "Cornell Box" pomiędzy wersją CPU, a CUDA-4, platforma I, głębokość 10.

Dla wersji CUDA-4 widzimy także zmianę w zużyciu pamięci w stosunku do wcześniejszych wersji. Rozmiar tablicy, która przechowuje wyniki z kernela CUDA rośnie wraz z ilością próbek. Test dla 10000 próbek zużywa o 216.88 MiB pamięci więcej niż test dla 40 próbek. Zużycie pamięci dla skrajnych parametrów testowych wzrosło ponad dwukrotnie.

Wersja	Ilość próbek								Średnie
	40	80	200	400	1000	2000	5000	10000	
CUDA-1	131.20	131.32	131.20	131.06	131.19	131.19	131.20	K/C	131.19
CUDA-4	156.80	156.55	160.05	164.55	174.30	195.30	266.30	375.68	206.19

Tab. 5.76: Zużycie pamięci systemowej dla sceny "Cornell Box", platforma I, głębokość 10, dane w MiB.

Wersja CUDA-4 jest pierwszą wersją, dla której widzimy zmianę w alokacji pamięci GPU przez program *tracer*. Do tej pory każda testowana wersja wykorzystująca tylko jeden kernel zużywała 226 MiB pamięci niezależnie od ilości próbek. Dla testów z przedziału 40 - 400 ilości próbek pozostaje niższa od tej wartości. Widzimy tylko minimalny wzrost w tym przedziale. Różnica pomiędzy testem dla 40 i 400 próbek wynosi tylko 8 MiB. Sytuacja zmienia się w przedziale 1000 - 10000. Różnica pomiędzy skrajnymi próbkami w tym przedziale wynosi 172 MiB pamięci. Maksymalna odnotowana alokacja pamięci wyniosła 394 MiB.

Wersja	Ilość próbek								Średnie
	40	80	200	400	1000	2000	5000	10000	
CUDA-4	204	204	208	212	222	242	300	394	248.5

Tab. 5.77: Zużycie pamięci GPU dla sceny "Cornell Box", platforma I, głębokość 10, dane w MiB

Platforma II

W tym przypadku także widzimy znaczne poprawy programu wraz z zwiększaniem się ilości próbek. Różnice w stosunku do wersji CPU są mniejsze niż to miało miejsce w przypadku platformy I.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:00:54.779	00:01:49.986	00:04:37.793	00:09:12.610
CUDA-4	00:58:53.139	01:14:37.974	02:35:19.518	04:49:54.324

Tab. 5.78: Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 40 - 400.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	00:23:05.738	00:46:17.228	01:55:07.566	04:21:15.404
CUDA-4	06:07:07.994	06:23:28.432	07:17:38.687	13:50:44.957

Tab. 5.79: Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100%	100%	100%	100%	100%	100%	100%	100%
CUDA-4	6449.81%	4071.40%	3354.84%	3147.67%	1589.62%	828.47%	380.14%	317.98%

Tab. 5.80: Porównanie czasu renderu sceny "Cornell Box" pomiędzy wersją CPU, a CUDA-4, platforma II, głębokość 10.

Wersja	Ilość próbek								Średnie
	40	80	200	400	1000	2000	5000	10000	
CUDA-1	135.55	135.56	135.80	135.69	135.81	135.84	K/C	K/C	135.71
CUDA-4	154.33	155.59	158.14	163.08	173.34	191.95	257.58	367.70	202.71

Tab. 5.81: Zużycie pamięci systemowej dla sceny "Cornell Box", platforma II, głębokość 10, dane w MiB.

W tym przypadku także obecne widzimy wzrost alokacji pamięci GPU wraz z wzrostem ilości próbek. Podobnie jak w przypadku innych wersji program *tracer* zużywa mniej pamięci na platformie II.

Wersja	Ilość próbek								Średnie
	40	80	200	400	1000	2000	5000	10000	
CUDA-4	74	74	78	80	90	112	170	266	118

Tab. 5.82: Zużycie pamięci GPU dla sceny "X", platforma Y, głębokość 10, dane w MiB

Lustra

Platforma I

Początkowo wyniki wersji CUDA-4 ponownie są gorsze od wersji CUDA-1. Dla testu dla 40 próbek jest ona wolniejsza o 8291.33% od wersji CPU. Dla testu dla 10000 próbek różnica ta wynosi jednak już tylko 144.56%. Pomimo dobrego skalowania wraz ze wzrostem próbek wersja CUDA-4 jest średnio wolniejsza o 1921.67% od wersji CPU.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:01:02.837	00:02:06.880	00:04:51.809	00:09:29.411
CUDA-4	01:15:17.657	01:34:37.132	03:20:13.801	06:14:28.716

Tab. 5.83: Czas potrzebny na wygenerowanie sceny "Lustra", platforma I, głębokość 10, próbki 40 - 400.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	00:23:59.375	00:47:04.660	02:01:29.943	03:59:41.314
CUDA-4	07:48:27.800	07:59:57.642	08:41:59.484	09:46:11.384

Tab. 5.84: Czas potrzebny na wygenerowanie sceny "Lustra", platforma I, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100%	100%	100%	100%	100%	100%	100%	100%
CUDA-4	7189.49%	4474.41%	4117.01%	3945.96%	1952.78%	1019.51%	429.63%	244.56%

Tab. 5.85: Porównanie czasu renderu sceny "Lustra" pomiędzy wersją CPU, a CUDA-4, platforma I, głębokość 10.

Dla sceny "Labirynt" obserwujemy takie samo zachowanie w zużyciu pamięci jak dla sceny "Cornell Box"

Wersja	Ilość próbek								Średnie
	40	80	200	400	1000	2000	5000	10000	
CUDA-1	131.07	131.14	131.19	131.06	131.06	131.31	K/C	K/C	131.14
CUDA-4	156.79	157.79	159.64	164.01	173.66	195.29	260.01	375.66	205.36

Tab. 5.86: Zużycie pamięci systemowej dla sceny "Lustra", platforma I, głębokość 10, dane w MiB.

Wartości zużycia pamięci GPU są dokładnie takie same jak w przypadku sceny "Cornell Box" dla tych samych próbek. Zużycie pamięci jest tu decydowane przez ilość próbek, a nie przez testowaną scenę.

Wersja	Ilość próbek								Średnie
	40	80	200	400	1000	2000	5000	10000	
CUDA-4	204	204	208	212	222	242	300	394	248.5

Tab. 5.87: Zużycie pamięci GPU dla sceny "Lustra", platforma I, głębokość 10, dane w MiB

Platforma II

Już na wet w przypadku sceny "Lustra" większość z testów nie zdołała zakończyć się w limicie czasu. Pomimo tego w dalszym ciągu widzimy tendencje zbliżania się osiągnięć wersji CUDA-4 do CPU. Dla 400 próbek różnica była ta już prawie dwukrotnie mniejsza niż w przypadku testu dla 40 próbek.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:01:34.121	00:03:10.404	00:07:53.938	00:15:52.267
CUDA-4	01:23:59.607	01:46:23.756	03:45:45.364	07:02:32.981

Tab. 5.88: Czas potrzebny na wygenerowanie sceny "Lustra", platforma II, głębokość 10, próbki 40 - 400.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	00:39:24.280	01:19:29.657	03:18:39.520	07:24:46.301
CUDA-4	14:22:27.281	K/C	K/C	K/C

Tab. 5.89: Czas potrzebny na wygenerowanie sceny "Lustra", platforma II, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100%	100%	100%	100%	100%	100%	100%	100%
CUDA-4	5354.39%	3352.74%	2858.05%	2662.38%	2188.71%	K/C	K/C	K/C

Tab. 5.90: Porównanie czasu renderu sceny "Lustra" pomiędzy wersją CPU, a CUDA-4, platforma II, głębokość 10.

Wersja	Ilość próbek								Średnie
	40	80	200	400	1000	2000	5000	10000	
CUDA-1	135.61	135.81	135.56	135.62	135.69	135.93	K/C	K/C	135.70
CUDA-4	154.83	155.70	158.45	162.64	172.52	K/C	K/C	K/C	160.83

Tab. 5.91: Zużycie pamięci systemowej dla sceny "Lustra", platforma II, głębokość 10, dane w MiB.

Zużycie pamięci pozostaje zbliżone do osiągnięć dla sceny "Cornell Box". Niestety ze względu na ograniczone dane nie wiemy czy ten trend utrzymałby się dla testów dla większej ilości próbek.

Wersja	Ilość próbek								Średnie
	40	80	200	400	1000	2000	5000	10000	
CUDA-4	74	74	78	80	90	K/C	K/C	K/C	79.2

Tab. 5.92: Zużycie pamięci GPU dla sceny "Lustra", platforma II, głębokość 10, dane w MiB

Labirynt

Platforma I

W testach dla platformy "Labirynt" widzimy największą różnicę pomiędzy wersją GPU, a CPU do tej pory. Dla 40 próbek wersja CUDA-4 jest wolniejsza o 22711.92%. Wraz z wzrostem ilości próbek wynik ten ulega znaczącej poprawie. Dla 200 próbek różnica względem wersji CPU wynosi już tylko 12680.29%. Niestety kolejne testy przekroczyły limit czasu egzekucji. Nie wiemy przez to na pewno jak sytuacja wyglądała by dla testu dla 10000 próbek. Bazując na wynikach, które mamy możemy przepuszczać, że obie wersje aplikacji zbliżały by się do siebie.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:01:18.306	00:02:36.515	00:06:30.393	00:13:12.198
CUDA-4	04:57:43.101	06:22:03.842	13:51:33.375	K/C

Tab. 5.93: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 40 - 400.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	00:33:08.371	01:04:28.290	02:43:52.173	05:26:06.177
CUDA-4	K/C	K/C	K/C	K/C

Tab. 5.94: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100%	100%	100%	100%	100%	100%	100%	100%
CUDA-4	22811.92%	14646.42%	12780.29%	K/C	K/C	K/C	K/C	K/C

Tab. 5.95: Porównanie czasu renderu sceny "Labirynt" pomiędzy wersją CPU, a CUDA-4, platforma I, głębokość 10.

Zbyt mało testów zostało ukończonych w limicie czasu aby można było wyciągnąć wnioski na temat zużycia pamięci.

Wersja	Ilość próbek								Średnie
	40	80	200	400	1000	2000	5000	10000	
CUDA-1	131.20	131.57	131.32	131.45	131.07	K/C	K/C	K/C	131.32
CUDA-4	156.29	156.79	159.91	K/C	K/C	K/C	K/C	K/C	157.66

Tab. 5.96: Zużycie pamięci systemowej dla sceny "Labirynt", platforma I, głębokość 10, dane w MiB.

Obserwujemy dokładnie taką samą alokację pamięci dla sceny "Labirynt" jak w przypadku sceny "Cornell Box" i "Lustra". Zużycie pamięci GPU zależy od ilości próbek, a nie od testowanej sceny.

Wersja	Ilość próbek								Średnie
	40	80	200	400	1000	2000	5000	10000	
CUDA-4	204	204	208	K/C	K/C	K/C	K/C	K/C	205.33

Tab. 5.97: Zużycie pamięci GPU dla sceny "Labirynt", platforma I, głębokość 10, dane w MiB

Platforma II

W tym wypadku także tylko trzy testy zostały ukończone. Nawet dla tak małej ilości testów widzimy maszyną poprawę pomiędzy próbką 40, 200. Niestety jest to za mało aby osiągnąć wynik, który kwalifikowałby tę wersję do dalszego rozwoju.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:02:28.903	00:05:04.538	00:12:49.285	00:25:35.789
CUDA-4	05:33:12.730	07:11:07.464	15:38:43.700	K/C

Tab. 5.98: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 40 - 400.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	01:04:02.628	02:07:38.369	05:21:00.246	11:36:22.172
CUDA-4	K/C	K/C	K/C	K/C

Tab. 5.99: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100%	100%	100%	100%	100%	100%	100%	100%
CUDA-4	13426.68%	8494.00%	7321.56%	K/C	K/C	K/C	K/C	K/C

Tab. 5.100: Porównanie czasu renderu sceny "Labirynt" pomiędzy wersją CPU, a CUDA-4, platforma II, głębokość 10.

Wersja	Ilość próbek								Średnie
	40	80	200	400	1000	2000	5000	10000	
CUDA-1	135.59	135.49	135.65	135.86	135.64	K/C	K/C	K/C	135.65
CUDA-4	154.50	155.46	158.59	K/C	K/C	K/C	K/C	K/C	156.18

Tab. 5.101: Zużycie pamięci systemowej dla sceny "Labirynt", platforma II, głębokość 10, dane w MiB.

Dla platformy II nie widzimy powtórzenia zależności pomiędzy ilością próbek, alokacją pamięci GPU. Dla sceny "Labirynt" zużycie pamięci dla testu próbek odbiega o 2 MiB w porównaniu do poprzednich scen. W dalszym ciągu wartości te pozostają do siebie bardzo zbliżone.

Wersja	Ilość próbek								Średnie
	40	80	200	400	1000	2000	5000	10000	
CUDA-4	74	74	76	K/C	K/C	K/C	K/C	K/C	74.67

Tab. 5.102: Zużycie pamięci GPU dla sceny "Labirynt", platforma I, głębokość 10, dane w MiB

Podsumowanie

Pomimo dobrego skalowania i akceptowalnych wyników dla testów 10000 próbek wersja CUDA-4 została odrzucona jako opcja do dalszego rozwoju. Wyniki, które osiąga ta wersja dla niskiej ilości próbek są znacznie gorsze nawet od najwolniejszej do tej pory wersji CUDA-1. Najwięcej czasu jest tracone na ciągłe transfery danych pomiędzy pamięcią systemową, a GPU.

5.2.5. CUDA-5

Cornell Box

Platforma I

Wersja CUDA-5 także zawodzi oczekiwania. Zwiększenie ilości pracy przydzielonej do GPU miało zmniejszyć ilość czasu marnowanego przez aplikację na kopiowanie danych pomiędzy pamięcią GPU, a systemową. Początkowo widzimy wyraźną poprawę nad wersją CUDA-4. Niestety znika też lepsze skalowanie się wraz z wzrostem ilości próbek. Wersja CUDA-5 pozostaje średnio wolniejsza o 1336.82%. Jest to wartość minimalnie lepsza od wersji CUDA-1, pozostaje ona jednak w dalszym ciągu znacznie za osiąganymi wersjami CUDA-2 i CUDA-3.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:00:34.133	00:01:13.430	00:02:55.697	00:05:43.358
CUDA-5	00:08:24.928	00:16:47.399	00:41:56.811	01:23:51.424

Tab. 5.103: Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma I, głębokość 10, próbki 40 - 400.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	00:14:50.694	00:29:02.565	01:12:05.939	02:26:13.639
CUDA-5	03:29:36.223	06:59:11.300	17:27:52.447	K/C

Tab. 5.104: Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma I, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100%	100%	100%	100%	100%	100%	100%	100%
CUDA-5	1479.30%	1371.92%	1432.47%	1465.36%	1411.96%	1443.35%	1453.38%	K/C

Tab. 5.105: Porównanie czasu renderu sceny "Cornell Box" pomiędzy wersją CPU, a CUDA-5, platforma I, głębokość 10.

Zużycie pamięci, choć większe niż wersja CUDA-1 - CUDA-3 pozostaje stabilne niezależnie od ilości próbek. Obserwujemy drobną anomalie przy teście dla 40 próbek. Jeżeli ją wykluczymy średni zużycie wersji CUDA-5 jest większe o 26.96 [MiB] od wersji CUDA-1.

Wersja	Ilość próbek								Średnie
	40	80	200	400	1000	2000	5000	10000	
CUDA-1	131.20	131.32	131.20	131.06	131.19	131.19	131.20	K/C	131.19
CUDA-5	224.85	158.41	156.79	158.04	158.91	158.36	158.41	K/C	167.68

Tab. 5.106: Zużycie pamięci systemowej dla sceny "Cornell Box", platforma I, głębokość 10, dane w MiB.

Widzimy też większą stabilność w alokacji pamięci GPU. Nie widzimy trendu wzrostowego jak w przypadku wersji CUDA-4 ale nie widzimy też perfekcyjnie stałego zużycia pamięci jak w przypadku pozostałych wersji.

Wersja	Ilość próbek								Średnie
	40	80	200	400	1000	2000	5000	10000	
CUDA-5	216	214	214	216	212	208	206	K/C	212.29

Tab. 5.107: Zużycie pamięci GPU dla sceny "Cornell Box", platforma I, głębokość 10, dane w MiB

Platforma II

Co Ciekawe w tym widzimy czasy obu platform są zbliżone do siebie znacznie bardziej niż w pozostałych wersjach. Sugeruje to nieoptymalne wykorzystanie karty graficznej w przypadku platformy I.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:00:54.779	00:01:49.986	00:04:37.793	00:09:12.610
CUDA-5	00:09:29.612	00:18:56.978	00:47:20.110	01:34:37.537

Tab. 5.108: Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 40 - 400.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	00:23:05.738	00:46:17.228	01:55:07.566	04:21:15.404
CUDA-5	03:58:32.145	08:08:13.124	19:37:44.925	K/C

Tab. 5.109: Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100%	100%	100%	100%	100%	100%	100%	100%
CUDA-5	1039.84%	1033.75%	1022.38%	1027.40%	1032.82%	1054.76%	1023.01%	K/C

Tab. 5.110: Porównanie czasu renderu sceny "Cornell Box" pomiędzy wersją CPU, a CUDA-5, platforma II, głębokość 10.

Wahania w zużyciu pamięci systemowej oraz alokacji pamięci GPU prezentują podobne zachowanie jak w przypadku platformy I. Obserwowane wartości są wyższe od zmierzonych w pozostałych wersjach aplikacji.

Wersja	Ilość próbek								Średnie
	40	80	200	400	1000	2000	5000	10000	
CUDA-1	135.55	135.56	135.80	135.69	135.81	135.84	K/C	K/C	135.71
CUDA-5	152.72	152.96	152.67	152.96	152.72	152.89	152.77	K/C	152.81

Tab. 5.111: Zużycie pamięci systemowej dla sceny "Cornell Box", platforma II, głębokość 10, dane w MiB.

Wersja	Ilość próbek								Średnie
	40	80	200	400	1000	2000	5000	10000	
CUDA-5	80	80	80	78	81	79	80	K/C	79.71

Tab. 5.112: Zużycie pamięci GPU dla sceny "Cornell Box", platforma II, głębokość 10, dane w MiB

Lustra

Platforma I

Dla sceny lustra widzimy podobne zachowanie jak w przypadku poprzedniej sceny. Wersja CUDA-5 utrzymuje swój dystans od wersji CPU niezależnie od ilości próbek. Jest ona średnio wolniejsza o 1336.82%. Jest to wynik minimalnie lepszy od osiągniętego przez wersję CUDA-1.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:01:02.837	00:02:06.880	00:04:51.809	00:09:29.411
CUDA-5	00:14:43.145	00:29:33.290	01:13:51.578	02:27:41.583

Tab. 5.113: Czas potrzebny na wygenerowanie sceny "Lustra", platforma I, głębokość 10, próbki 40 - 400.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	00:23:59.375	00:47:04.660	02:01:29.943	03:59:41.314
CUDA-5	06:09:12.337	12:18:25.472	K/C	K/C

Tab. 5.114: Czas potrzebny na wygenerowanie sceny "Lustra", platforma I, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100%	100%	100%	100%	100%	100%	100%	100%
CUDA-5	1405.45%	1397.61%	1518.66%	1556.27%	1539.02%	1568.52%	K/C	K/C

Tab. 5.115: Porównanie czasu renderu sceny "Lustra" pomiędzy wersją CPU, a CUDA-5, platforma I, głębokość 10.

Wersja	Ilość próbek								Średnie
	40	80	200	400	1000	2000	5000	10000	
CUDA-1	131.07	131.14	131.19	131.06	131.06	131.31	K/C	K/C	131.14
CUDA-5	158.18	158.82	158.29	158.92	158.30	158.18	K/C	K/C	158.45

Tab. 5.116: Zużycie pamięci systemowej dla sceny "Lustra", platforma I, głębokość 10, dane w MiB.

W przeciwieństwie do wersji CUDA-4 nie widzimy zgodności w alokacji pamięci pomiędzy scenami. Średnio zużycie oscyluje wokół tej samej wartości jednak w zależności od ilości próbek widzimy inną wartość przy każdej z testowanych scen.

Wersja	Ilość próbek								Średnie
	40	80	200	400	1000	2000	5000	10000	
CUDA-5	214	214	214	212	208	206	K/C	K/C	211.33

Tab. 5.117: Zużycie pamięci GPU dla sceny "Lustra", platforma I, głębokość 10, dane w MiB

Platforma II

Osiągi obu platform testowych pozostają do siebie bardzo zbliżone. Sugeruje to, że nie jest zjawisko wyizolowane do konkretnej sceny. Wyniki zużycia pamięci dla tej sceny nie różnią się znacząco od poprzedniej.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:01:34.121	00:03:10.404	00:07:53.938	00:15:52.267
CUDA-5	00:16:40.270	00:33:19.974	01:23:17.626	02:46:33.771

Tab. 5.118: Czas potrzebny na wygenerowanie sceny "Lustra", platforma II, głębokość 10, próbki 40 - 400.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	00:39:24.280	01:19:29.657	03:18:39.520	07:24:46.301
CUDA-5	06:51:56.843	14:18:13.621	K/C	K/C

Tab. 5.119: Czas potrzebny na wygenerowanie sceny "Lustra", platforma II, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100%	100%	100%	100%	100%	100%	100%	100%
CUDA-5	1062.75%	1050.38%	1054.49%	1049.47%	1045.43%	1079.61%	K/C	K/C

Tab. 5.120: Porównanie czasu renderu sceny "Lustra" pomiędzy wersją CPU, a CUDA-5, platforma II, głębokość 10.

Wersja	Ilość próbek								Średnie
	40	80	200	400	1000	2000	5000	10000	
CUDA-1	135.61	135.81	135.56	135.62	135.69	135.93	K/C	K/C	135.70
CUDA-5	152.95	152.89	153.08	153.14	153.02	152.98	K/C	K/C	153.01

Tab. 5.121: Zużycie pamięci systemowej dla sceny "Lustra", platforma II, głębokość 10, dane w MiB.

Wersja	Ilość próbek								Średnie
	40	80	200	400	1000	2000	5000	10000	
CUDA-5	80	80	80	76	78	80	K/C	K/C	79

Tab. 5.122: Zużycie pamięci GPU dla sceny "Lustra", platforma II, głębokość 10, dane w MiB

Labirynt

Platforma I

Wersja CUDA-5 jest wolniejsza od wersji CPU o 3346.62%. Jest to wynik gorszy od wersji CUDA-1.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:01:18.306	00:02:36.515	00:06:30.393	00:13:12.198
CUDA-5	00:45:12.587	01:30:25.302	03:45:55.442	07:31:50.894

Tab. 5.123: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 40 - 400.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	00:33:08.371	01:04:28.290	02:43:52.173	05:26:06.177
CUDA-5	18:49:27.480	K/C	K/C	K/C

Tab. 5.124: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100%	100%	100%	100%	100%	100%	100%	100%
CUDA-5	3464.09%	3466.31%	3472.26%	3422.24%	3408.19%	K/C	K/C	K/C

Tab. 5.125: Porównanie czasu renderu sceny "Labirynt" pomiędzy wersją CPU, a CUDA-5, platforma I, głębokość 10.

Wersja	Ilość próbek								Średnie
	40	80	200	400	1000	2000	5000	10000	
CUDA-1	131.20	131.57	131.32	131.45	131.07	K/C	K/C	K/C	131.32
CUDA-5	156.68	157.34	158.09	158.84	158.18	K/C	K/C	K/C	157.83

Tab. 5.126: Zużycie pamięci systemowej dla sceny "Labirynt", platforma I, głębokość 10, dane w MiB.

Ponownie dla tej sceny nie widzimy żadnego wzorca w alokacji pamięci w porównaniu do poprzednich scen. Jest to jedyna wersja aplikacji, dla której obserwujemy takie zachowanie.

Wersja	Ilość próbek								Średnie
	40	80	200	400	1000	2000	5000	10000	
CUDA-5	214	214	210	206	204	K/C	K/C	K/C	209.6

Tab. 5.127: Zużycie pamięci GPU dla sceny "Labirynt", platforma I, głębokość 10, dane w MiB

Platforma II

Dla ostatniej sceny testowej obie platformy testowej osiągają zbliżone wyniki. Tak, jak wspomniano przy scenie "Cornell Box" wcześniej sugeruje to nieoptymalne wykorzystanie zasobów przez tą wersję aplikacji.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:02:28.903	00:05:04.538	00:12:49.285	00:25:35.789
CUDA-5	00:51:03.760	01:42:03.884	04:15:04.617	08:30:06.450

Tab. 5.128: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 40 - 400.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	01:04:02.628	02:07:38.369	05:21:00.246	11:36:22.172
CUDA-5	21:22:28.157	K/C	K/C	K/C

Tab. 5.129: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100%	100%	100%	100%	100%	100%	100%	100%
CUDA-5	2057.55%	2010.88%	1989.46%	1992.88%	2002.49%	K/C	K/C	K/C

Tab. 5.130: Porównanie czasu renderu sceny "Labirynt" pomiędzy wersją CPU, a CUDA-5, platforma II, głębokość 10.

Wersja	Ilość próbek								Średnie
	40	80	200	400	1000	2000	5000	10000	
CUDA-1	135.59	135.49	135.65	135.86	135.64	K/C	K/C	K/C	135.65
CUDA-5	152.75	152.99	153.11	152.87	152.90	K/C	K/C	K/C	152.92

Tab. 5.131: Zużycie pamięci systemowej dla sceny "Labirynt", platforma II, głębokość 10, dane w MiB.

Wersja	Ilość próbek								Średnie
	40	80	200	400	1000	2000	5000	10000	
CUDA-5	80	78	76	74	78	K/C	K/C	K/C	77.2

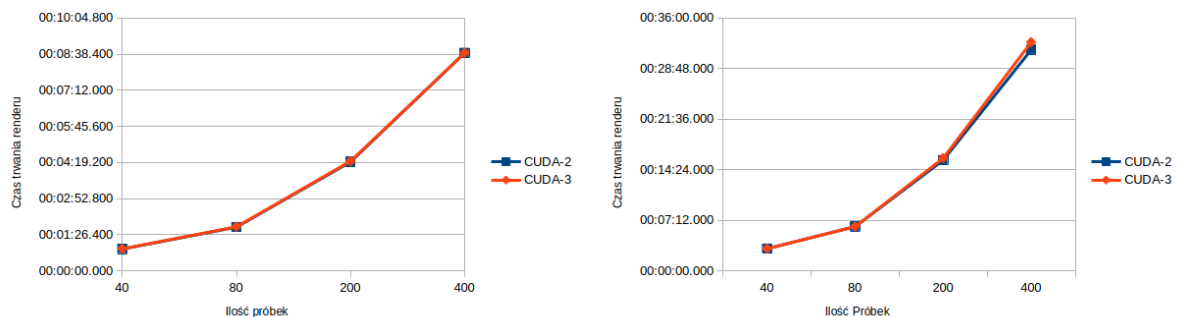
Tab. 5.132: Zużycie pamięci GPU dla sceny "Labirynt", platforma II, głębokość 10, dane w MiB

Podsumowanie

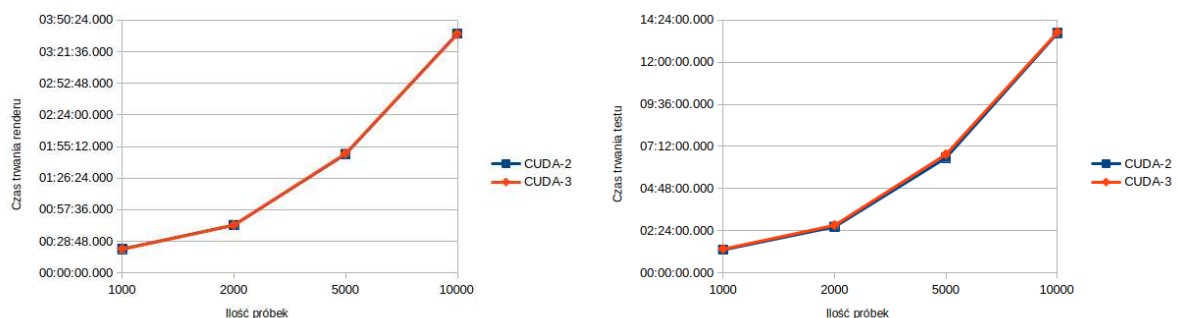
Wersja CUDA-5 była próbą zachowania zalet wersji CUDA-4 przy skróceniu znacznie wydłużonych czasów renderów dla niskiej ilości próbek. Próba ta się nie udała. Wersja CUDA-5 jest minimalnie lepsza od wersji CUDA-1 dla wszystkich scen oprócz sceny "Labirynt". Podkreśla to problemy z nieoptymalnym wykorzystaniem zasobów. Pomimo zwiększonego zużycia pamięci systemowej wersja ta nie oferuje żadnych zalet. Wersja została wykluczona z dalszego rozwoju.

5.2.6. Dalszy rozwój

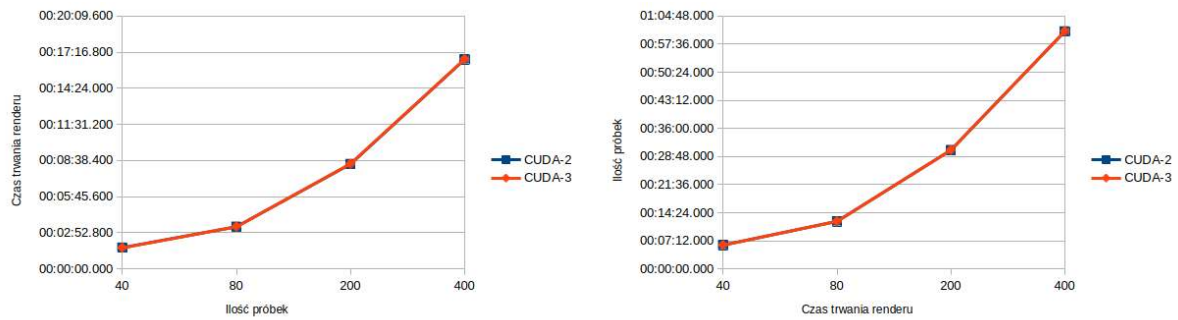
Wybór wersji do dalszego rozwoju stanął pomiędzy wersją CUDA-2 oraz CUDA-3. Obie wersje różnią się sposobem przechowywania danych wewnątrz kernela CUDA. Metoda wykorzystana w przypadku wersji CUDA-2 powoduje, że na scenie może znaleźć się maksymalnie 100 obiektów. Jest to wystarczające w przypadku scen testowych ogranicza jednak potencjalnie możliwości tworzenia nowych scen.



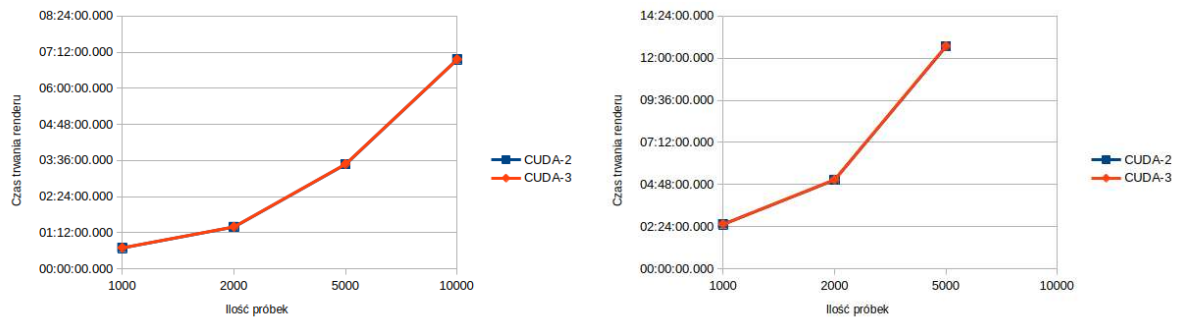
Rys. 5.7: Porównanie render sceny "Cronell Box". Od lewej: platforma I, platforma II. Przedział 40 - 400, głębokość 10.



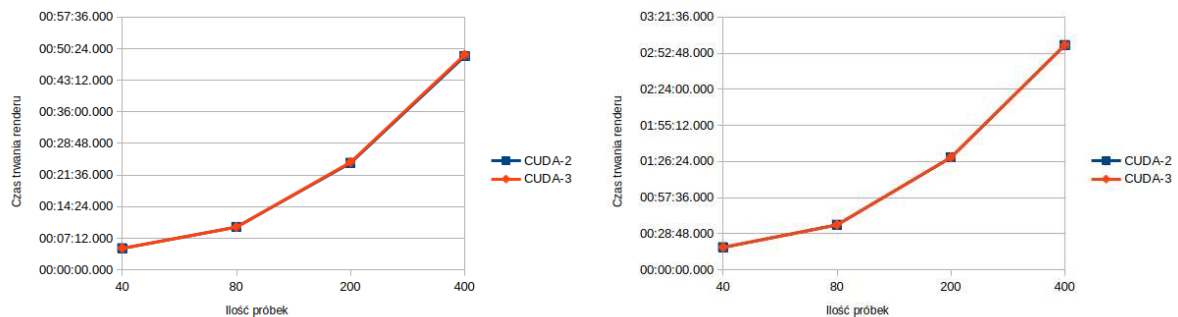
Rys. 5.8: Porównanie render sceny "Cronell Box". Od lewej: platforma I, platforma II. Przedział 1000 - 10000, głębokość 10.



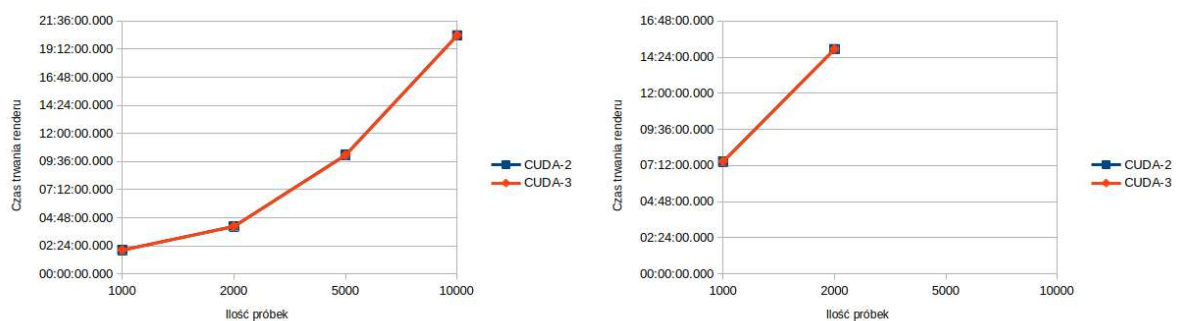
Rys. 5.9: Porównanie render sceny "Lustra". Od lewej: platforma I, platforma II. Przedział 40 - 400, głębokość 10.



Rys. 5.10: Porównanie render sceny "Lustra". Od lewej: platforma I, platforma II. Przedział 1000 - 10000, głębokość 10.



Rys. 5.11: Porównanie render sceny "Labirynt". Od lewej: platforma I, platforma II. Przedział 40 - 400, głębokość 10.



Rys. 5.12: Porównanie render sceny "Labirynt". Od lewej: platforma I, platforma II. Przedział 1000 - 10000, głębokość 10.

Jak widzimy na wykresach 5.7, 5.8, 5.9, 5.10, 5.11 oraz 5.12 wyniki z obu wersji funkcji *tracer* pokrywają się prawie idealnie. Tylko na niektórych wykresach widać niewielkie odchylenia na korzyść jednej lub drugiej wersji. Przykładowo: Na wykresie zakresu 1000 - 10000 dla sceny "Cornell Box" możemy zaobserwować niewielką przewagę na rzecz wersji CUDA-2 w teście dla 5000 próbek. Różnice te są jednak na tyle nie wielkie, że ostatecznie wybrana została wersja CUDA-3, ponieważ nie posiada ona ograniczeń ilości obiektów na tworzonych scenach.

Rozdział 6

Optymalizacja

Na podstawie wybranej wcześniej wersji aplikacji zostały stworzone trzy wersje aplikacji modelujące różne warianty systemu oświetlenia.

6.1. Model klasyczny

Model klasyczny implementuje algorytm path tracingu bez żadnych zmian. Na scenę wysyłane są promienie (ich punkt startowy jest brany losowo używając filtru namiotowego), następnie promień odbija się od obiektów znajdujących się na scenie:

- obiekty typu *diffuse* (rozpraszające światło) - odbicie promienia się jest losowe,
- obiekty typu *specular* (lustro) - odbicie zgodnie ze wzorem,
- obiekty typu *refractive* (szkło) - załamanie oraz odbicie światła zgodnie ze wzorem.

6.1.1. CUDA-3.1

W trakcie analizy kodu programu zwróciłem uwagę, że najwięcej czasu zajmuje przetwarzanie kodu związanego z obiektem płaszczyzny. Wersja CUDA-3.1 skupia się głównie na optymalizacji tej części kodu.

Cornell Box

Platforma I

Dla sceny "Cornell Box" od razu widzimy efekt wprowadzonych zmian - jest to pierwsza wersja, dla której implementacja GPU jest szybsza od wersji CPU.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:00:34.133	00:01:13.430	00:02:55.697	00:05:43.358
CUDA-3.1	00:00:28.741	00:00:57.505	00:02:23.617	00:04:46.448

Tab. 6.1: Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma I, głębokość 10, próbki 40 - 400.

Wersja CUDA-3.1 jest średnio szybsza o 18.72% w stosunku do wersji CPU.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	00:14:50.694	00:29:02.565	01:12:05.939	02:26:13.639
CUDA-Z	00:11:54.811	00:23:48.541	00:59:28.795	01:58:54.209

Tab. 6.2: Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma I, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
CUDA-3.1	84.20%	78.31%	81.74%	83.43%	80.25%	81.98%	82.50%	81.31%

Tab. 6.3: Porównanie czasu renderu sceny "Cornell Box" pomiędzy wersją CPU, a CUDA-3.1, platforma I, głębokość 10.

Platforma II

Wyniki dla w przypadku platformy II są w dalszym ciągu gorsze od wersji CPU. Wersja CUDA-3.1 jest około dwukrotnie wolniejsza od wersji bazowej.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:00:54.779	00:01:49.986	00:04:37.793	00:09:12.610
CUDA-3.1	00:01:55.429	00:03:50.383	00:09:35.164	00:19:08.951

Tab. 6.4: Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 40 - 400.

Pomimo tego wersja ta pozostaje poprawą w w stosunku do wersji poprzedniej. Średnio wersja CUDA-3 była wolniejsza o 244.78%, w porównaniu do 103.38% dla wersji CUDA-3.1. Co ciekawe dla 10000 próbek obserwujemy zmniejszenie tej różnicy od wersji CPU tylko do 68.40%.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	00:23:05.738	00:46:17.228	01:55:07.566	04:21:15.404
CUDA-3.1	00:47:57.565	01:36:05.769	04:00:03.720	07:19:10.900

Tab. 6.5: Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
CUDA-3.1	210.72%	209.47%	207.05%	207.91%	207.66%	207.61%	208.52%	168.10%

Tab. 6.6: Porównanie czasu renderu sceny "Cornell Box" pomiędzy wersją CPU, a CUDA-3.1, platforma II, głębokość 10.

Lustra**Platforma I**

Dla sceny "Lustra" także widzimy wyniki lepsze od wersji CPU.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:01:02.837	00:02:06.880	00:04:51.809	00:09:29.411
CUDA-3.1	00:00:53.515	00:01:46.920	00:04:28.430	00:08:55.933

Tab. 6.7: Czas potrzebny na wygenerowanie sceny "Lustra", platforma I, głębokość 10, próbki 40 - 400.

Początkowo dla 40 próbek scena "Cornell Box" i "Lustra" mają bardzo podobne wyniki. Około 14% szybciej do wersji CPU. W przypadku obecnej sceny różnica ta jednak spada dla testów o wyższych próbek i utrzymuje się na poziomie około 9% szybciej od wersji CPU.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	00:23:59.375	00:47:04.660	02:01:29.943	03:59:41.314
CUDA-3.1	00:22:17.426	00:44:32.477	01:51:15.638	00:00:00.000

Tab. 6.8: Czas potrzebny na wygenerowanie sceny "Lustra", platforma I, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
CUDA-3.1	85.16%	84.27%	91.99%	94.12%	92.92%	94.61%	91.57%	92.90%

Tab. 6.9: Porównanie czasu renderu sceny "Lustra" pomiędzy wersją CPU, a CUDA-3.1, platforma I, głębokość 10.

Platforma II

Scena "Lustra" jest wolniejsza od wersji CPU o około kolejne 26% w porównaniu do sceny "Cornell Box".

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:01:34.121	00:03:10.404	00:07:53.938	00:15:52.267
CUDA-3.1	00:03:37.811	00:07:15.651	00:18:11.181	00:36:20.642

Tab. 6.10: Czas potrzebny na wygenerowanie sceny "Lustra", platforma II, głębokość 10, próbki 40 - 400.

Ponownie dla testu dla 10000 próbek widzimy zmniejszenie się różnicy pomiędzy wersją CPU, CUDA-3.1 (Jest ona wolniejsza o 88.40%).

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	00:39:24.280	01:19:29.657	03:18:39.520	07:24:46.301
CUDA-3.1	01:30:52.773	03:01:58.228	07:34:33.138	13:57:55.670

Tab. 6.11: Czas potrzebny na wygenerowanie sceny "Lustra", platforma II, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
CUDA-3.1	231.42%	228.80%	230.24%	228.99%	230.63%	228.91%	228.81%	188.40%

Tab. 6.12: Porównanie czasu renderu sceny "Lustra" pomiędzy wersją CPU, a CUDA-3.1, platforma II, głębokość 10.

Labirynt

Platforma I

Dla sceny "Labirynt" wersja CUDA-3.1 jest wolniejsza od wersji bazowej. Analizując dotychczasowe wyniki można zauważyć zależność gdzie wersja uruchamiana na GPU pogarsza swoje wyniki w stosunku do wersji CPU im więcej obiektów znajduje się na scenie.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:01:18.306	00:02:36.515	00:06:30.393	00:13:12.198
CUDA-3.1	00:02:31.267	00:05:03.884	00:12:36.741	00:25:13.620

Tab. 6.13: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 40 - 400.

Średnio wersja obecna wersja jest wolniejsza o 92.87% od wersji CPU. Jest to znaczna porawa w stosunku do wersji CUDA-3, która była wolniejsza o 273.45%.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	00:33:08.371	01:04:28.290	02:43:52.173	05:26:06.177
CUDA-3.1	01:02:58.585	02:05:53.287	05:14:31.135	10:31:01.279

Tab. 6.14: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
CUDA-3.1	193.17%	194.16%	193.84%	191.07%	190.03%	195.26%	191.93%	193.50%

Tab. 6.15: Porównanie czasu renderu sceny "Labirynt" pomiędzy wersją CPU, a CUDA-3.1, platforma I, głębokość 10.

Platforma II

Dla platformy II scena "Labirynt" także zwiększa dystans od wersji CPU. Jest ono średnio o 267.47% wolniejsza.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:02:28.903	00:05:04.538	00:12:49.285	00:25:35.789
CUDA-3.1	00:09:14.845	00:18:28.230	00:46:51.675	01:34:12.842

Tab. 6.16: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 40 - 400.

Niestety pomimo poprawek w stosunku do wersji CUDA-3 w dalszym ciągu jeden z testów nie zakończył się ze względu na przekroczenie limitu (W przypadku wersji CUDA-3 dwa testy przekroczyły limit czasu). Niestety brak wyników z tego testu nie pozwala nam stwierdzić czy trend skrócenia czasu renderu dla ostatniego testu utrzymał by się też dla tego przypadku.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	01:04:02.628	02:07:38.369	05:21:00.246	11:36:22.172
CUDA-3.1	03:55:24.730	07:48:37.679	21:13:25.633	K/C

Tab. 6.17: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
CUDA-3.1	372.62%	363.91%	365.49%	368.07%	367.58%	367.15%	396.70%	K/C

Tab. 6.18: Porównanie czasu renderu sceny "Labirynt" pomiędzy wersją CPU, a CUDA-3.1, platforma II, głębokość 10.

6.1.2. CUDA-3.2

W poprzednich wersjach promień był odbijany dalej po trafieniu w źródło światła. Było to nieefektywne ponieważ nie wносиło już żadnej nowej informacji do końcowego obrazu. W związku z tym wersja ta skupia się na optymalizacji obsługi źródeł światła.

Cornell Box

Platforma I

W przypadku pierwszej sceny zmiany te nie dały zauważalnych zmian w kodzie. Różnica pomiędzy wersją CUDA-3.1, a CUDA-3.2 wynosi mniej niż 1% w stosunku do wersji CPU. Nie możemy też wskazać wersji.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:00:34.133	00:01:13.430	00:02:55.697	00:05:43.358
CUDA-3.2	00:00:28.556	00:00:57.122	00:02:22.579	00:04:44.695

Tab. 6.19: Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma I, głębokość 10, próbki 40 - 400.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	00:14:50.694	00:29:02.565	01:12:05.939	02:26:13.639
CUDA-3.2	00:11:51.791	00:23:48.729	00:59:04.412	01:59:06.269

Tab. 6.20: Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma I, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
CUDA-3.2	83.66%	77.79%	81.15%	82.91%	79.91%	81.99%	81.93%	81.45%

Tab. 6.21: Porównanie czasu renderu sceny "Cornell Box" pomiędzy wersją CPU, a CUDA-3.2, platforma I, głębokość 10.

Platforma II

Sytuacja zmienia się dla platformy II. Już dla pierwszej partii wyników widzimy znacznie mniejszą różnicę do wersji CPU.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:00:54.779	00:01:49.986	00:04:37.793	00:09:12.610
CUDA-3.2	00:01:43.286	00:03:27.610	00:08:37.355	00:17:14.533

Tab. 6.22: Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 40 - 400.

Średnio wersja CUDA-3.2 jest wolniejsza o 85.07% od wersji CPU - dla wersji CUDA-3.1 wartość ta wynosiła 103.38%. Co ciekawe zmiany wprowadzone w obecnej nie miały żadnego wpływu na wynik testu dla 10000 próbek. Jest on wolniejszy o 67.21% w stosunku do wersji CPU. Jest to tylko nieco ponad 1% różnicy w stosunku do wyniku wersji CUDA-3.1.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	00:23:05.738	00:46:17.228	01:55:07.566	04:21:15.404
CUDA-3.2	00:43:08.714	01:26:44.900	03:36:49.972	07:16:50.171

Tab. 6.23: Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
CUDA-3.2	188.55%	188.76%	186.24%	187.21%	186.81%	187.41%	188.34%	167.21%

Tab. 6.24: Porównanie czasu renderu sceny "Cornell Box" pomiędzy wersją CPU, a CUDA-3.2, platforma II, głębokość 10.

Lustra

Platforma I

Podobnie jak w przypadku sceny "Cornell Box" nie ma żadnych zmian w stosunku do wersji CUDA-3.1.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:01:02.837	00:02:06.880	00:04:51.809	00:09:29.411
CUDA-3.2	00:00:53.453	00:01:46.810	00:04:26.977	00:08:53.240

Tab. 6.25: Czas potrzebny na wygenerowanie sceny "Lustra", platforma I, głębokość 10, próbki 40 - 400.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	00:23:59.375	00:47:04.660	02:01:29.943	03:59:41.314
CUDA-3.2	00:22:11.579	00:44:20.477	01:50:47.381	03:41:30.905

Tab. 6.26: Czas potrzebny na wygenerowanie sceny "Lustra", platforma I, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
CUDA-3.2	85.07%	84.18%	91.49%	93.65%	92.51%	94.19%	91.19%	94.42%

Tab. 6.27: Porównanie czasu renderu sceny "Lustra" pomiędzy wersją CPU, a CUDA-3.2, platforma I, głębokość 10.

Wersja	Ilość próbek								Średnie
	40	80	200	400	1000	2000	5000	10000	
CPU	57.27	57.39	57.32	57.00	57.64	57.60	57.37	57.27	57.36
CUDA-3.2	131.70	131.70	131.70	131.70	131.57	131.70	131.57	131.19	131.68

Tab. 6.28: Zużycie pamięci systemowej dla sceny "Lustra", platforma I, głębokość 10, dane w MiB.

Platforma II

Dla platformy II znowu widzimy poprawę wyników w stosunku do wersji CUDA-3.1 Średnio 19.12%. Jest to wartość minimalnie większa od średniej dla sceny "Cornell Box", która wyniosła 18.31%.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:01:34.121	00:03:10.404	00:07:53.938	00:15:52.267
CUDA-3.2	00:03:16.731	00:06:35.562	00:16:27.549	00:32:50.846

Tab. 6.29: Czas potrzebny na wygenerowanie sceny "Lustra", platforma I, głębokość 10, próbki 40 - 400.

Widzimy także podobne zachowanie jak w przypadku sceny "Cornell Box" gdzie poprawki dodane w tej wersji nie mają wpływu na wynik testów dla 10000 próbek.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	00:39:24.280	01:19:29.657	03:18:39.520	07:24:46.301
CUDA-3.2	01:22:12.220	02:43:53.694	06:50:53.303	14:02:55.746

Tab. 6.30: Czas potrzebny na wygenerowanie sceny "Lustra", platforma I, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
CUDA-3.2	209.02%	207.75%	208.37%	206.96%	208.61%	206.17%	206.83%	189.52%

Tab. 6.31: Porównanie czasu renderu sceny "Lustra" pomiędzy wersją CPU, a CUDA-3.2, platforma I, głębokość 10.

Labirynt

Platforma I

Dla sceny labirynt w przypadku platformy I w dalszym ciągu nie obserwujemy zmian względem wersji CUDA-3.1.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:01:18.306	00:02:36.515	00:06:30.393	00:13:12.198
CUDA-3.2	00:02:30.705	00:04:59.767	00:12:28.837	00:24:46.480

Tab. 6.32: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 40 - 400.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	00:33:08.371	01:04:28.290	02:43:52.173	05:26:06.177
CUDA-3.2	01:01:53.252	02:03:43.995	05:11:11.105	10:22:37.490

Tab. 6.33: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
CUDA-3.2	192.46%	191.53%	191.82%	187.64%	186.75%	191.92%	189.90%	190.93%

Tab. 6.34: Porównanie czasu renderu sceny "Labirynt" pomiędzy wersją CPU, a CUDA-3.2, platforma I, głębokość 10.

Platforma II

Scena "Labirynt" w dalszym ciągu osiąga lepsze wyniki niż wersja CUDA-1. Różnica ta jednak spada wraz z wzrostem złożoności sceny testowej. Dla wersji CUDA-3.1 była ona wolniejsza o 267.47% w porównaniu do wersji CPU. Obecnie testowana wersja jest wolniejsza o 258.33%. Jest to tylko 9.14% różnicy pomiędzy obiema wersjami.

Wersja	Ilość próbek			
	40	80	200	400
CPU	00:02:28.903	00:05:04.538	00:12:49.285	00:25:35.789
CUDA-3.2	00:09:14.845	00:18:28.230	00:46:51.675	01:34:12.842

Tab. 6.35: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 40 - 400.

Poprawi w stosunku do wersji CUDA-1 nie były wystarczające żeby pozwolić na dokończenie testu dla 10000 próbek w limicie czasowym. Nie jesteśmy w stanie przez to zaobserwować czy brak poprawy w wersji CUDA-2 dla tego testu miał by także miejsce w przypadku tej sceny.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CPU	01:04:02.628	02:07:38.369	05:21:00.246	11:36:22.172
CUDA-3.2	03:55:24.730	07:48:37.679	21:13:25.633	K/C

Tab. 6.36: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CPU	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
CUDA-3.2	367.94%	359.13%	355.54%	356.11%	355.86%	357.13%	356.60%	K/C

Tab. 6.37: Porównanie czasu renderu sceny "Labirynt" pomiędzy wersją CPU, a CUDA-3.2, platforma II, głębokość 10.

6.1.3. Podsumowanie

Wersja CUDA-3.2 jest bardzo zbliżona w osiągniętych wynikach do wersji CUDA-3.1 w przypadku platformy I. Jeżeli spojrzymy na dane przedstawione w tabeli 6.38 wygląda to zupełnie inaczej dla II platformy testowej. Szczególnie dla sceny "Cornell Box" i "Lustra" obserwujemy przewagę na poziomie kilkunastu procent na rzecz wersji CUDA-3.2. Różnica ta jest także widoczna dla sceny "Labirynt" jednak w jej przypadku wynosi ona tylko kilka procent. Ze względu na tą przewagę w przypadku II platformy testowej wersja CUDA-3.2 została wybrana jako reprezentant dla klasycznego modelu oświetlenia.

Scena	Platforma I		Platforma II	
	CUDA-3.1	CUDA-3.2	CUDA-3.1	CUDA-3.2
Cornell Box	81.72%	81.35%	203.38%	185.07%
Lustra	90.94%	90.59%	229.69%	207.67%
Labirynt	192.87%	190.37%	371.65%	358.33%

Tab. 6.38: Porównanie średniej zmiany wyniku dla wersji CUDA-3.1 i CUDA-3.2 względem wersji CPU, głębokość 10.

6.2. Model hybrydowy

Model hybrydowy oświetlenia działa w bardzo podobny sposób do modelu klasycznego. Jednak w przypadku gdy dany promień odbijając się losowo na scenie nie trafił w źródło w światła wyszukuje on dodatkowo czy pierwszy punkt, w który promień wysłany z ekranu jest oświetlony przez źródło światła. Jeżeli jest jest ta informacja dodawana do końcowego obrazu. Unika to sytuacji, w której promień wysłany z kamery odbija się na scenie nie wnosząc nowej informacji do końcowego obrazu. Zachowanie to też jest wykonywane dla odbić w lustrze jeżeli pierwszym obiektem, w który trafił promień po wysłaniu z kamery było właśnie lustro.

Dla tego modelu oświetlenia powstały dwie wersje programu *tracer*:

- CUDA.3.3 - początkowa implementacja tego modelu oświetlenia, bazuje na wersji CUDA-3.2.
- CUDA-3.4 - wprowadza sortowanie ekstremów w punktach oświetlenia (punktów względem, których sprawdzane jest czy dane źródło światła oświetla obiekt) względem centralnego punktu sceny.

Ponieważ obie wersje korzystają z innego modelu oświetlenia niż wersja CPU zostały one porównane tylko między sobą.

6.2.1. Cornell Box

Platforma I

Scena pierwsza jest oświetlona przez jedno źródło światła znajdujące się na samej górze sceny. Dodatkowo obiekt, który jest źródłem światła znajduje się w większości poza pudełkiem ograniczającym scenę. Tylko jedno z ekstremów względem, których sprawdzane jest oświetlenie znajduje się wewnątrz pudełka ograniczającego scenę. W tym warunkach posortowanie ekstremów pozwoliło zaoszczędzić średnio 6.16% względem wersji CUDA-3.3.

Wersja	Ilość próbek			
	40	80	200	400
CUDA-3.3	00:00:31.868	00:01:03.931	00:02:39.255	00:05:19.835
CUDA-3.4	00:00:29.957	00:01:00.170	00:02:30.127	00:04:59.601

Tab. 6.39: Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma I, głębokość 10, próbki 40 - 400.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CUDA-3.3	00:13:19.483	00:26:34.272	01:06:24.808	02:13:08.144
CUDA-3.4	00:12:27.574	00:24:55.218	01:02:11.826	02:04:42.792

Tab. 6.40: Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma I, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CUDA-3.3	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
CUDA-3.4	94.00%	94.12%	94.27%	93.67%	93.51%	93.79%	93.65%	93.67%

Tab. 6.41: Porównanie czasu renderu sceny "Cornell Box" pomiędzy wersją CUDA-3.3, a CUDA-3.4, platforma I, głębokość 10.

Platforma II

W przypadku platformy II wersja CUDA-3.4 jest szybsza o 6.91% w stosunku do wersji CUDA-3.3. Jest to różnica zbliżonej do tej zaobserwowanej w przypadku platformy I.

Wersja	Ilość próbek			
	40	80	200	400
CUDA-3.3	00:01:55.240	00:03:47.394	00:09:50.819	00:20:11.695
CUDA-3.4	00:01:46.558	00:03:30.730	00:09:14.700	00:18:49.470

Tab. 6.42: Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 40 - 400.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CUDA-3.3	00:50:32.293	01:40:53.376	04:11:18.290	08:25:09.844
CUDA-3.4	00:46:55.678	01:34:13.687	03:55:21.356	07:47:34.985

Tab. 6.43: Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CUDA-3.3	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
CUDA-3.4	92.47%	92.67%	93.89%	93.21%	92.86%	93.40%	93.65%	92.56%

Tab. 6.44: Porównanie czasu renderu sceny "Cornell Box" pomiędzy wersją CUDA-3.3, a CUDA-3.4, platforma II, głębokość 10.

6.2.2. Labirynt

Platforma I

Na scenie labirynt znajdują się dwa źródła światła. Znajdują się one na samej górze sceny, w większości poza pudełkiem, które ogranicza scenie. Jest to układ bardzo podobny jak w przy-

padku sceny "Cornell Box". Wersja CUDA-3.4 jest szybsza średnio o 6.56% w stosunku do wersji CUDA-3.3.

Wersja	Ilość próbek			
	40	80	200	400
CUDA-3.3	00:01:05.973	00:02:11.122	00:05:28.467	00:10:55.894
CUDA-3.4	00:01:01.899	00:02:02.345	00:05:06.408	00:10:14.589

Tab. 6.45: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 40 - 400.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CUDA-3.3	00:27:17.600	00:54:42.883	02:16:48.714	04:32:57.505
CUDA-3.4	00:25:33.256	00:51:02.964	02:07:34.315	04:14:28.941

Tab. 6.46: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CUDA-3.3	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
CUDA-3.4	93.82%	93.31%	93.28%	93.70%	93.63%	93.30%	93.25%	93.23%

Tab. 6.47: Porównanie czasu renderu sceny "Labirynt" pomiędzy wersją CUDA-3.3, a CUDA-3.4, platforma I, głębokość 10.

Platforma II

Dla sceny "Labirynt" wersja CUDA-3.4 jest średnio szybsza o 6.71% szybsza od wersji CUDA-3.3. Tak samo jak w przypadku platformy I, różnica pomiędzy obiema wersjami jest zbliżona dla sceny "Cornell Box" i "Labirynt".

Wersja	Ilość próbek			
	40	80	200	400
CUDA-3.3	00:03:58.651	00:07:56.861	00:19:51.598	00:41:31.132
CUDA-3.4	00:03:41.678	00:07:22.799	00:18:50.781	00:38:43.530

Tab. 6.48: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 40 - 400.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CUDA-3.3	01:44:16.494	03:27:21.171	08:38:14.876	17:22:20.643
CUDA-3.4	01:36:39.260	03:13:25.617	08:02:28.483	16:13:04.832

Tab. 6.49: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CUDA-3.3	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
CUDA-3.4	92.89%	92.86%	94.90%	93.27%	92.69%	93.28%	93.10%	93.36%

Tab. 6.50: Porównanie czasu renderu sceny "Labirynt" pomiędzy wersją CUDA-3.3, a CUDA-3.4, platforma II, głębokość 10.

6.2.3. Labirynt

Platforma I

Scena ta odchodzi od schematu bazującego na scenie "Cornell Box". Na scenie znajduje się 5 mniejszych źródeł światła. Są one w całości na scenie. Zostały one umieszczone wewnątrz labiryntu. Oznacza to, że są one przemieszane z zwykłymi obiektami na scenie i każdy z badanych ekstremów może oświetlić fragment sceny. Dodatkowo scena "Labirynt" nie jest symetryczna w związku z czym środek sceny nie wypada idealnie na środku pola widzenia kamery.

Wersja	Ilość próbek			
	40	80	200	400
CUDA-3.3	00:04:57.210	00:09:55.520	00:24:44.310	00:49:05.468
CUDA-3.4	00:04:59.558	00:09:53.710	00:24:37.259	00:49:14.734

Tab. 6.51: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 40 - 400.

W tym wypadku wersja CUDA-3.4 jest średnio wolniejsza o 0.10% od wersji CUDA-3.3. Jeżeli przyjrzymy się wynikom dokładnie zauważymy, że w 4 przypadkach (testy dla 80, 200, 5000 oraz 10000 próbek) wersja CUDA-3.4 jest szybsza. Natomiast dla drugiej połowy testów (dla 40, 400, 1000 oraz 2000) jest ona wolniejsza.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CUDA-3.3	02:03:16.385	04:06:37.226	10:16:20.166	20:33:43.758
CUDA-3.4	02:03:48.930	04:07:34.609	10:14:31.348	20:33:01.214

Tab. 6.52: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CUDA-3.3	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
CUDA-3.4	100.79%	99.70%	99.52%	100.31%	100.44%	100.39%	99.71%	99.94%

Tab. 6.53: Porównanie czasu renderu sceny "Labirynt" pomiędzy wersją CUDA-3.3, a CUDA-3.4, platforma I, głębokość 10.

Platforma II

W przypadku platformy II widzimy minimalnie lepsze wyniki. Średnio wersja CUDA-3.4 jest szybsza o 0.45% od wersji CUDA-3.3. Dla 6 testów, które ukończyły się w limicie czasu, w 5 wersja CUDA-3.4 była szybsza. Jest to jednak minimalna różnica pomiędzy obiema wersjami.

Wersja	Ilość próbek			
	40	80	200	400
CUDA-3.3	00:18:29.667	00:37:13.456	01:32:52.779	03:05:29.370
CUDA-3.4	00:18:24.163	00:36:50.298	01:32:16.787	03:04:28.350

Tab. 6.54: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 40 - 400.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CUDA-3.3	07:40:52.384	15:21:36.891	K/C	K/C
CUDA-3.4	07:42:16.778	15:19:19.248	K/C	K/C

Tab. 6.55: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CUDA-3.3	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	K/C	K/C
CUDA-3.4	99.50%	98.96%	99.35%	99.45%	100.31%	99.75%	K/C	K/C

Tab. 6.56: Porównanie czasu renderu sceny "Labirynt" pomiędzy wersją CUDA-3.3, a CUDA-3.4, platforma II, głębokość 10.

6.2.4. Podsumowanie

W tabeli 6.57 przedstawiono podsumowanie porównań pomiędzy obiema wersjami. Jak widzimy w większości przypadków wersja CUDA-3.4 jest szybsza od wersji CUDA-3.3. Dla sceny "Cornell Box" i "Lustra" różnica ta jest znacząca - na poziomie kilku procent. W przypadku sceny "Labirynt" zmiany wprowadzone w wersji CUDA-3.4 nie przynoszą zauważalnych różnic. Zarówno w przypadku Platformy I jak i Platformy II różnica w stosunku do wersji CUDA-3.3 pozostaje poniżej 1%.

Scena	Platforma I	Platforma II
Cornell Box	93.84%	93.09%
Lustra	93.44%	93.29%
Labirynt	100.10%	99.55%
Średnia	95.79%	95.31%

Tab. 6.57: Średni wynik sceny CUDA-3.4 względem sceny CUDA-3.3.

Ostatecznie wersja CUDA-3.4 została wybrana jako reprezentant hybrydowego modelu oświetlenia do ostatecznego porównania.

6.3. Model rozszerzony

W przypadku tego modelu dla każdego obiektu trafionego przez promień sprawdzane jest czy dany punkt jest oświetlony przez źródło światła. W przeciwieństwie do modelu hybrydowego sprawdzenie nie jest przerywane po znalezieniu pierwszego źródła światła, które spełnia ten

warunek. Efekt ten jest sumowany dla elementów znajdujących na liście obiektów emitujących światło. Ponadto operacja ta jest wykonywana dla każdego odbicia, nie tylko dla promieni wyemitowanych z kamery.

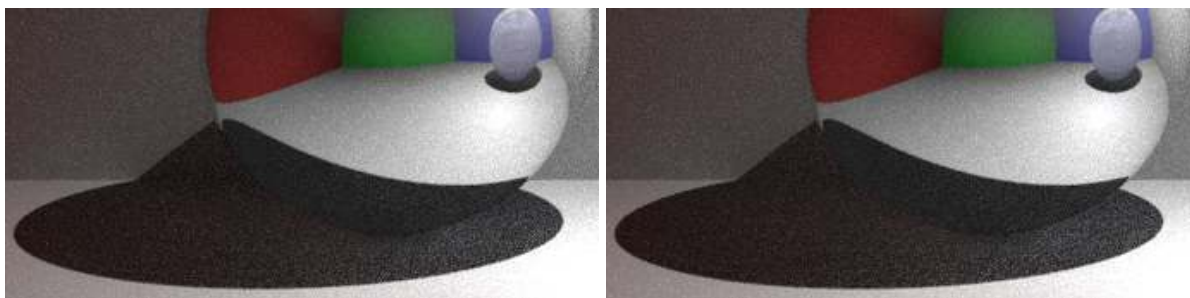
Podobnie jak w przypadku modelu hybrydowego powstały dwie wersje programu *tracer* implementujące tą metodę obsługi oświetlenia.

- CUDA-3.5 - początkowa implementacja, bazuje na wersji CUDA-3.2,
- CUDA-3.6 - zmiana obsługi odbić w częściach sceny nie oświetlonych bezpośrednio przez źródło światła.

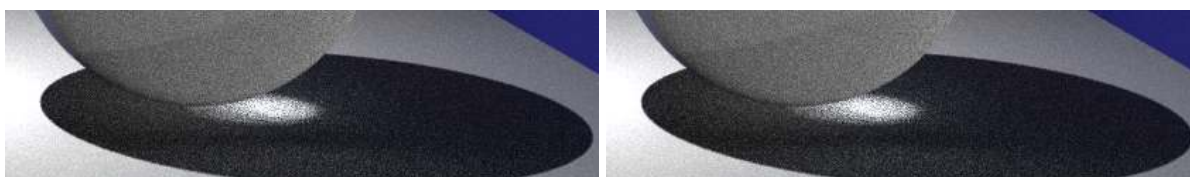
Porównanie w przypadku tych dwóch wersji wymaga szczególnej uwagi, ponieważ zmiana wprowadzona w wersji CUDA-3.4 wpływa nie tylko na osiągi aplikacji ale także na wygląd wygenerowanego obrazu.

6.3.1. Cornell Box

Na scenie "Cornell Box" jest bardzo mało zacienionych elementów. Elementy, na które powinniśmy zwrócić uwagę są widoczne pod obiema kulami oraz dodatkowo w lustrzanej kuli pod odbiciem szklanej kuli. Pierwsze porównanie wykonano dla testu dla 40 próbek. Porównania interesujących na elementów zostały przedstawione na rysunkach 6.1 oraz 6.2. Na razie nie widzimy różnicy pomiędzy obiema wersjami.

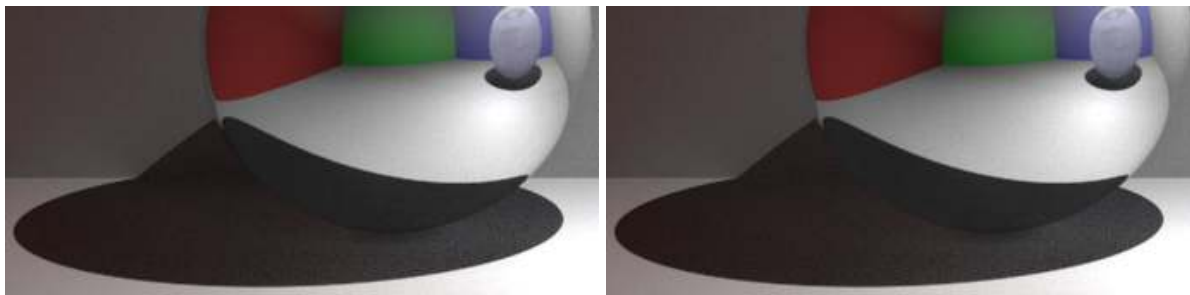


Rys. 6.1: Porównanie jakości renderów dla sceny "Cornell Box" pomiędzy wersją CUDA-3.5 i CUDA-3.6. Zbliżenie na lustrzaną kulę, 40 próbek, głębokość 10.



Rys. 6.2: Porównanie jakości renderów dla sceny "Cornell Box" pomiędzy wersją CUDA-3.5 i CUDA-3.6. Zbliżenie na szklaną kulę, 40 próbek, głębokość 10.

Kolejne porównanie wykonano przy teście dla 400 próbek. Porównanie tych samych elementów sceny przedstawiono rysunkach 6.3 oraz 6.4. Dla 400 próbek widzimy znacznie mniej szumów na końcowym obrazie niż dla 40 próbek. Niestety w dalszym ciągu nie możemy wskazać znaczącej różnicy pomiędzy wersją CUDA-3.3, a CUDA-3.4.

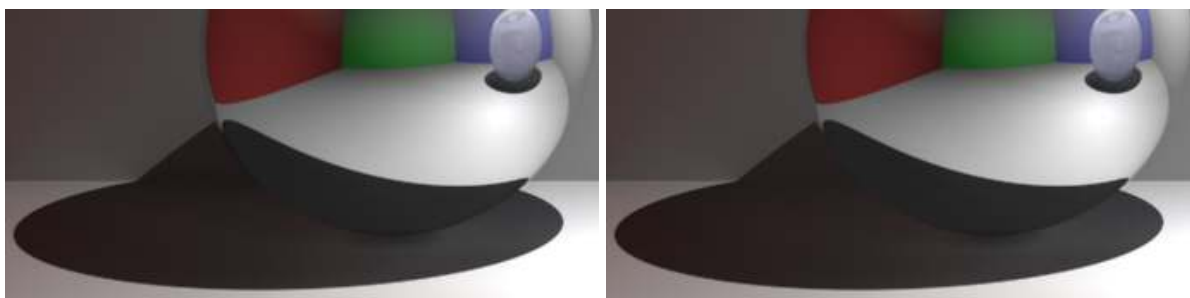


Rys. 6.3: Porównanie jakości rendererów dla sceny "Cornell Box" pomiędzy wersją CUDA-3.5 i CUDA-3.6. Zbliżenie na lustrzaną kulę, 400 próbek, głębokość 10.



Rys. 6.4: Porównanie jakości rendererów dla sceny "Cornell Box" pomiędzy wersją CUDA-3.5 i CUDA-3.6. Zbliżenie na szklaną kulę, 400 próbek, głębokość 10.

Ostatnie wizualne porównanie przedstawiono na rysunkach 6.5 i 6.6 dla 5000 próbek. Ponownie widzimy poprawę jakości związaną z zwiększeniem ilości próbek ale brak różnic pomiędzy testowanymi wersjami.



Rys. 6.5: Porównanie jakości rendererów dla sceny "Cornell Box" pomiędzy wersją CUDA-3.5 i CUDA-3.6. Zbliżenie na lustrzaną kulę, 5000 próbek, głębokość 10.



Rys. 6.6: Porównanie jakości rendererów dla sceny "Cornell Box" pomiędzy wersją CUDA-3.5 i CUDA-3.6. Zbliżenie na szklaną kulę, 5000 próbek, głębokość 10.

Platforma I

Wersja CUDA-3.6 jest średnio wolniejsza o 5.69% od wersji CUDA-3.5.

Wersja	Ilość próbek			
	40	80	200	400
CUDA-3.5	00:00:28.625	00:00:57.840	00:02:22.338	00:04:44.464
CUDA-3.6	00:00:31.534	00:01:00.400	00:02:29.395	00:04:59.770

Tab. 6.58: Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma I, głębokość 10, próbki 40 - 400.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CUDA-3.5	00:11:49.180	00:23:39.650	00:58:55.766	01:58:15.782
CUDA-3.6	00:12:28.194	00:24:53.800	01:01:58.359	02:03:51.912

Tab. 6.59: Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma I, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CUDA-3.5	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
CUDA-3.6	110.16%	104.43%	104.96%	105.38%	105.50%	105.22%	105.16%	104.74%

Tab. 6.60: Porównanie czasu renderu sceny "Cornell Box" pomiędzy wersją CUDA-3.5, a CUDA-3.6, platforma I, głębokość 10.

Platforma II

W tym przypadku obserwujemy wzrost różnicy pomiędzy wersjami wraz z zmianą ilości próbek. Początkowo wersja CUDA-3.6 jest wolniejsza o 5.27%. Jest to różnica zbliżona do średniej w przypadku platformy I. Jednak dla ostatniego przetestowanego przypadku opóźnienie to wynosi już 19.70%. Średnio sprawia to, że dla platformy II wersja CUDA-3.6 jest wolniejsza o 109.37%.

Wersja	Ilość próbek			
	40	80	200	400
CUDA-3.5	00:01:44.468	00:03:25.544	00:08:33.849	00:17:07.352
CUDA-3.6	00:01:49.976	00:03:36.600	00:09:00.973	00:18:37.176

Tab. 6.61: Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 40 - 400.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CUDA-3.5	00:42:48.653	01:25:37.212	03:34:01.270	07:19:16.967
CUDA-3.6	00:46:28.436	01:32:53.315	04:03:04.494	08:45:49.825

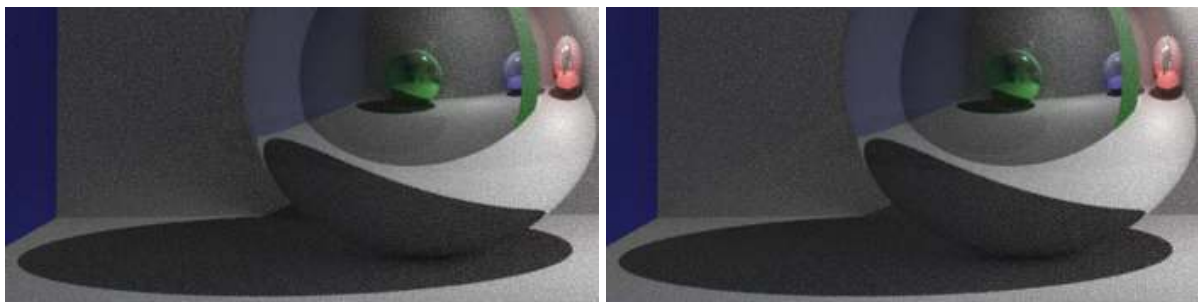
Tab. 6.62: Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CUDA-3.5	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
CUDA-3.6	105.27%	105.38%	105.28%	108.74%	108.56%	108.49%	113.58%	119.70%

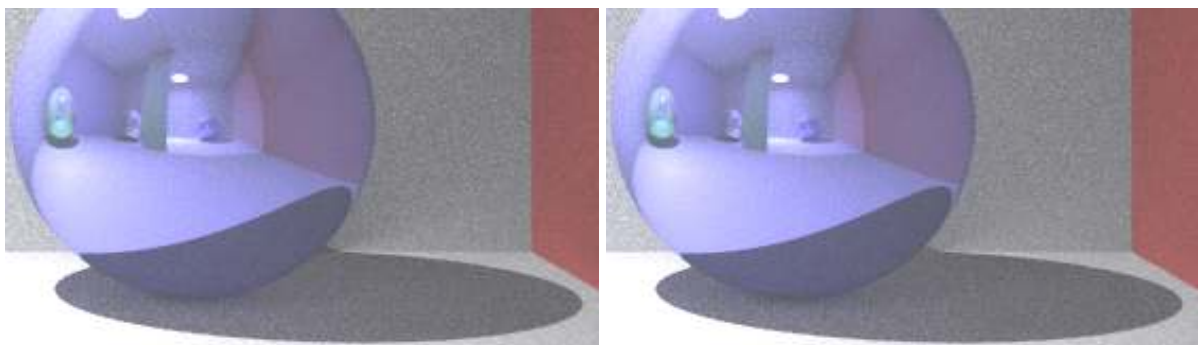
Tab. 6.63: Porównanie czasu renderu sceny "Cornell Box" pomiędzy wersją CUDA-3.5, a CUDA-3.6, platforma II, głębokość 10.

6.3.2. Lustra

Dla sceny "Lustra" zacienione miejsca znajdują się także pod kulami. Jest ich jednak więcej niż w przypadku sceny "Cornell Box" i żadna nie jest widziana bezpośrednio przez kamerę. Po lewej stronie kula znajduje się za taflą szkła, natomiast po lewej stronie widzimy obraz odbity w lustrze. Każda kula znajdująca się scenie ma właściwości lustra w związku z czym widzimy obraz, który często był wielokrotnie odbity. Pierwsze porównanie wykonano dla 40 próbek. Zostało ono przedstawione na rysunkach 6.7 oraz 6.8. Podobnie jak w przypadku sceny "Cornell Box" w tym wypadku także jest ciężko wskazać na różnicę pomiędzy wersjami.

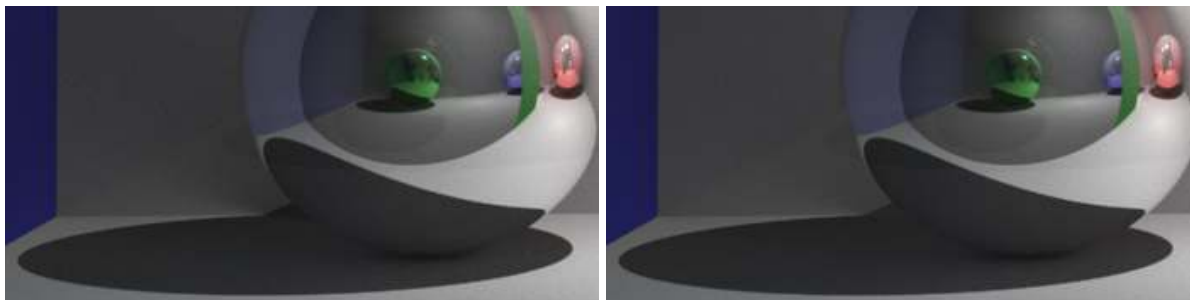


Rys. 6.7: Porównanie jakości rendererów dla sceny "Lustra" pomiędzy wersją CUDA-3.5 i CUDA-3.6. Zbliżenie na szybę, 40 próbek, głębokość 10.

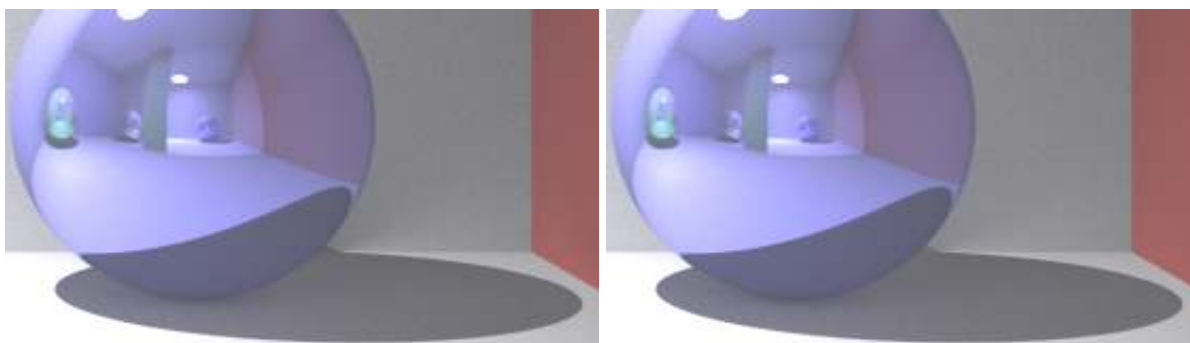


Rys. 6.8: Porównanie jakości rendererów dla sceny "Lustra" pomiędzy wersją CUDA-3.5 i CUDA-3.6. Zbliżenie na lustro, 40 próbek, głębokość 10.

Porównanie dla 400 próbek zostało przedstawione na rysunkach 6.9 oraz 6.10. Obrazy te są znacznie lepsze w porównaniu do testów dla 40 próbek. W dalszym ciągu brak wyraźniej różnicy pomiędzy wersją CUDA-3.5, a CUDA-3.6 dla tej samej ilości próbek.

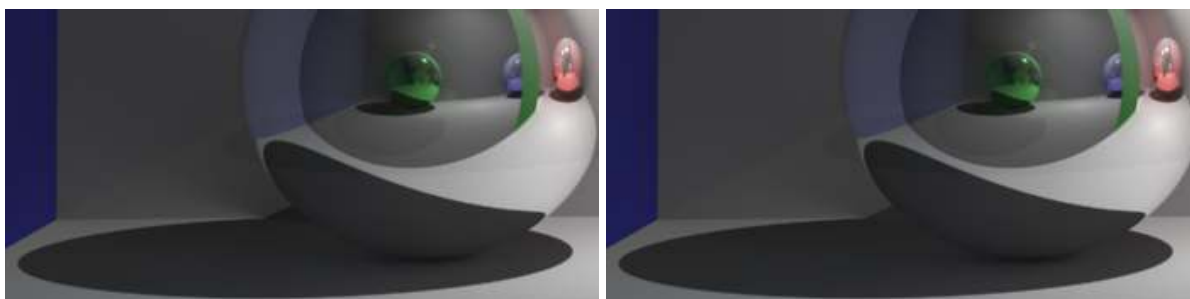


Rys. 6.9: Porównanie jakości rendererów dla sceny "Lustra" pomiędzy wersją CUDA-3.5 i CUDA-3.6. Zbliżenie na szybę, 40 próbek, głębokość 10.

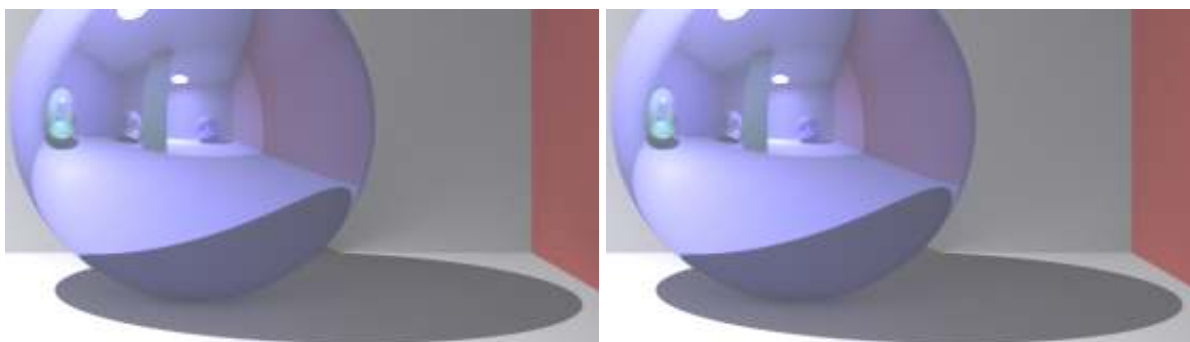


Rys. 6.10: Porównanie jakości rendererów dla sceny "Lustra" pomiędzy wersją CUDA-3.5 i CUDA-3.6. Zbliżenie na lustro, 40 próbek, głębokość 10.

Ostatnie porównanie wykonano dla 5000 próbek. Zostało one przedstawione na rysunkach 6.11 oraz 6.12. Ponownie brak widocznych różnic pomiędzy testowanymi wersjami.



Rys. 6.11: Porównanie jakości rendererów dla sceny "Lustra" pomiędzy wersją CUDA-3.5 i CUDA-3.6. Zbliżenie na szybę, 40 próbek, głębokość 10.



Rys. 6.12: Porównanie jakości rendererów dla sceny "Lustra" pomiędzy wersją CUDA-3.5 i CUDA-3.6. Zbliżenie na lustro, 40 próbek, głębokość 10.

Platforma I

Wersja CUDA-3.6 jest wolniejsza średnio o 4.01% w porównaniu do wersji CUDA-3.5. W połączeniu z brakiem widocznej różnicy w obrazie końcowym wyniki wersji CUDA-3.6 są niezadowalające.

Wersja	Ilość próbek			
	40	80	200	400
CUDA-3.5	00:02:54.552	00:05:47.904	00:14:37.430	00:28:57.693
CUDA-3.6	00:03:03.369	00:06:02.204	00:15:02.212	00:30:04.647

Tab. 6.64: Czas potrzebny na wygenerowanie sceny "Lustra", platforma I, głębokość 10, próbki 40 - 400.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CUDA-3.5	01:12:10.393	02:24:20.806	06:00:34.560	12:00:48.342
CUDA-3.6	01:15:06.000	02:30:03.738	06:15:07.847	12:31:00.214

Tab. 6.65: Czas potrzebny na wygenerowanie sceny "Lustra", platforma I, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CUDA-3.5	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
CUDA-3.6	105.05%	104.11%	102.82%	103.85%	104.06%	103.96%	104.04%	104.19%

Tab. 6.66: Porównanie czasu renderu sceny "Lustra" pomiędzy wersją CUDA-3.5, a CUDA-3.6, platforma I, głębokość 10.

Platforma II

W tym wypadku widzimy znacznie mniejsze różnice pomiędzy testowanymi wersjami niż w przypadku poprzedniej sceny. Co ciekawe dla test dla próbek 2000 i 5000 wersja CUDA-3.6 osiąga lepsze wyniki od wersji CUDA.3.5. Sprawia to, że pierwsza z tych wersji ukończyła jeden test więcej. Po przeanalizowaniu wyników dla Platformy I wyniki te są zaskakujące. Jednakże całościowo wersja CUDA-3.6 jest wolniejsza o 3.79% od wersji CUDA-3.5.-

Wersja	Ilość próbek			
	40	80	200	400
CUDA-3.5	00:10:34.110	00:21:07.887	00:52:49.276	01:46:01.611
CUDA-3.6	00:11:02.577	00:22:04.614	00:56:13.184	01:52:22.500

Tab. 6.67: Czas potrzebny na wygenerowanie sceny "Lustra", platforma II, głębokość 10, próbki 40 - 400.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CUDA-3.5	04:29:21.290	09:36:56.207	K/C	K/C
CUDA-3.6	04:40:33.306	09:20:47.245	23:23:47.238	K/C

Tab. 6.68: Czas potrzebny na wygenerowanie sceny "Lustra", platforma II, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CUDA-3.5	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	K/C	K/C
CUDA-3.6	104.49%	104.47%	106.43%	105.99%	104.16%	97.20%	B/D	K/C

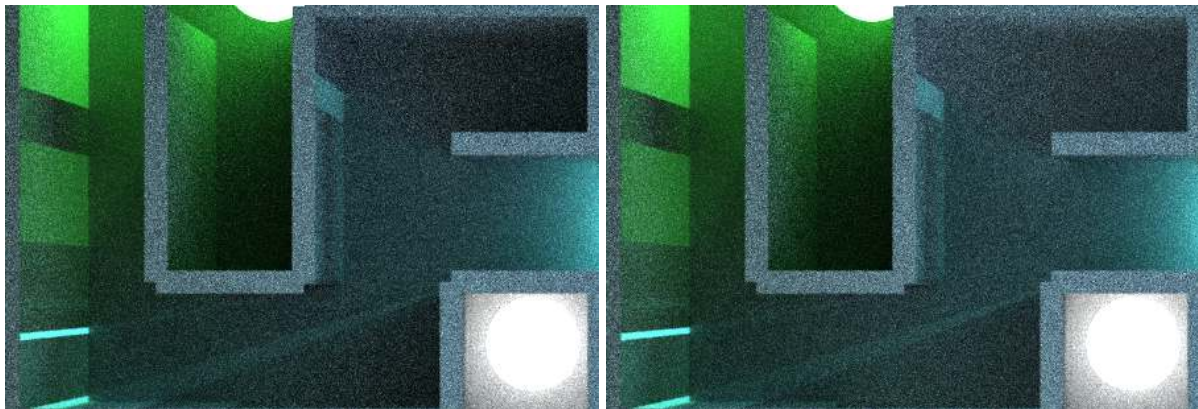
Tab. 6.69: Porównanie czasu renderu sceny "Lustra" pomiędzy wersją CUDA-3.5, a CUDA-3.6, platforma II, głębokość 10.

6.3.3. Labirynt

Scena labirynt zbudowana jest w celu zmaksymalizowania zakamarków, do których nie dociera źródło światła. Wersja CUDA-3.6 powstała głównie w celu poprawienia wyników na tej scenie. W celach porównawczych zostały wyróżnione trzy elementy sceny:

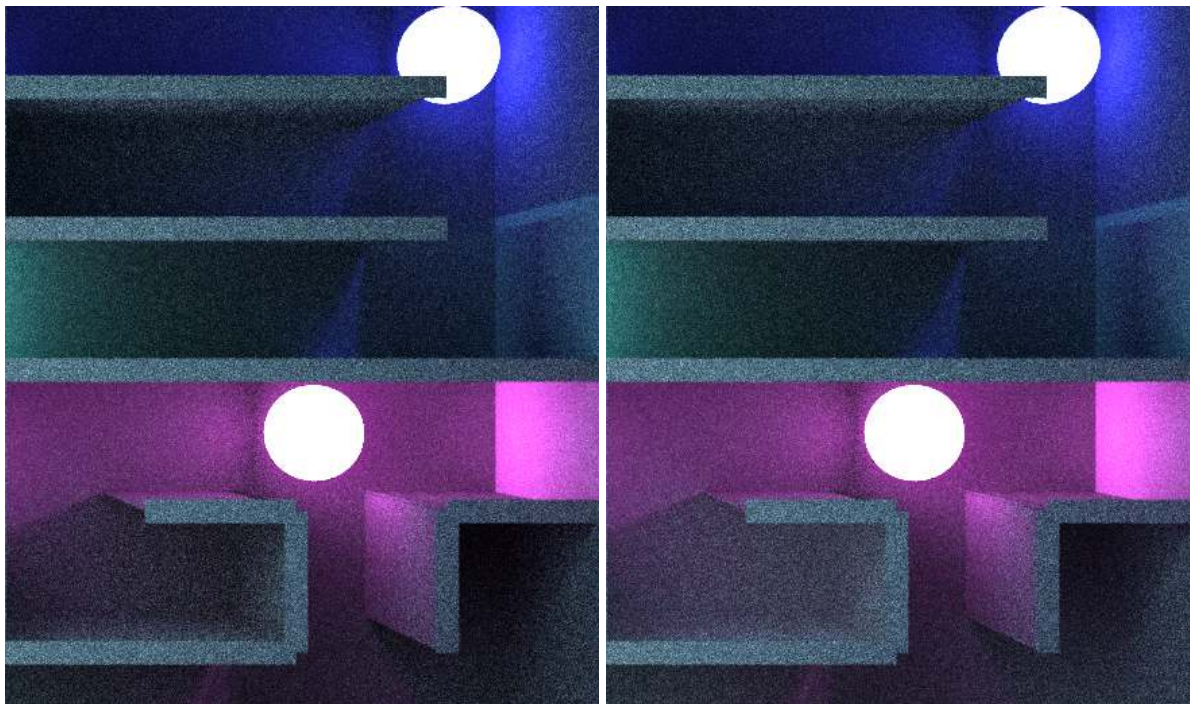
- Korytarz południowo-zachodni,
- Obszar centralny,
- Obszar wschodni.

Zostały one na przedstawione na rysunkach 6.13, 6.14 oraz 6.15. W tym wypadku jesteśmy w stanie wskazać konkretne różnice pomiędzy wersjami. Dla obszaru centralnego, ślepy zaułek znajdujący się nad białym źródłem jest zdecydowanie jaśniejszy niż w wersji CUDA-3.5. Efekt ten jednak wygląda nienaturalnie w porównaniu do korytarza znajdującego się pod zielonym źródłem światła. Jest on jaśniejszy niż obszar oświetlony bezpośrednio. Dodatkowo różnica pomiędzy ścianą, a sufitem w labiryncie przestała być widoczna. Cienie znajdujące się w korytarzu na samym dole wycinka są także jaśniejsze.



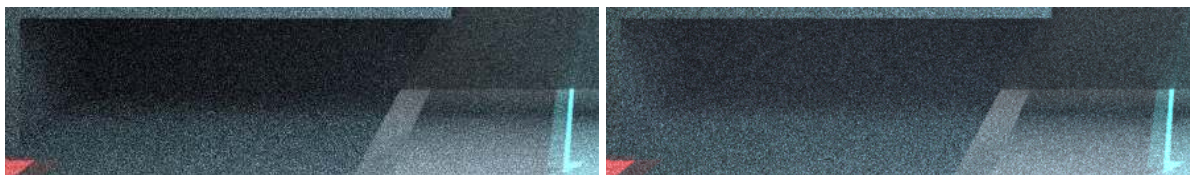
Rys. 6.13: Porównanie jakości rendererów dla sceny "Labirynt" pomiędzy wersją CUDA-3.5 i CUDA-3.6. Zbliżenie na obszar centralny, 40 próbek, głębokość 10.

W przypadku części wschodniej pierwszą różnicą jest korytarz znajdujący się pod po lewej stronie pod fioletowym źródłem światła. Niestety zamiast pomóc wydobyć to więcej szczegółów obraz korytarz ten w wersji CUDA-3.6 jest bardziej niewyraźny. Lepszy efekt jest widoczny w zakamarku znajdującym się na tej samej wysokości po prawej stronie. Efekt jest minimalny możemy jednak zobaczyć więcej szczegółów w ścianie. Ostatnim elementem tego fragmentu sceny jest korytarz znajdujący się pod niebieskim światłem. Widzimy w nim minimalnie więcej światła. Nie wpłynęło to jednak negatywnie ani pozytywnie na ilość szczegółów widocznych w tym korytarzu.



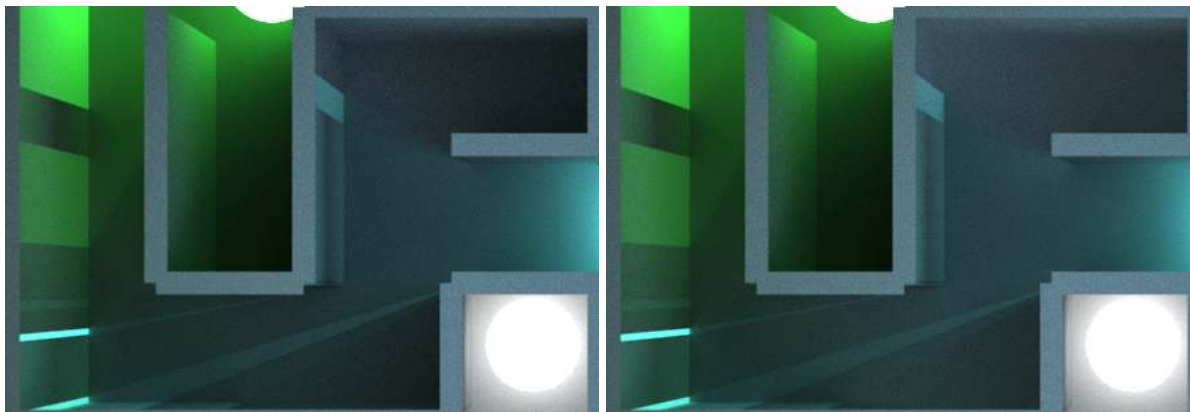
Rys. 6.14: Porównanie jakości rendererów dla sceny "Labirynt" pomiędzy wersją CUDA-3.5 i CUDA-3.6. Zbliżenie na obszar wschodni, 40 próbek, głębokość 10.

Ostatnim elementem sceny analizowanym tutaj jest korytarz południowo-zachodni. W obrazie wygenerowanym przez wersję CUDA-3.6 jest on jaśniejszy ale szczegóły (granica pomiędzy ścianami i podłogą) są ze sobą bardziej zlane.



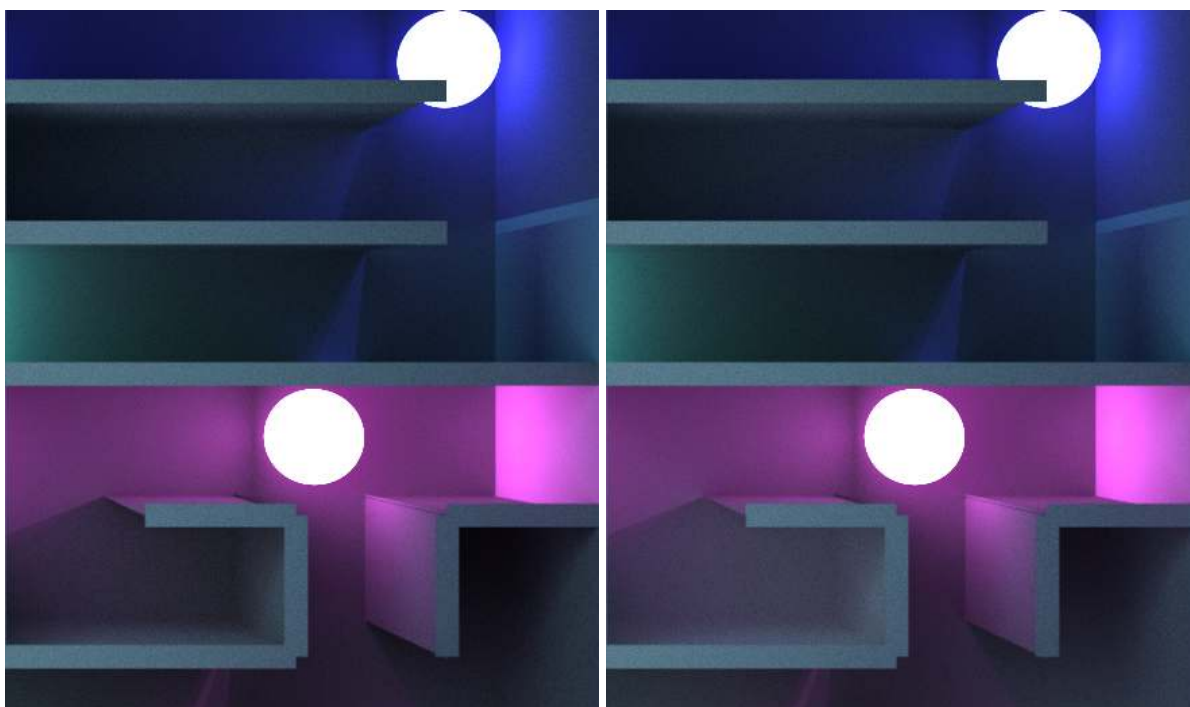
Rys. 6.15: Porównanie jakości rendererów dla sceny "Labirynt" pomiędzy wersją CUDA-3.5 i CUDA-3.6. Zbliżenie na korytarz południowo-wschodni, 40 próbek, głębokość 10.

Kolejne porównanie zostało przeprowadzone dla 1000 próbek. Wartość ta została wybrana jako kompromis ponieważ jest to maksymalna ilość próbek, dla której obie wersje ukończyły działanie w limicie czasu. Wyniki zostały przedstawione na rysunkach 6.16, 6.17 oraz 6.18. Wersja CUDA-3.6 uległa zdecydowanej poprawie w stosunku do wersji dla 40 próbek. W ślepych zaułku nad białym źródłem światła ponownie widzimy granicę pomiędzy ścianami, a podłogą. Różnica ta jest jednak znacznie lepiej widoczna w wersji CUDA-3.5. Dodatkowo zaułek ten pozostaje zbyt jasny w stosunku do korytarza pod zielonym źródłem światła.



Rys. 6.16: Porównanie jakości rendererów dla sceny "Labirynt" pomiędzy wersją CUDA-3.5 i CUDA-3.6. Zbliżenie na obszar centralny, 1000 próbek, głębokość 10.

Korytarz znajdujący się na lewo pod źródłem światła jest zbyt jasny w porównaniu do zaułka znajdującego się na tej samej wysokości po prawej stronie. Drugi od góry korytarz jest wyraźnie jaśniejszy niż w wersji CUDA-3.5. Pomimo tego nie poprawiło ilości szczegółów widocznych w tym korytarzu.



Rys. 6.17: Porównanie jakości rendererów dla sceny "Labirynt" pomiędzy wersją CUDA-3.5 i CUDA-3.6. Zbliżenie na obszar wschodni, 1000 próbek, głębokość 10.

Korytarz południowo-zachodni odzyskał szczegóły na granicy ścian i podłogi. Porównując obie wersje wyrwane z kontekstu można by się pokusić, że w tym wypadku wersja CUDA-3.6 wygląda lepiej. Korytarz ten jednak nie jest oświetlony bezpośrednio przez żadne źródło światła. Jest on zbyt jasny. Wystarczy porównać to do obszaru centralnego.



Rys. 6.18: Porównanie jakości rendererów dla sceny "Labirynt" pomiędzy wersją CUDA-3.5 i CUDA-3.6. Zbliżenie na korytarz południowo-wschodni, 1000 próbek, głębokość 10.

Platforma I

Wersja CUDA-3.6 jest średnio wolniejsza o 45.78% od wersji CUDA-3.5. Różnica jest na tyle duża, że w przypadku tej sceny wersja CUDA-3.5 kończy o jeden więcej test.

Wersja	Ilość próbek			
	40	80	200	400
CUDA-3.5	00:24:06.207	00:47:50.761	02:00:07.164	04:00:12.717
CUDA-3.6	00:34:52.409	01:09:44.618	02:54:46.740	05:48:48.908

Tab. 6.70: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 40 - 400.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CUDA-3.5	09:57:36.645	19:56:42.946	K/C	K/C
CUDA-3.6	14:42:45.476	K/C	K/C	K/C

Tab. 6.71: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CUDA-3.5	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	K/C	K/C
CUDA-3.6	144.68%	145.77%	145.50%	145.21%	147.71%	K/C	K/C	K/C

Tab. 6.72: Porównanie czasu renderu sceny "Labirynt" pomiędzy wersją CUDA-3.5, a CUDA-3.6, platforma I, głębokość 10.

Platforma II

Wyniki zarówno wersji CUDA-3.5 jak i CUDA-3.6 są jednymi z najgorszych do tej pory. Żadna z testowanych wersji nie ukończyła żadnego z testów w przedziale 1000 - 10000. Z obu wersji CUDA-3.5 jest lepsza. Wersja CUDA-3.6 jest od niej wolniejsza o 57.87% jest to wynik gorszy niż w przypadku platformy I.

Wersja	Ilość próbek			
	40	80	200	400
CUDA-3.5	01:28:42.264	02:58:06.633	08:09:39.698	16:29:10.271
CUDA-3.6	02:24:44.621	04:50:11.702	12:02:17.266	K/C

Tab. 6.73: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 40 - 400.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CUDA-3.5	100.00%	100.00%	100.00%	100.00%	K/C	K/C	K/C	K/C
CUDA-3.6	163.18%	162.93%	147.51%	K/C	K/C	K/C	K/C	K/C

Tab. 6.74: Porównanie czasu renderu sceny "Labirynt" pomiędzy wersją CUDA-3.5, a CUDA-3.6, platforma II, głębokość 10.

6.3.4. Podsumowanie

Tabela 6.75 przedstawia podsumowanie porównań obu wersji. Dla platformy I wersja CUDA-3.6 jest wolniejsza średnio o 18.49%. W przypadku platformy II różnica to wynosi 23.68% pomimo faktu, że wersja była lepsza dla paru testów dla sceny "Lustra". Pomimo tego jeżeli weźmiemy wyniki z wszystkich testów pod uwagę wersja CUDA-3.6 jest gorsza od wersji CUDA-3.5.

Ponadto dla sceny "Cornell Box"i "Lustra" dłuższy czas pracy nie przekłada się na lepszy wynik. Scena "Labirynt", która głównie miała skorzystać na zmianach z tej wersji nie spełniła oczekiwań autora. Ciemnie nie oświetlone korytarze są jaśniejsze ale wygląda to nienaturalnie w porównaniu do reszty sceny. Zmiana ta zmniejsza też różnice pomiędzy najciemniejszym, a najjaśniejszym elementem sceny. Wpływa to negatywnie na kontrast końcowego obrazu.

Scena	Platforma I	Platforma II
Cornell Box	105.69%	109.37%
Lustra	104.01%	103.79%
Labirynt	145.78%	157.87%
Średnia	118.49%	123.68%

Tab. 6.75: Średni wynik sceny CUDA-3.4 względem sceny CUDA-3.3.

Wersja CUDA-3.6 nie spełniła założeń, które były brane pod uwagę przy jej tworzeniu. W związku z tym do końcowego porównania modeli oświetlenia wybrana została wersja CUDA-3.5.

Rozdział 7

Porównanie modeli oświetlenia

W poprzednim rozdziale następujące wersje programu *tracer* zostały wybrane do ostatecznego porównania modeli oświetlenia:

- CUDA-3.2 - model klasyczny,
- CUDA-3.4 - model hybrydowy,
- CUDA-3.5 - model rozszerzony.

W trakcie badań w poprzednim rozdziale ustaliliśmy, że wersje te nie różnią się w znaczący sposób zużyciem pamięci systemowej ani GPU. W związku z tym porównania w tym rozdziale skupią się na czasie potrzebnym na dokończenie renderu oraz porównaniu wizualnym wygenerowanych obrazów.

Uwaga! Na wszystkich rysunkach w tym rozdziale wersje są zawsze przedstawione w kolejności od lewej strony: CUDA-3.2, CUDA-3.4 oraz CUDA-3.5.

7.1. Cornell Box

7.1.1. Wydajność

Platforma I

Dla pierwszej sceny testowej model hybrydowy jest najwolniejszy. Wersja klasyczna jest szybsza od niej o 4.81%, a wersja rozszerzona o 4.90%. Różnica w średniej pomiędzy obiema wersjami wynosi tylko 0.09%. Możemy przyjąć, że oba modele w przypadku sceny "Cornell Box" są funkcjonalnie takie same.

Wersja	Ilość próbek			
	40	80	200	400
CUDA-3.2	00:00:28.556	00:00:57.122	00:02:22.579	00:04:44.695
CUDA-3.4	00:00:29.957	00:01:00.170	00:02:30.127	00:04:59.601
CUDA-3.5	00:00:28.625	00:00:57.840	00:02:22.338	00:04:44.464

Tab. 7.1: Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma I, głębokość 10, próbki 40 - 400.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CUDA-3.2	00:11:51.791	00:23:48.729	00:59:04.412	01:59:06.269
CUDA-3.4	00:12:27.574	00:24:55.218	01:02:11.826	02:04:42.792
CUDA-3.5	00:11:49.180	00:23:39.650	00:58:55.766	01:58:15.782

Tab. 7.2: Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma I, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CUDA-3.2	95.32%	94.93%	94.97%	95.02%	95.21%	95.55%	94.98%	95.50%
CUDA-3.4	100%	100%	100%	100%	100%	100%	100%	100%
CUDA-3.5	95.55%	96.13%	94.81%	94.95%	94.86%	94.95%	94.75%	94.83%

Tab. 7.3: Porównanie czasu renderów sceny "Cornell Box" pomiędzy modelami oświetlenia. Model hybrydowy jako 100%. Platforma I, głębokość 10.

Platforma II

W przypadku platformy II wersja klasyczna jest szybsza o 6.67% od wersji hybrydowej. Wersja rozszerzona jest szybsza o 6.73%. Pozycja każdej z wersji pozostaje taka sama, ale różnice pomiędzy nimi są minimalnie większe niż w przypadku platformy I.

Wersja	Ilość próbek			
	40	80	200	400
CUDA-3.2	00:01:43.286	00:03:27.610	00:08:37.355	00:17:14.533
CUDA-3.4	00:01:46.558	00:03:30.730	00:09:14.700	00:18:49.470
CUDA-3.5	00:01:44.468	00:03:25.544	00:08:33.849	00:17:07.352

Tab. 7.4: Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 40 - 400.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CUDA-3.2	00:43:08.714	01:26:44.900	03:36:49.972	07:16:50.171
CUDA-3.4	00:46:55.678	01:34:13.687	03:55:21.356	07:47:34.985
CUDA-3.5	00:42:48.653	01:25:37.212	03:34:01.270	07:19:16.967

Tab. 7.5: Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CUDA-3.2	96.93%	98.52%	93.27%	91.59%	91.94%	92.06%	92.13%	93.42%
CUDA-3.4	100%	100%	100%	100%	100%	100%	100%	100%
CUDA-3.5	98.04%	97.54%	92.64%	90.96%	91.23%	90.86%	90.94%	93.95%

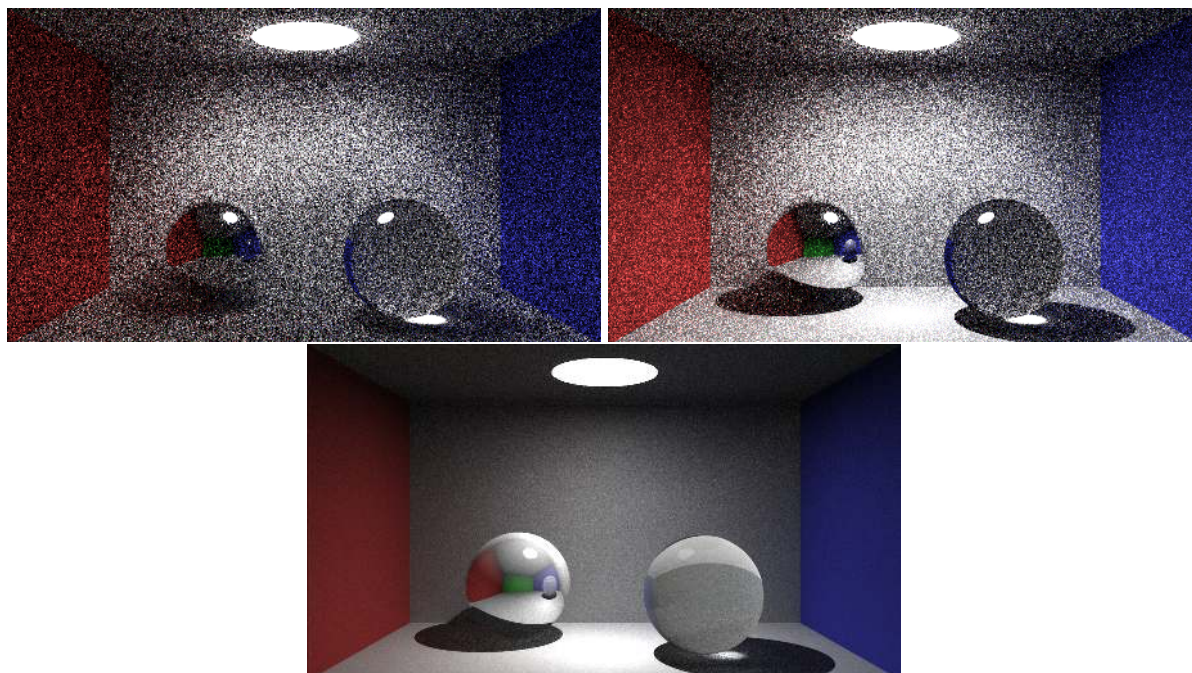
Tab. 7.6: Porównanie czasu renderów sceny "Cornell Box" pomiędzy modelami oświetlenia. Model hybrydowy jako 100%. Platforma II, głębokość 10.

Podsumowanie

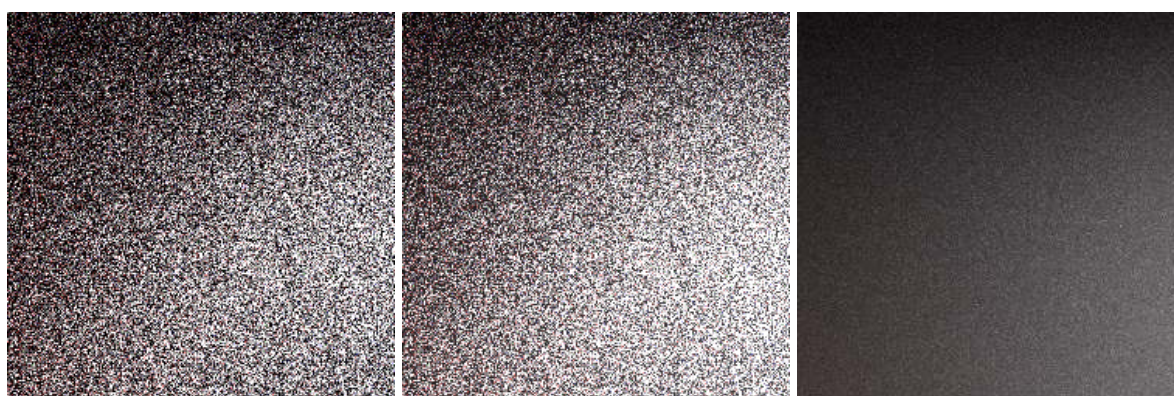
7.1.2. Porównanie graficzne

40 próbek

Na rysunku 7.1 przedstawiono porównanie wyników dla trzech wersji aplikacji. Model hybrydowy jest zdecydowanie najjaśniejszy. Centralne elementy podłogi są nawet zbyt jasne. Światło załamane przez szklaną kulę wygląda znacznie naturalnej w wersji rozszerzonej. Wersja ta jest też najmniej zaszumiona. Zbliżenie na ścianę zostało przedstawione na rysunku 7.2. Wersja hybrydowa oraz wersja klasyczna są do siebie zbliżone. Na wersji rozszerzonej na zbliżeniu widzimy, że szum jest dalej obecny, ale jest go znacznie mniej niż w pozostałych wersjach.



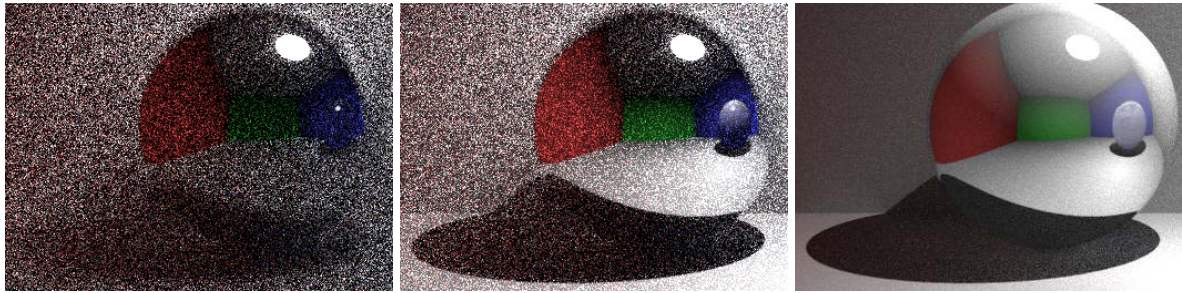
Rys. 7.1: Porównanie sceny "Cornell Box", 40 próbek, głębokość 10.



Rys. 7.2: Porównanie szumu na scenie "Cornell Box", 40 próbek, głębokość 10.

Na rysunku 7.3 przedstawiono zbliżenie na kulę odbijającą światło. W przypadku modelu klasycznego granice pomiędzy obiektem kuli, ścianami się zlewają. Granice cienia rzucanego przez kulę są także bardzo niewyraźne. Wygląda to znacznie lepiej w przypadku wersji hybrydowej. Widzimy granicę pomiędzy ścianą i sufitem za kulą. Granice cienia rzucanego przez kulę są także znacznie lepiej zdefiniowane. Wersja rozszerzona oferuje to same zalety z mniejszą ilością

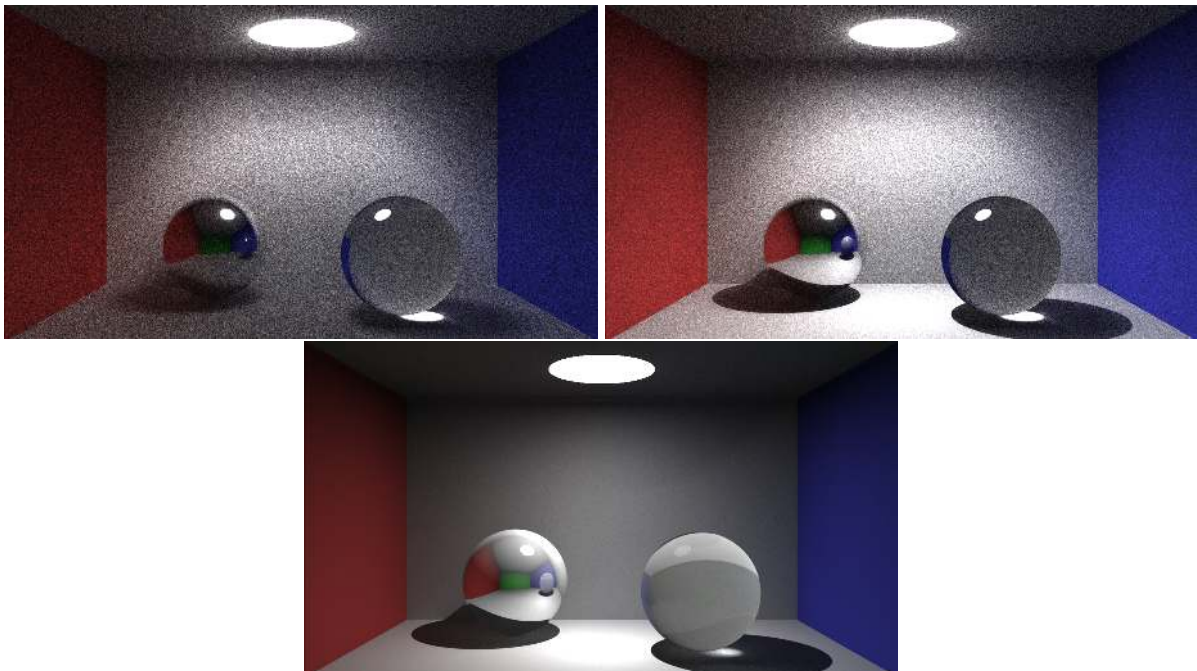
szumu. Odbicie światła w górnej prawej części jest także znacznie bardziej rozwinięte. Światło nie jest odbijane tylko punktowo. Dzięki temu kula ta zlew się mniej z tłem w tym miejscu niż w pozostałych wersjach.



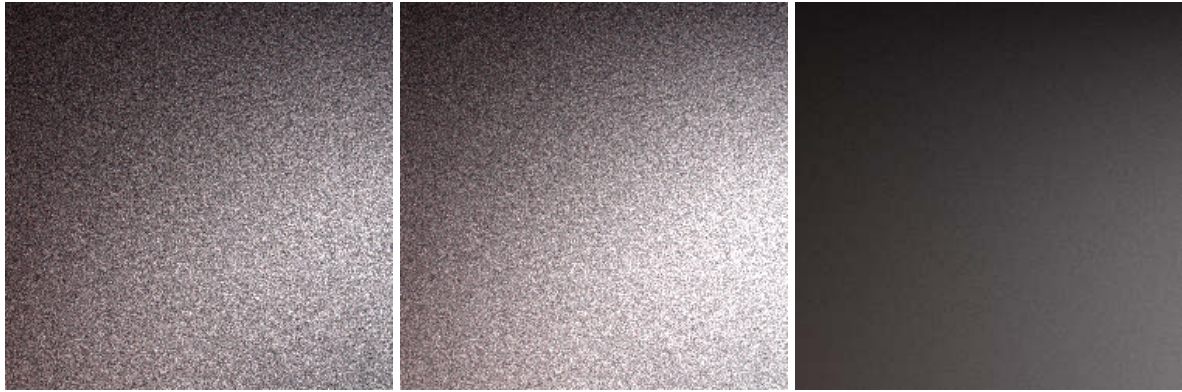
Rys. 7.3: Zbliżenie na lustrzaną kulę, scena "Cornell Box", 40 próbek, głębokość 10.

400 próbek

Pomimo zwiększenia ilości próbek na rysunku 7.4 w dalszym ciągu granica pomiędzy ścianą, a podłogą nie jest widoczna. Granice pomiędzy cieniem, a oświetloną częścią podłogi także pozostaje bardzo niewyraźna. Jeżeli spojrzymy na porównanie szumu na rysunku 7.5 wersja hybrydowa i klasyczna są prawie identyczne. Pomimo dużej poprawy pozostają one gorsze od wersji rozszerzonej nawet dla 40 próbek. Wersja hybrydowa wyróżnia się głównie lepszą definicją cieni oraz granic między płaszczyznami. Wadą w dalszym ciągu pozostaje znaczne zwiększenie jasności w stosunku do wersji klasycznej.

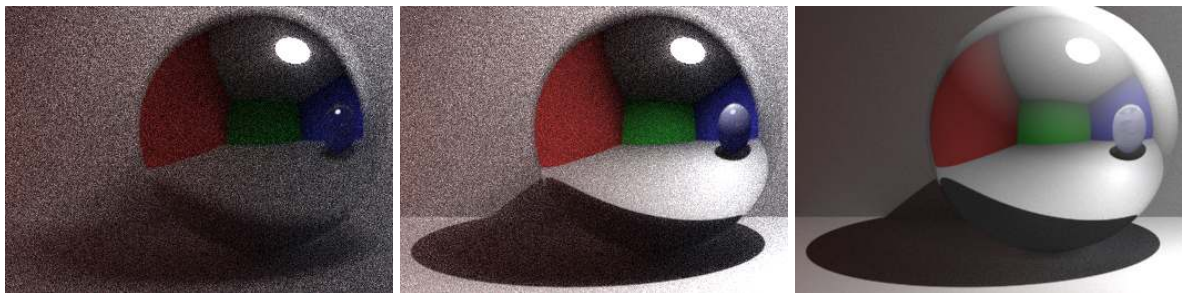


Rys. 7.4: Porównanie sceny "Cornell Box", 400 próbek, głębokość 10.



Rys. 7.5: Porównanie szumu na scenie "Cornell Box", 400 próbek, głębokość 10.

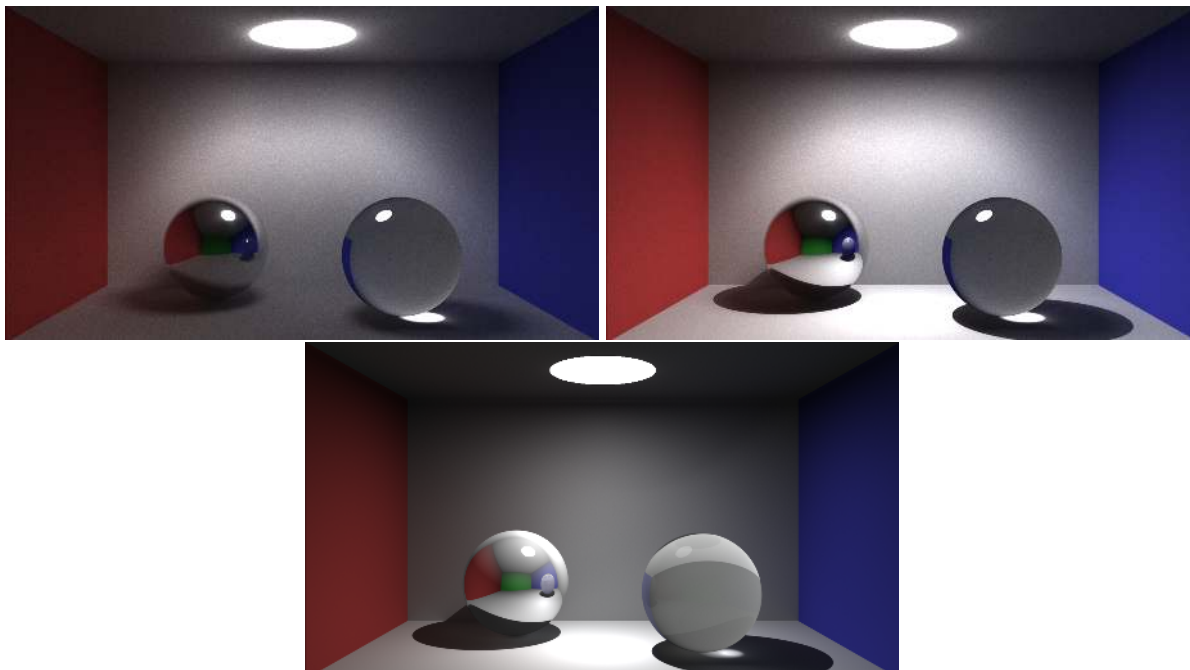
Wersja rozszerzona wyróżnia się w ilościach szczegółów, które są widoczne nawet w nie oświetlonych elementach sceny. Jest to dobrze widoczne na rysunku 7.6.



Rys. 7.6: Zbliżenie na lustrzaną kulę, scena "Cornell Box", 400 próbek, głębokość 10.

10000 próbek

Ostatnie porównanie przedstawiono na rysunku 7.7 dla testu z największą ilością próbek. W dalszym ciągu granice pomiędzy zacienioną strefą, a oświetloną pozostają niewyraźne w wersji klasycznej. Widzimy też w końcu granicę pomiędzy ścianą i sufitem jest to jednak znacznie mniej wyraźnie niż nawet w przypadku modelu hybrydowego. Na rysunku 7.8 przedstawiono porównanie szumu pomiędzy modelami. Dla tej ilości próbek dla wersji hybrydowej szum w ogóle nie jest widoczny. Wersje klasyczne i hybrydowe prezentują podobny poziom szumu. Jest to ilość zbliżona do wersji rozszerzonej dla 400 próbek.

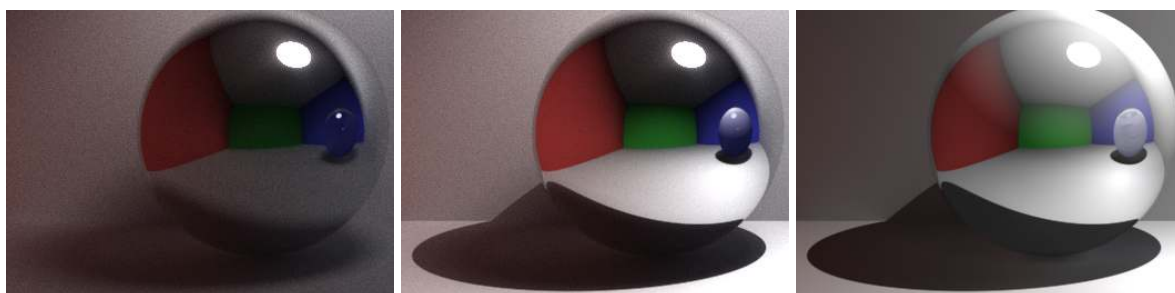


Rys. 7.7: Porównanie sceny "Cornell Box", 10000 próbek, głębokość 10.



Rys. 7.8: Porównanie szumu na scenie "Cornell Box", 10000 próbek, głębokość 10.

Porównując zbliżenie na kulę przedstawione na rysunku 7.9. Wersja klasyczna wygląda znacznie mniej wyraźnie w porównaniu do pozostałych. W wersji rozszerzonej wyróżnia się jaśniejsza część cienia widoczna bezpośrednio pod kulą. Jest to efekt światła odbitego od lustrzanej powierzchni kuli.



Rys. 7.9: Zbliżenie na lustrzaną kulę, scena "Cornell Box", 10000 próbek, głębokość 10.

7.1.3. Podsumowanie

Dla sceny "Cornell Box" wersja rozszerzona jest bezkonkurencyjna. Oferuje ona obraz lepszej jakości bez straty czasu względem wersji klasycznej.

Wersja hybrydowa wbrew nadzieję autora nie pomaga w ograniczeniu ilości szumu w stosunku do wersji klasycznej. Zwiększaniu ulega rozpiętość tonalna obrazu co skutkuje wyraźniejszymi granicami między obiektami i obszarami zacienionymi. Niestety obraz jest w niektórych elementach aż zbyt jasny, szczególnie podłoga. Dodając do tego wolniejszy czas egzekucji niż w wersji klasycznej i rozszerzonej wersja hybrydowa nie jest warta wykorzystania w przypadku tej sceny.

7.2. Lustra

7.2.1. Wydajność

Platforma I

W przypadku drugiej sceny testowej na scenie znajdują się dwa źródła światła. Zmienia to całkowicie wyniki aplikacji w stosunku do sceny "Cornell Box". Wersja klasyczna jest średnio szybsza o 13.11% od sceny hybrydowej. Wersja rozszerzona jest natomiast wolniejsza od niej o 183.32%. Sprawia to, że pod względem czasu trwania renderu test dla 2000 próbek z wersji rozszerzonej odpowiada najbliższej testowi dla 5000 próbek w wersji hybrydowej i dla 10000 próbek w wersji klasycznej.

Wersja	Ilość próbek			
	40	80	200	400
CUDA-3.2	00:00:53.453	00:01:46.810	00:04:26.977	00:08:53.240
CUDA-3.4	00:01:01.899	00:02:02.345	00:05:06.408	00:10:14.589
CUDA-3.5	00:02:54.552	00:05:47.904	00:14:37.430	00:28:57.693

Tab. 7.7: Czas potrzebny na wygenerowanie sceny "Lustra", platforma I, głębokość 10, próbki 40 - 400.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CUDA-3.2	00:22:11.579	00:44:20.477	01:50:47.381	03:41:30.905
CUDA-3.4	00:25:33.256	00:51:02.964	02:07:34.315	04:14:28.941
CUDA-3.5	01:12:10.393	02:24:20.806	06:00:34.560	12:00:48.342

Tab. 7.8: Czas potrzebny na wygenerowanie sceny "Lustra", platforma I, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CUDA-3.2	86.36%	87.30%	87.13%	86.76%	86.85%	86.86%	86.84%	87.05%
CUDA-3.4	100%	100%	100%	100%	100%	100%	100%	100%
CUDA-3.5	281.99%	284.36%	286.36%	282.74%	282.43%	282.76%	282.65%	283.24%

Tab. 7.9: Porównanie czasu renderów sceny "Lustra" pomiędzy modelami oświetlenia. Model hybrydowy jako 100%. Platforma I, głębokość 10.

Platforma II

Różnice pomiędzy wersjami są identyczne jak w przypadku platformy I. Wersja klasyczna jest szybsza od hybrydowej o 13.52%, natomiast wersja rozszerzona jest wolniejsza o 183.90%. Ponieważ testy na platformie II są wolniejsze testy dla 5000 próbek dla wersji rozszerzonej nie zostały ukończone w limicie czasu.

Wersja	Ilość próbek			
	40	80	200	400
CUDA-3.2	00:03:16.731	00:06:35.562	00:16:27.549	00:32:50.846
CUDA-3.4	00:03:41.678	00:07:22.799	00:18:50.781	00:38:43.530
CUDA-3.5	00:10:34.110	00:21:07.887	00:52:49.276	01:46:01.611

Tab. 7.10: Czas potrzebny na wygenerowanie sceny "Lustra", platforma II, głębokość 10, próbki 40 - 400.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CUDA-3.2	01:22:12.220	02:43:53.694	06:50:53.303	14:02:55.746
CUDA-3.4	01:36:39.260	03:13:25.617	08:02:28.483	16:13:04.832
CUDA-3.5	04:29:21.290	09:36:56.207	K/C	K/C

Tab. 7.11: Czas potrzebny na wygenerowanie sceny "Lustra", platforma II, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CUDA-3.2	88.75%	89.33%	87.33%	84.82%	85.05%	84.73%	85.16%	86.62%
CUDA-3.4	100%	100%	100%	100%	100%	100%	100%	100%
CUDA-3.5	286.05%	286.33%	280.27%	273.79%	278.68%	298.27%	K/C	K/C

Tab. 7.12: Porównanie czasu renderów sceny "Lustra" pomiędzy modelami oświetlenia. Model hybrydowy jako 100%. Platforma II, głębokość 10.

Podsumowanie

7.2.2. Porównanie graficzne

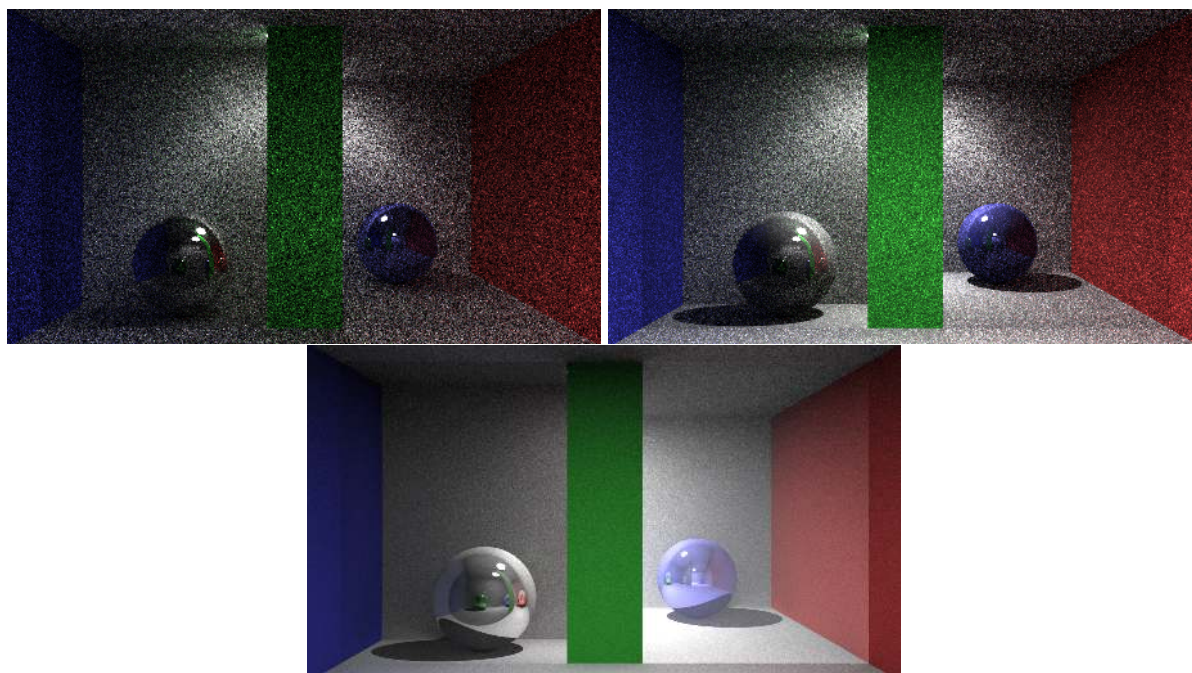
W przypadku tej wersji porównanie końcowej jakości renderów jest niepraktyczne ze względu na dużą różnicę w czasach pomiędzy wersjami dla tej samej ilości próbek. W związku z tym do porównań zostały wybrane obrazy z testów, których wyniki są najbardziej zbliżone czasowo dla platformy I.

2.5 minuty

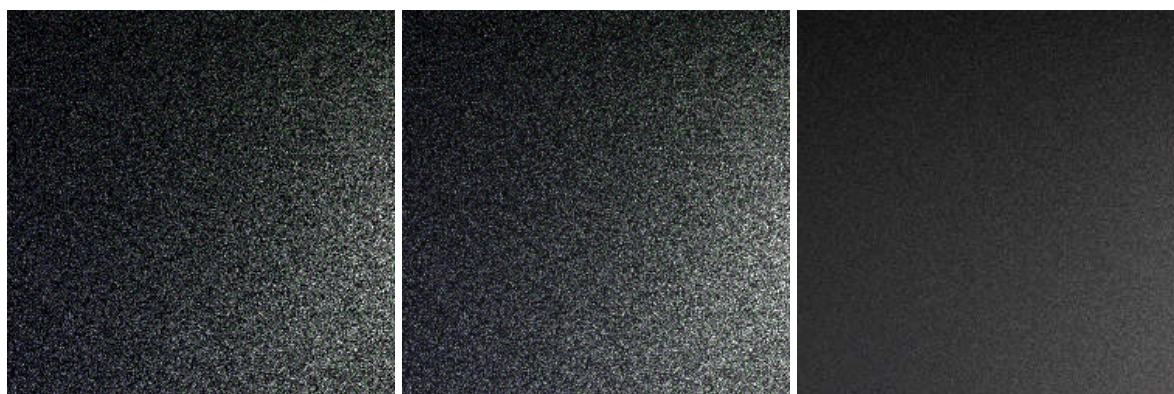
Pierwsze porównanie zostało wykonane dla obrazu z testu dla 40 próbek w wersji rozszerzonej. Do porównania z wersji hybrydowej został wybrany test dla 80 próbek (00:00:52.207 krócej). Z wersji klasycznej wybrany został także test dla 80 próbek (00:01:07.742 krócej). Wyniki tych testów są najbliższe dla testu z wersji rozszerzonej.

Na rysunku 7.10 wersja rozszerzona wyróżnia w jakości odbicia w lustrze znajdującym się po

prawej stronie sceny. W przypadku wersji klasycznej i hybrydowej nic poza geometrią obiektów w odbiciu nie wskazuje, że patrzymy na odbicie. W wersji rozszerzonej wyraźnie widzimy jak światło odbijające się w lustrze wpływa na odbicie. Pomimo różnicy w ilości próbek na rysunku 7.11 widzimy, że wersja rozszerzona w oferuje najmniejszą ilość szumu.

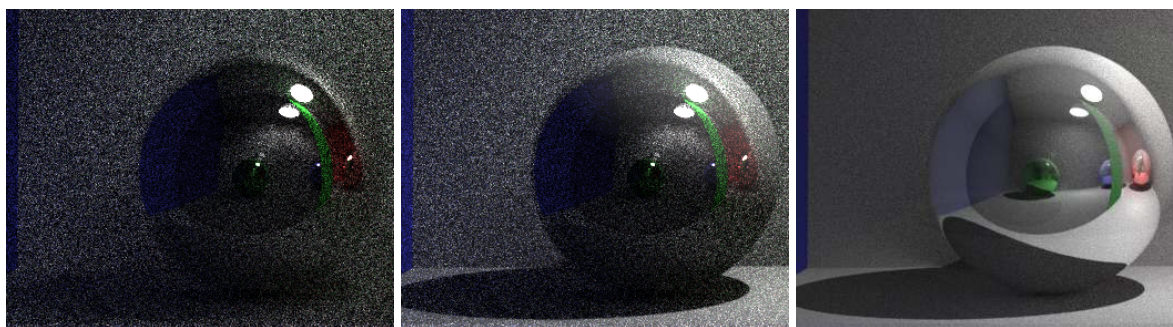


Rys. 7.10: Porównanie sceny "Lustra", 80|80|40 próbek, głębokość 10.



Rys. 7.11: Porównanie szumu na scenie "Lustra", 80|80|40 próbek, głębokość 10.

W zbliżeniu przedstawionym na rysunku 7.12 wersja rozszerzona ma znaczną przewagę w jakości odbić widocznych na kuli. W odbiciu widzimy dokładnie wszystkie detale sceny. Jesteśmy w stanie nawet dostrzec szczegóły w odbiciach wtórnych widocznych w kolorowych kulach. W przypadku wersji hybrydowej dość sporo szczegółów zostało zachowanych w pierwszym odbiciu. Obraz jednak jest w dalszym ciągu znacznie gorszej jakości niż w przypadku wersji rozszerzonej. Przykładowo cień kuli nie jest prawie w ogóle widoczny w odbiciu. Wersja klasyczna wypada w tym wypadku najgorzej. Obraz w odbiciu jest bardzo ciemny i ciężko jest wskazać konkretne elementy znajdujące się na scenie. Przykładowo niebieska kula jest prawie w ogóle niewidoczna.

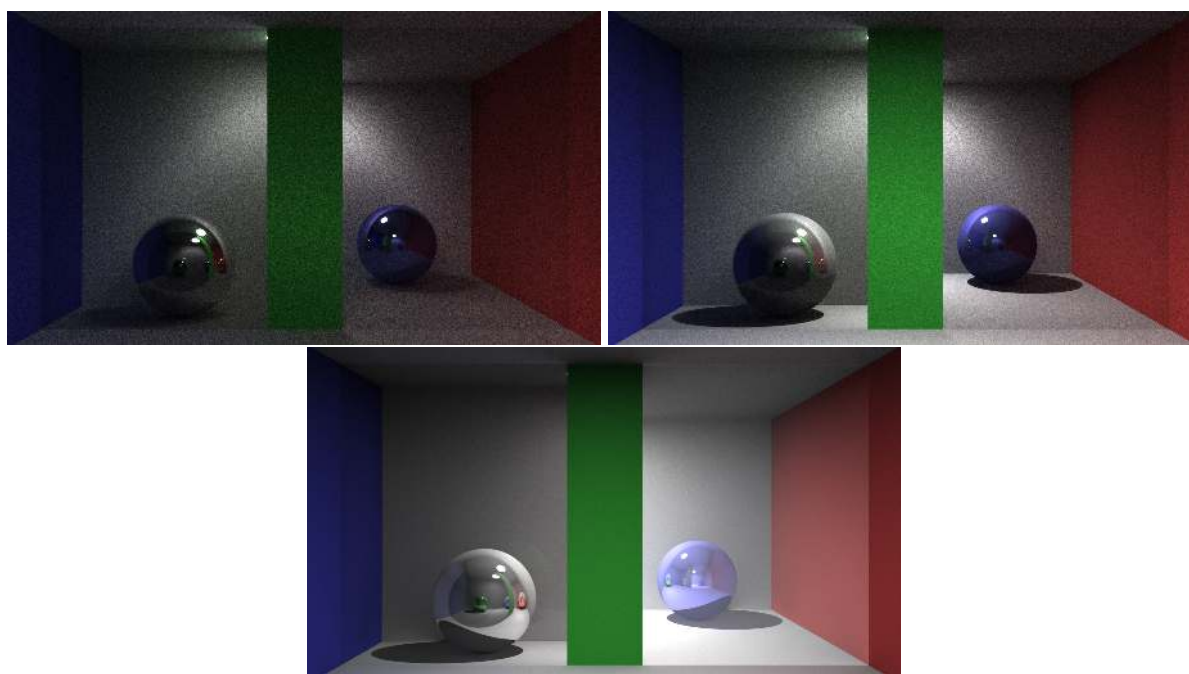


Rys. 7.12: Zbliżenie na kulę widoczną po drugiej stronie tafli szkła, scena "Lustra", 80|80|40 próbek, głębokość 10.

0.5 godziny

Kolejne testy do porównania zostały dobrane do czasu testu dla 400 próbek w wersji rozszerzonej. W wersji hybrydowej jest to test dla 1000 próbek (krótszy o 00:03:24.437). W przypadku wersji klasycznej także jest to test dla 1000 próbek (krótszy o 00:06:46.114).

Nawet przy 1000 próbek wersje hybrydowa oraz klasyczna nie radzą sobie z wielokrotnymi odbiciami widocznymi na tej scenie. Jak przedstawiono na rysunku 7.4 model hybrydowy w tej wersji nie cierpi prześwietlenia elementów sceny jak to miało miejsce przy scenie "Cornell Box". Pomimo braku różnicy w ilości szumu (jak przedstawiono w porównaniu na rysunku 7.5) pozwala to na osiągnięcie znacznie lepszego efektu niż w przypadku wersji klasycznej. Model rozszerzony wyróżnia się też lepszą obsługą szkła. Symetrycznie do niebieskiej kuli widocznej po prawej scenie w tafli szkła możemy zobaczyć odbicie zielonej kuli znajdującej się za kamerą. Efekt jest także zaimplementowany w pozostałych wersjach. Jednak ze względu na większą ilość szumu i inne zachowanie modeli oświetlenia nie jest on widoczny.

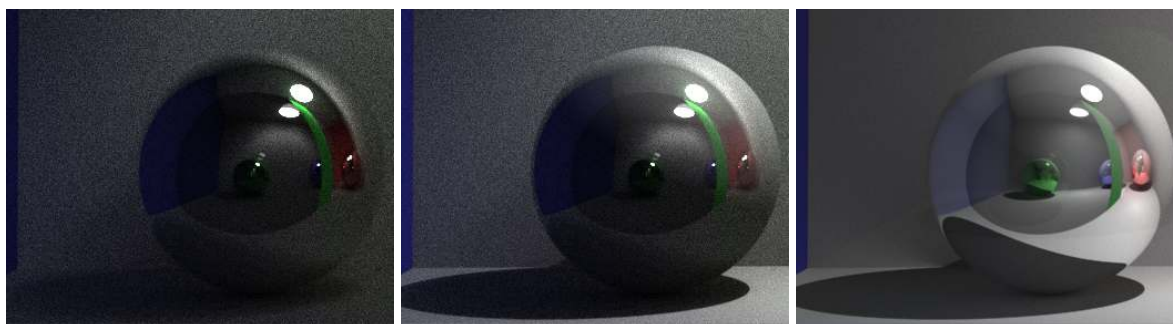


Rys. 7.13: Porównanie sceny "Lustra", 1000|1000|400 próbek, głębokość 10.



Rys. 7.14: Porównanie szumu na scenie "Lustra", 1000|1000|400 próbek, głębokość 10.

W porównaniu na rysunku 7.15 widzimy, że nawet dla 400 próbek w modelu rozszerzonym jesteśmy w stanie dostrzec wszystkie elementy sceny nawet we wtórnych odbiciach. Jedynym element ograniczającym nast tutaj jest rozdzielczość wygenerowanego obrazu. Na wersji hybrydowej i klasycznej widzimy poprawę w stosunku do wcześniejszych wersji. W dalszym ciągu obraz w odbiciach jest bardzo ciemny i ciężko jest zobaczyć w nim coś na więcej niż pierwsze odbicie.

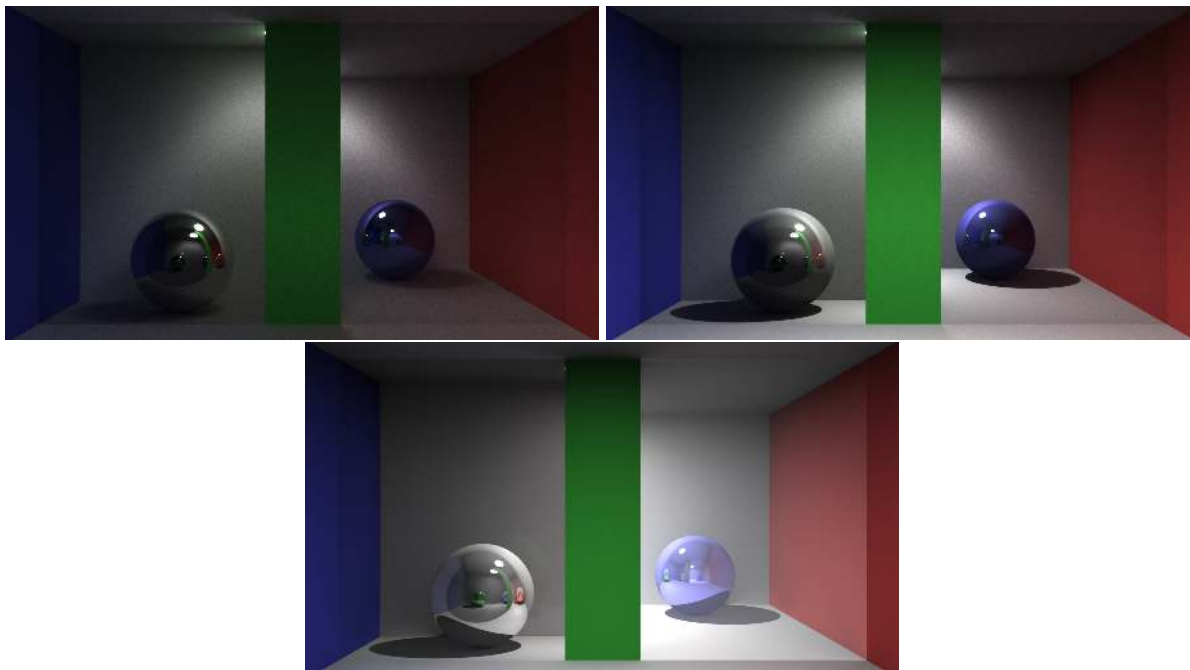


Rys. 7.15: Zbliżenie na kulę widoczną po drugiej stronie tafli szkła, scena "Lustra", 1000|1000|400 próbek, głębokość 10.

3.5 godziny - maximum

Kolejne porównanie zostało dobrane dla najlepszego renderu w wersji klasycznej. Zajął w tej wersji nieco ponad 3 godziny 40 minut. Dla wersji hybrydowej najbliższy wynik został osiągnięty także dla 10000 próbek (00:32:58.036 dłużej). W wersji rozszerzonej jest to test dla 2000 próbek (01:17:10.099 krócej).

Na rysunku 7.16 przedstawiono główne porównanie. Pod względem ilości szumu dopiero dla 10000 próbek model klasyczny i hybrydowy są porównywalne do wersji rozszerzonej dla 400 próbek. Potwierdza to porównanie przedstawione na rysunku 7.17. Na wersji rozszerzonej dla 2000 próbek szum prawie nie jest widoczny. Pomimo znacznej poprawy modele oświetlenia klasyczny i hybrydowy w dalszym ciągu nie są w stanie zreplikować efektów odbicia w tafli szkła i rozproszenia światła na powierzchni lustra.

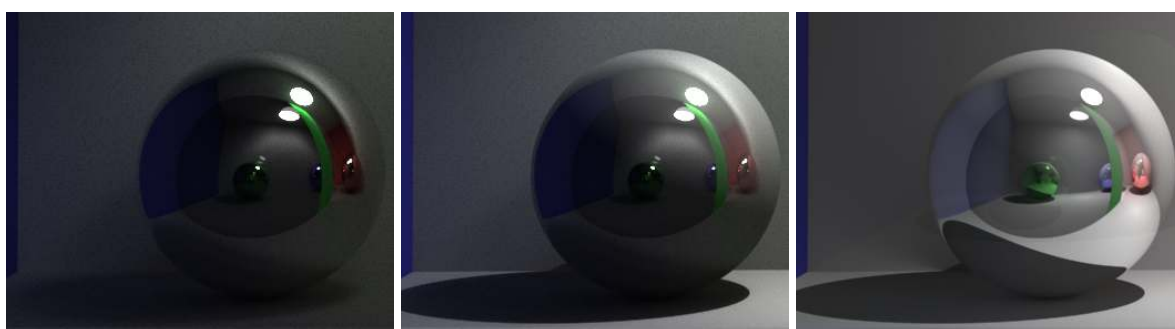


Rys. 7.16: Porównanie sceny "Lustra", 10000|10000|2000 próbek, głębokość 10.



Rys. 7.17: Porównanie szumu na scenie "Lustra", 10000|10000|2000 próbek, głębokość 10.

W porównaniu jakości odbić przedstawiony na rysunku 7.18 wersja rozszerzona w dalszym ciągu pozostaje bezkonkurencyjna. Na głębokość 1 odbicia wersja hybrydowa także osiąga akceptowalne wyniki. Nie radzi sobie jednak z załamaniem światła w szkłe po odbiciu. Przykładowo cień rzucany przez kulę po drugiej stronie tafli szkła nie jest w ogóle widoczny. W modelu klasycznym nawet przy 10000 próbek bardzo dużo szczegółów sceny jest traconych.



Rys. 7.18: Zbliżenie na kulę widoczną po drugiej stronie tafli szkła, scena "Lustra", 10000|10000|2000 próbek, głębokość 10.

7.2.3. Podsumowanie

Dla sceny "Lustra" nawet pomimo gorszych czasowo wyników niż przy scenie "Cornell Box" wersja rozszerzona oferuje najlepszą jakość w stosunku do czasu potrzebnego na ukończenie renderu. Wiele z efektów takich jak odbicia w szkłe czy rozpraszanie się światła w lustrze są widoczne tylko przy tym modelu oświetlenia. Na drugim miejscu dla tej sceny wyróżnia się model hybrydowy. Nie jest widoczny efekt przeświecenia sceny jak w przypadku poprzedniej sceny, a kontrast pomiędzy obiektami ulega znacznemu poprawieniu. Nie radzi sobie on jednak zupełnie z wielokrotnymi odbiciami w porównaniu do modelu rozszerzonego. Model klasyczny w tym wypadku zawodzi całkowicie. Scena cierpi na niedobór światła, który szczególnie w odbiciach sprawia, że ciężko jest dostrzec szczegóły.

7.3. Labirynt

7.3.1. Wydajność

7.3.2. Platforma I

Dla ostatniej sceny testowej widzimy największą rozbieżność pomiędzy testowanymi modelami. Wersja klasyczna jest szybsza o 49.64% od wersji hybrydowej. Natomiast wersja rozszerzona jest wolniejsza o 383.67%. Różnica ta wynika z dużej liczby źródeł światła znajdujących się na scenie. Opóźnienie dla wersji rozszerzonej jest na tyle duże, że testy dla 5000 i 10000 próbek przekroczyły maksymalny możliwy czas egzekucji.

Wersja	Ilość próbek			
	40	80	200	400
CUDA-3.2	00:02:30.705	00:04:59.767	00:12:28.837	00:24:46.480
CUDA-3.4	00:04:59.558	00:09:53.710	00:24:37.259	00:49:14.734
CUDA-3.5	00:24:06.207	00:47:50.761	02:00:07.164	04:00:12.717

Tab. 7.13: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 40 - 400.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CUDA-3.2	01:01:53.252	02:03:43.995	05:11:11.105	10:22:37.490
CUDA-3.4	02:03:48.930	04:07:34.609	10:14:31.348	20:33:01.214
CUDA-3.5	09:57:36.645	19:56:42.946	K/C	K/C

Tab. 7.14: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CUDA-3.2	50.31%	50.49%	50.69%	50.31%	49.98%	49.98%	50.64%	50.50%
CUDA-3.4	100%	100%	100%	100%	100%	100%	100%	100%
CUDA-3.5	482.78%	483.53%	487.87%	487.78%	482.66%	483.37%	K/C	K/C

Tab. 7.15: Porównanie czasu renderów sceny "Labirynt" pomiędzy modelami oświetlenia. Model hybrydowy jako 100%. Platforma I, głębokość 10.

Platforma II

W przypadku tej sceny skalowanie pomiędzy testowanymi wersjami pozostaje zbliżone do zaobserwowanego w przypadku platformy I. Dla II platformy testowej żaden z algorytmów nie ukończył testu dla 10000 próbek. Wersja rozszerzona, która w tym wypadku jest wolniejsza o 408.09% od wersji hybrydowej nie ukończyła żadnego testu z przedziału 1000 - 1000.

Wersja	Ilość próbek			
	40	80	200	400
CUDA-3.2	00:09:07.879	00:18:13.688	00:45:35.125	01:31:09.156
CUDA-3.4	00:18:24.163	00:36:50.298	01:32:16.787	03:04:28.350
CUDA-3.5	01:28:42.264	02:58:06.633	08:09:39.698	16:29:10.271

Tab. 7.16: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 40 - 400.

Wersja	Ilość próbek			
	1000	2000	5000	10000
CUDA-3.2	03:47:54.300	07:35:50.550	19:04:41.391	K/C
CUDA-3.4	07:42:16.778	15:19:19.248	K/C	K/C
CUDA-3.5	K/C	K/C	K/C	K/C

Tab. 7.17: Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 1000 - 10000.

Wersja	Ilość próbek							
	40	80	200	400	1000	2000	5000	10000
CUDA-3.2	49.62%	49.48%	49.40%	49.41%	49.30%	49.58%	B/I	K/C
CUDA-3.4	100%	100%	100%	100%	100%	100%	K/C	K/C
CUDA-3.5	482.02%	483.49%	530.63%	536.22%	K/C	K/C	K/C	K/C

Tab. 7.18: Porównanie czasu renderów sceny "Labirynt" pomiędzy modelami oświetlenia. Model hybrydowy jako 100%. Platforma II, głębokość 10.

Podsumowanie

Wersja rozszerzona spowalnia względem wersji klasycznej z każdym źródłem światła dodanym do sceny. Dotyka to także wersji hybrydowej, ale w mniejszym stopniu.

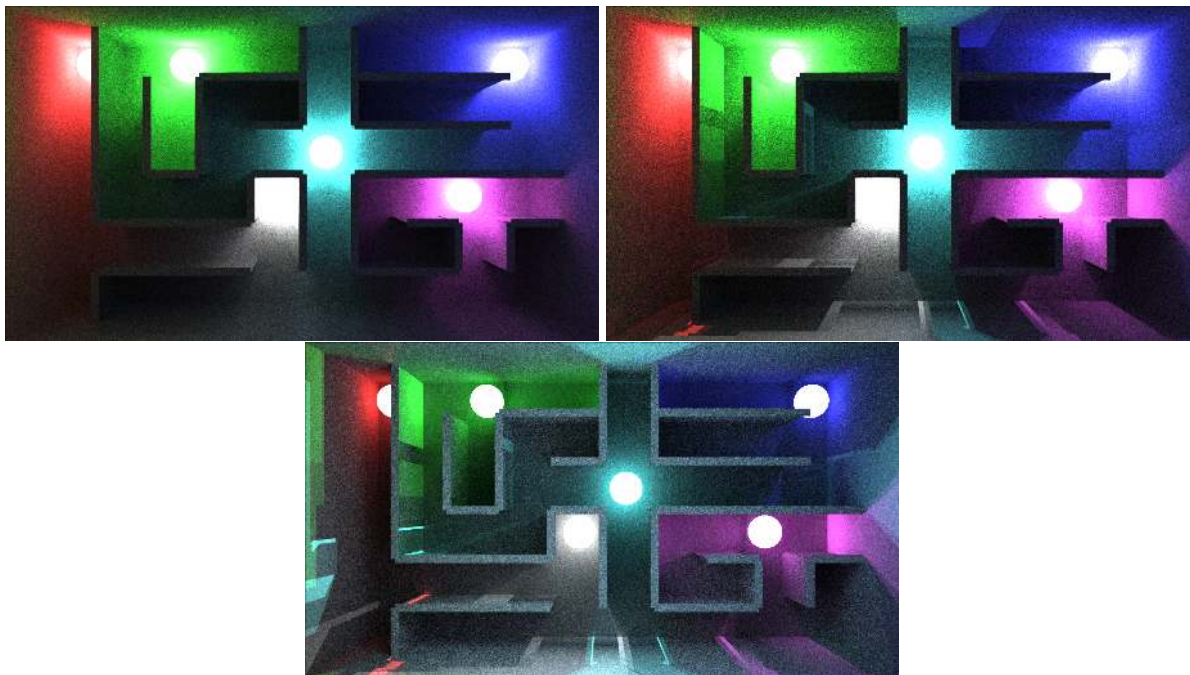
7.3.3. Porównanie graficzne

Podobnie jak w przypadkach porównania dla sceny "Lustra" ze względu na duże różnice w czasie potrzebnym na wygenerowanie obrazu - porównania zostały przeprowadzone według czasu, a nie ilości próbek.

24 minuty

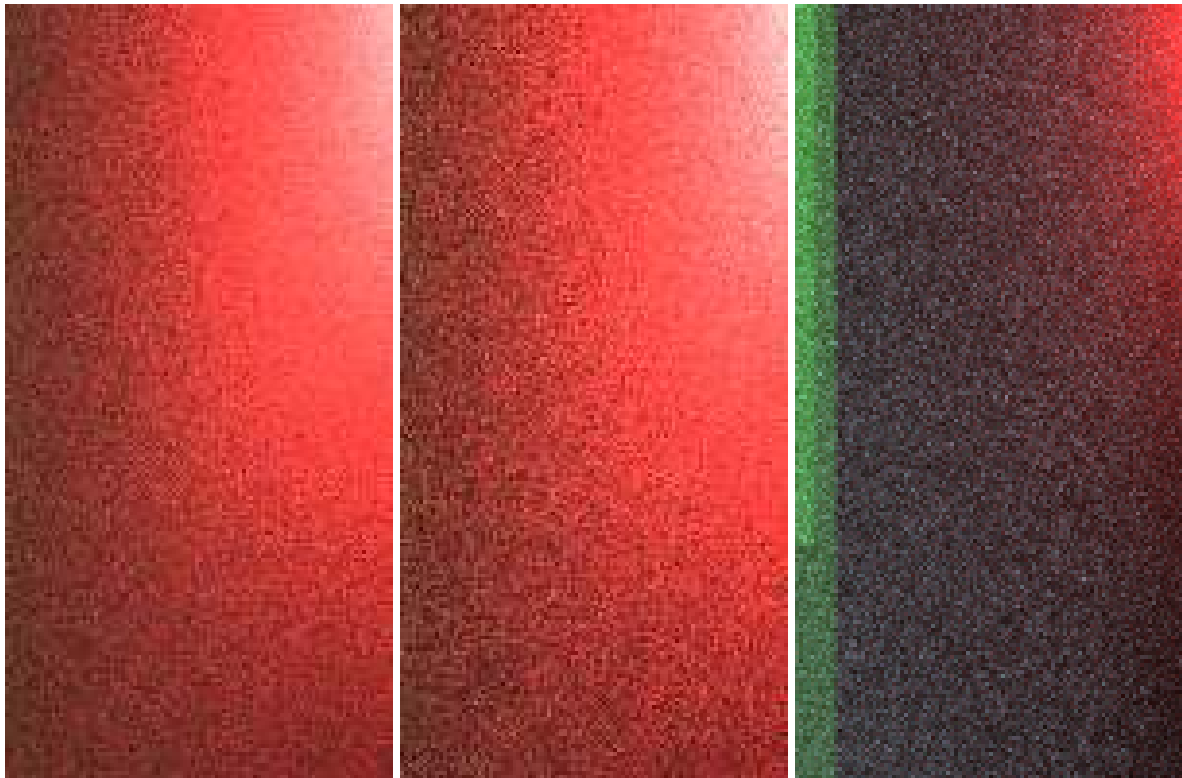
Pierwsze porównanie zostało dopasowane do wyniku testu wersji rozszerzonej dla 40 próbek. Najbliżej zbliżony czasowo w wersji hybrydowej był test dla 200 próbek (dłużej o 00:00:31.052), a dla wersji klasycznej test dla 400 próbek (dłużej o 00:00:40.273).

Na rysunku 7.19 model rozszerzony wyróżnia się większą jasnością sceny. Źródła światła są mniej intensywne w tej wersji. Widzimy dzięki temu więcej detali w okolicach źródeł światła. W przypadku wersji klasycznej źródła światła są bardzo jasne blisko źródła. Bardzo dużo detali ginie blisko nich. Pomimo tego scena jest ogólnie ciemniejsza, szczególnie w korytarzach gdzie nie ma bezpośredniego dostępu do źródła światła. Zdaniem autora wynik wersji rozszerzonej jest bardziej realistyczny. Należy pamiętać, że na tej scenie za kamerą znajduje się lustro. W związku z czym korytarze i zakamarki znajdujące się w labiryncie nie mogą być całkowicie nie oświetlone.



Rys. 7.19: Porównanie sceny "Labirynt", 400|200|40 próbek, głębokość 10.

W przypadku porównania ilości szumu na scenie przedstawionego na rysunku 7.20 nie widać dużej w ilości szumu pomiędzy wersją klasyczną, a wersją rozszerzoną. Jedynie wersja hybrydowa wygląda wyraźnie gorzej od pozostałych wersji.

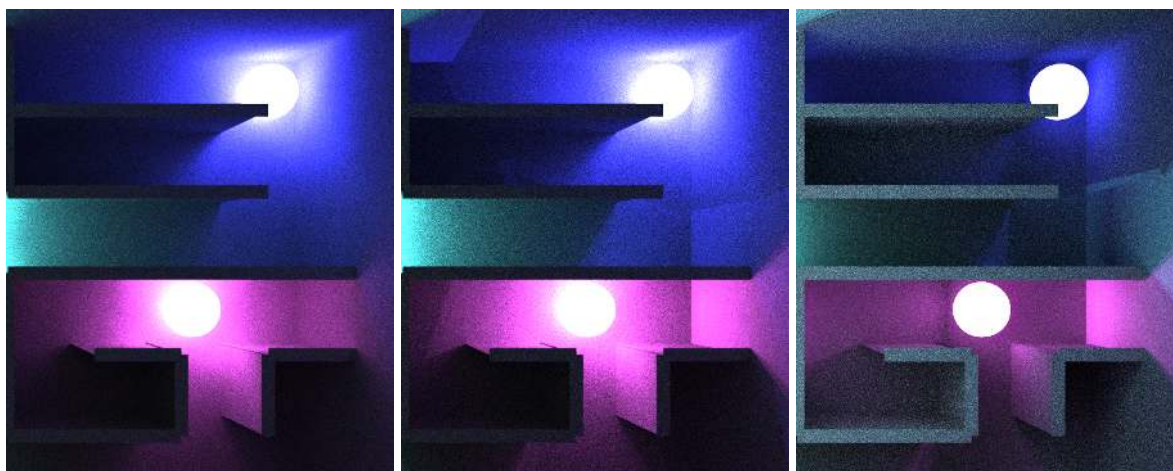


Rys. 7.20: Porównanie szumu na scenie "Lustra", 400|200|40 próbek, głębokość 10.

Jedną z większych wad wersji rozszerzonej jest brak brania pod uwagę odległości oświetlonego punktu od źródła światła. Przez to niestety mieszanie się źródeł światła przedstawione na rysunku 7.21 wygląda najgorzej w przypadku tej wersji. Gdzie granice pomiędzy kolorami są bardzo ostre. W wersji klasycznej kolory mieszają się w znacznie bardziej naturalnie wyglądający sposób. Niestety przez małą ilość światła w tej wersji widzimy bardzo mało szczegółów na scenie. Zdaniem autora pod tym względem najlepiej wygląda wersja hybrydowa.



Rys. 7.21: Zjawisko mieszania się światła z różnych źródeł światła na scenie "Labirynt", 400|200|40 próbek, głębokość 10.

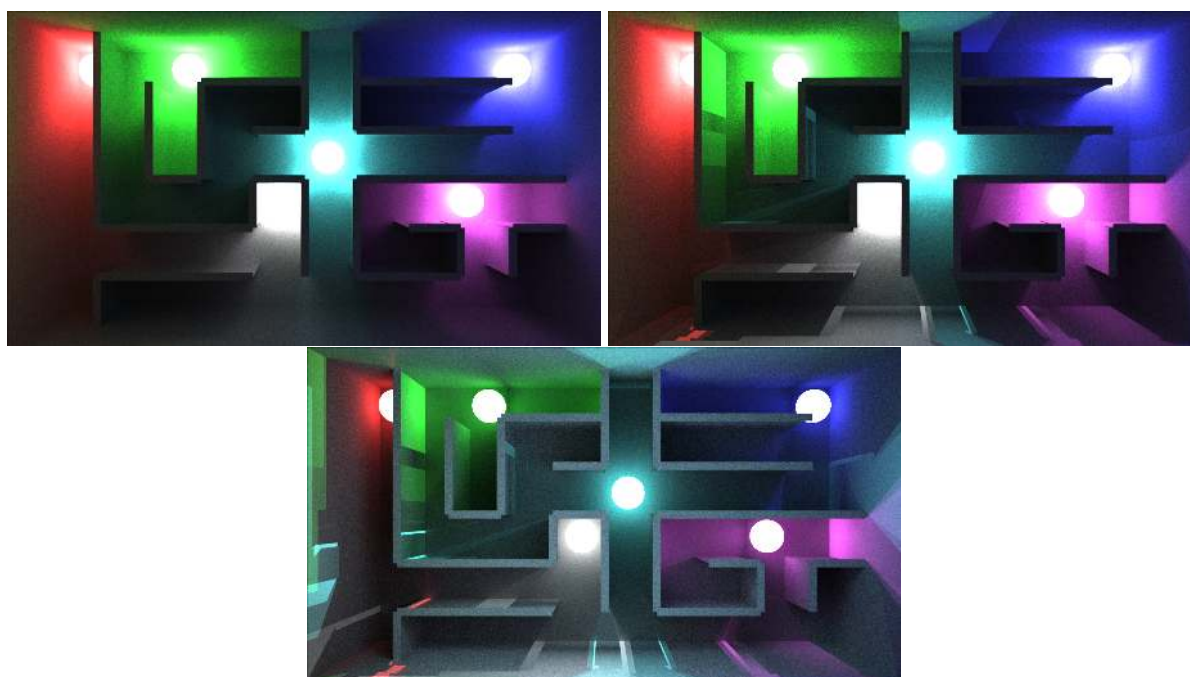


Rys. 7.22: Zbliżenie na wschodnią część sceny "Labirynt", 400|200|40 próbek, głębokość 10.

2 godziny

Kolejną porównanie zostało dobrane do testu dla 200 próbek w przypadku modelu rozszerzonego. Najbardziej zbliżone czasowo z modelu klasycznego był test dla 2000 próbek (dłużej o 00:03:36.831), a dla wersji hybrydowej dla 1000 próbek (dłużej o 00:03:41.766).

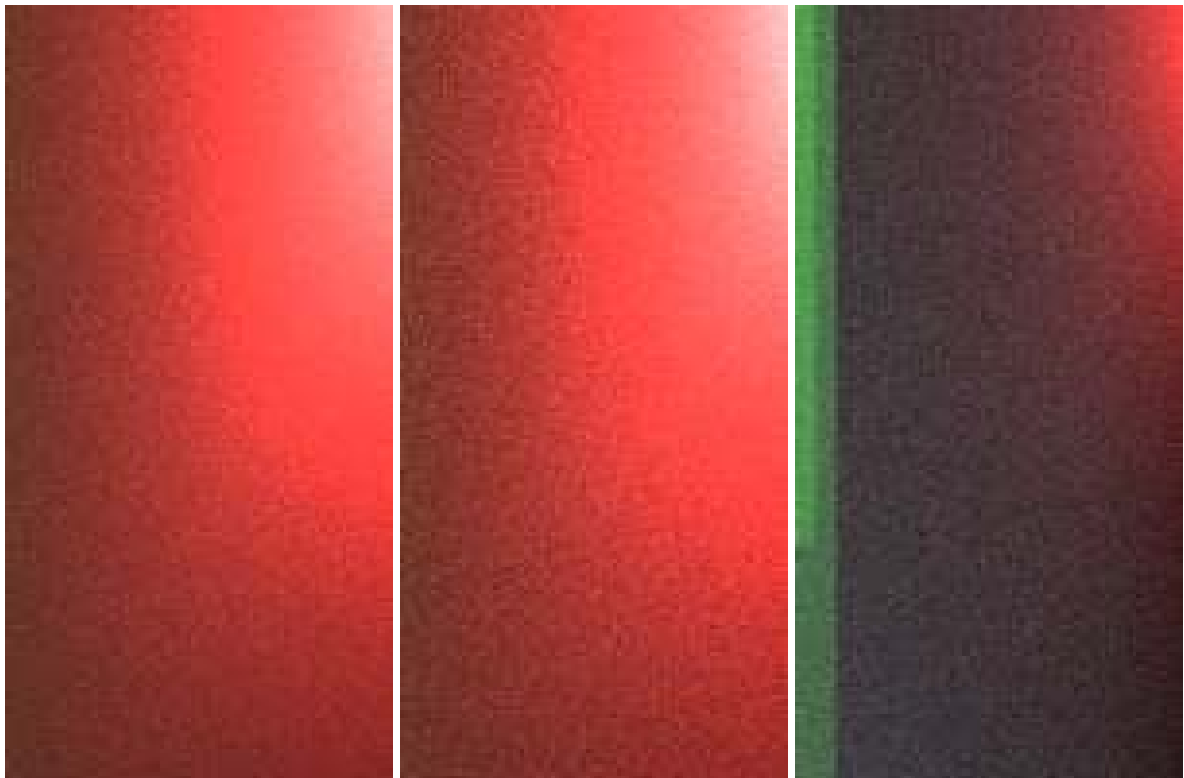
Na rysunku 7.23 przedstawiono porównanie wszystkich modeli. Wersja rozszerzona pozostaje w dalszym ciągu najlepiej radzi sobie ze światłem pochodzącym z lustra odbitego za kamerą. Nie ma niestety poprawy w mieszaniu się światła pomiędzy różnymi źródłami światła. Oprócz głównego porównania problemy są widoczne na rysunkach 7.25 i 7.26. Wersja klasyczna wygląda w tym wypadku najlepiej niestety jednak ze względu na bardzo małą ilość światła jest na niej widoczne mało szczegółów. Ponownie autor skłania się ku wnioskowi, że w testowanym tu układzie wersja hybrydowa wygląda ogólnie najlepiej.



Rys. 7.23: Porównanie sceny "Labirynt", 2000|1000|200 próbek, głębokość 10.

Analizując szum w tym wypadku widzimy powtórkę z poprzedniej grupy testowej. Wersja rozszerzona i hybrydowa wyglądają bardzo podobnie. Wskazanie, która wersja jest lepsza pomiędzy

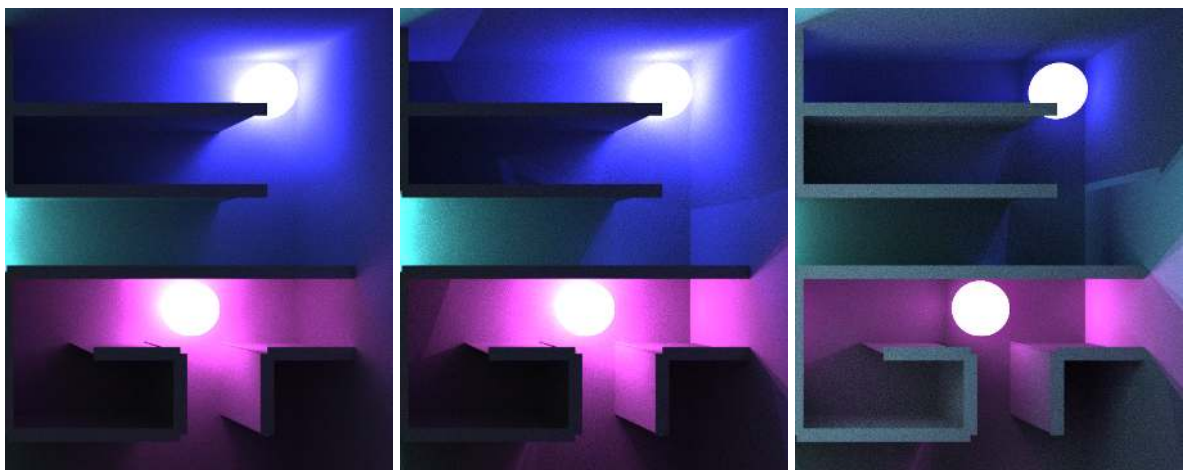
tymi dwoma jest bardzo trudne. Wersja hybrydowa pozostaje bardziej zaszumiona w porównaniu do pozostałych.



Rys. 7.24: Porównanie szumu na scenie "Lustra", 2000|1000|200 próbek, głębokość 10.



Rys. 7.25: Zjawisko mieszania się światła z różnych źródeł światła na scenie "Labirynt", 2000|1000|40 próbek, głębokość 10.

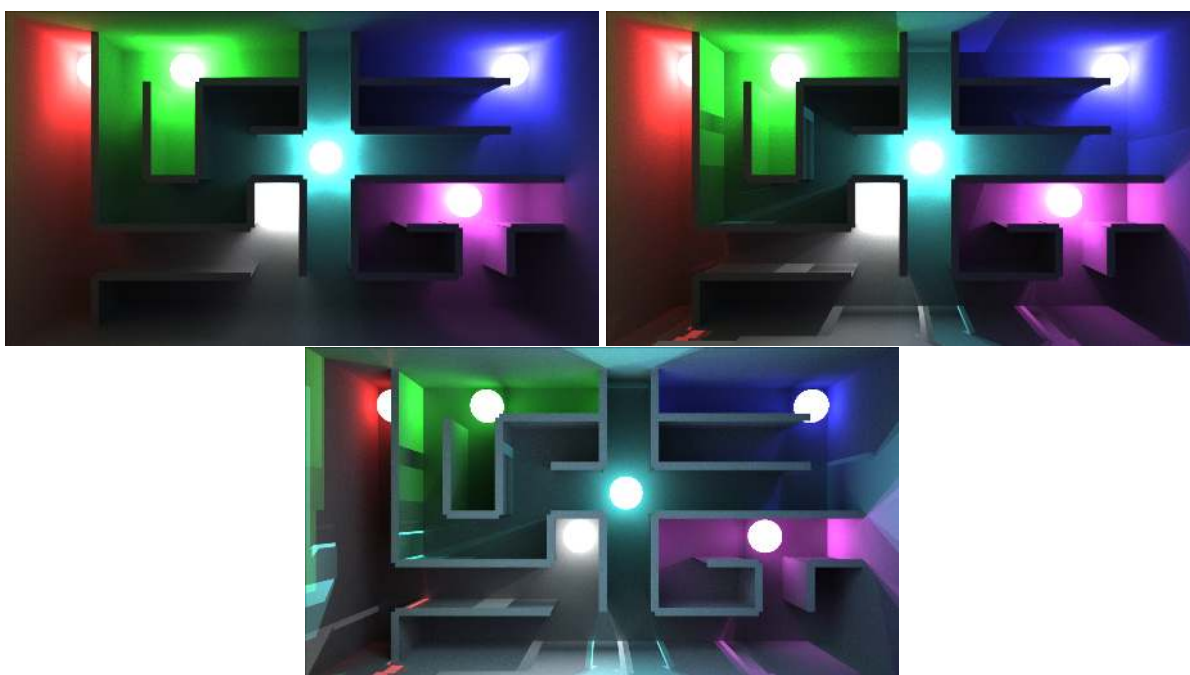


Rys. 7.26: Zbliżenie na wschodnią część sceny "Labirynt", 2000|1000|200 próbek, głębokość 10.

10 godzin - maximum

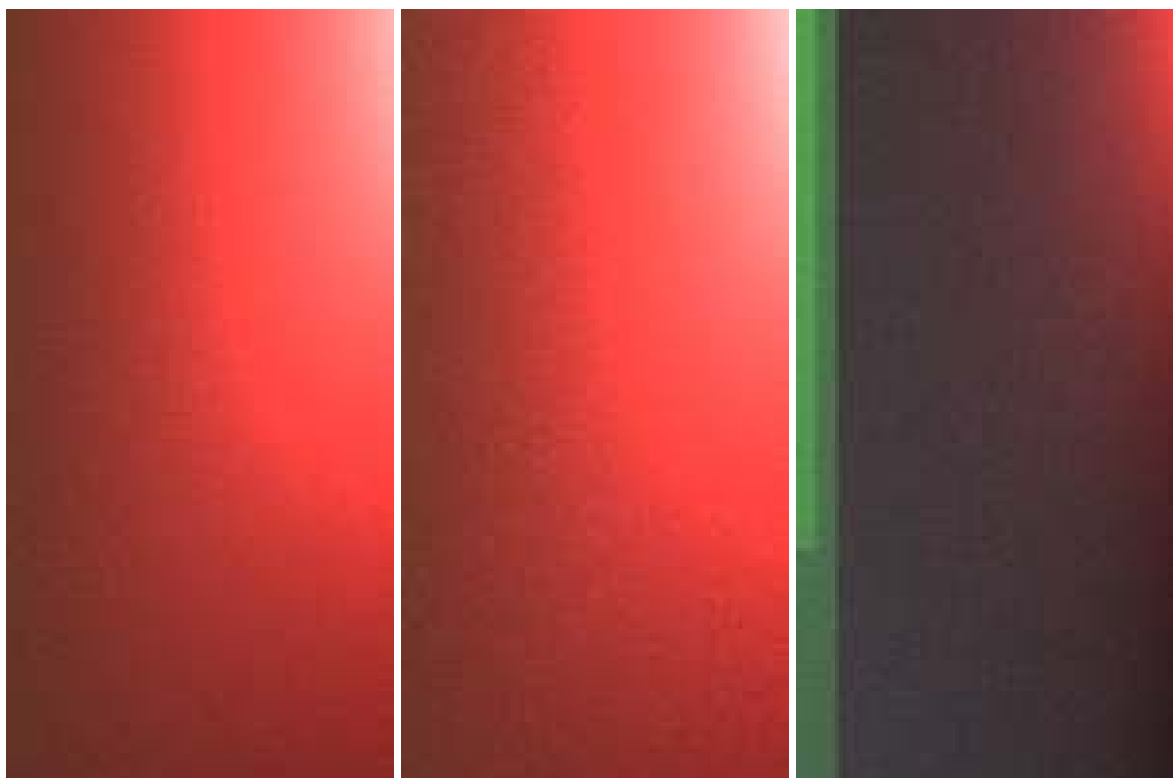
Do finalnego porównania zostały wybrane testy zbliżone czasowo do najdłuższego w wersji klasycznej, czyli testu dla 10000 próbek. Dla wersji hybrydowej najbardziej zbliżony był test dla 5000 próbek (krótszy o 00:08:06.142), a dla wersji rozszerzonej dla 1000 próbek (krótszy o 00:25:01.845).

Na rysunku 7.27 widzimy, że dla modelu klasycznego w dalszym ciągu utrzymuj się problem przesadnie jasnych okolic źródeł światła, które szybko tracą swoją moc. Przez jak widoczne na rysunku 7.29 oraz 7.30 nawet dla 1000 próbek granice pomiędzy ścianą, a podłogą w niektórych miejscach labiryntu jest bardzo słabo widoczna. Wersja hybrydowa wygląda lepiej jedna w dalszym ciągu nie widzimy, że scena jest oświetlona przez światło odbite od lustra znajdującego się za kamerą. Pod tym względem na prowadzenie wysuwa się wersja rozszerzona. Jest ona najlepiej oświetlona. Problemem pozostaje mieszanie się światła z różnych źródeł, które wygląda mniej naturalnie niż w przypadku wersji klasycznej.



Rys. 7.27: Porównanie sceny "Labirynt", 1000|5000|1000 próbek, głębokość 10.

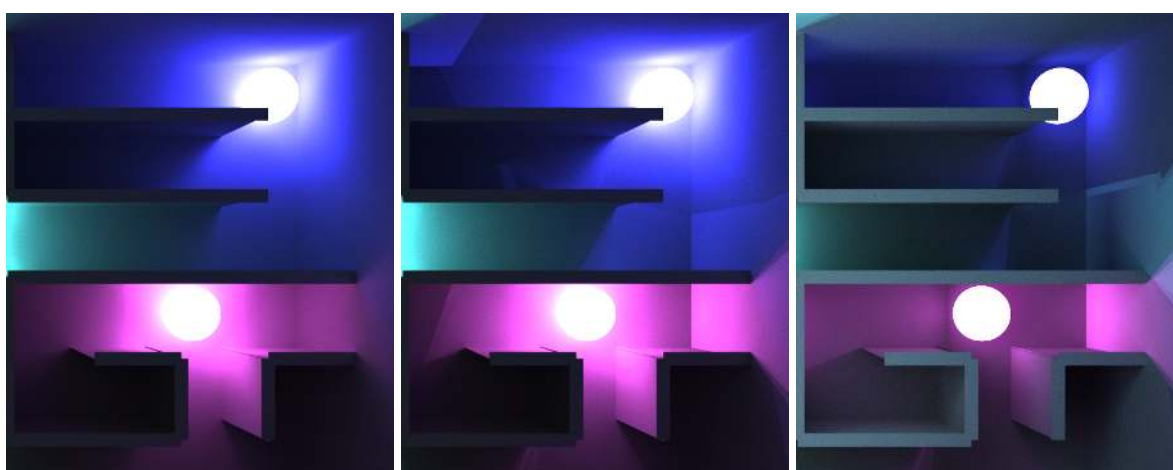
Analizując szum na scenie na rysunku 7.28 ciężko jest wskazać różnicę pomiędzy wersją klasyczną i rozszerzoną. Wygenerowane obrazy z obu scen oferują obraz podobnej jakości. Podobnie jak w poprzednich porównaniach wersja hybrydowa osiąga najgorszy wynik.



Rys. 7.28: Porównanie szumu na scenie "Lustra", 1000|5000|1000 próbek, głębokość 10.



Rys. 7.29: Zjawisko mieszania się światła z różnych źródeł światła na scenie "Labirynt", 10000|5000|1000 próbek, głębokość 10.



Rys. 7.30: Zbliżenie na wschodnią część sceny "Labirynt", 10000|5000|1000 próbek, głębokość 10.

Podsumowanie

Dla sceny labirynt wyniki wszystkich wersji są najbardziej do siebie zbliżone pod względem wizualnym. Przy znormalizowaniu wyników pod względem czasu wszystkie wersje oferują po-

dobne poziomy szumu na końcowych obrazach. W wersji klasycznej obraz jest bardzo ciemny, przez wiele szczegółów znajdujących się dalej od źródeł światła posiada mało szczegółów. Dodatkowo elementy znajdujące się blisko źródeł światła są zbyt jasne co w połączeniu tworzy bardzo dziwny efekt na końcowej scenie. Wersja hybrydowa poprawia tą sytuację znacząco pod względem oświetlenia sceny jednak wprowadza ona problemy z mieszaniem się światła z różnych źródeł. Wersja rozszerzona także cierpi na ten problem. Jest on nawet w niektórych bardziej intensywny. Natomiast wersja ta jako jedyna poprawnie wzięła pod uwagę fakt, że za kamerą znajduje się lustro, które odbija światło z powrotem na scenę.

W tym wypadku ciężko jest jednoznacznie wskazać, która wersja jest najlepsza. Zależy to od elementu sceny, na którym najbardziej nam zależy. Zdaniem autora dla tej sceny wersja rozszerzona także wypadła najlepiej ponieważ jako jedyna poprawnie za symulowała wpływ lustra znajdującego się za sceną.

Rozdział 8

Podsumowanie

8.1. Wnioski

Głównym celem badań przeprowadzonych w tej pracy było przedstawienie wad i zalet proponowanej zmiany w algorytmie path-tracingu. Zmiany te są reprezentowane przede wszystkim przez rozszerzony. W klasycznej wersji algorytmu promienie odbijają się losowo na scenie od obiektów rozpraszających światło. Jak było we wstępie do tej pracy, jest to nieafektywne ponieważ nie mamy gwarancji, że wysłany promień trafi w źródło światła. W związku z czym potrzebujemy wielu próbek na pojedynczy piksel obrazu aby uzyskać informację. Model rozszerzony zachowuje losowe odbicia, ale dla każdego trafionego punktu na scenie śledzi promień do źródła światła.

Jak wykazały badania przeprowadzone w tej pracy jest to szczególnie skuteczne w przypadku scen z pojedynczym źródłem światła. Model rozszerzony jest nie gorszy czasowo od wersji klasycznej dla takiej samej ilości próbek, oferując przy tym wygenerowane obrazy o znacznie lepszej jakości. Przede wszystkim wyróżnia się ilość szumu w porównaniu do innych modeli oraz jakość symulacji zjawiska załamania światła. Przy większej ilości źródeł oświetlenia aplikacja ta spowalnia. W testach dla sceny "Lustra" wersja klasyczna kończy test dla takiej samej ilości próbek w zdecydowanie szybszym czasie. Jak wykazano w porównaniu graficznym jeżeli porównamy wyniki z testów najbardziej do siebie zbliżonych czasowo wersja rozszerzona w dalszym ciągu oferuje znaczną przewagę pod względem ilości czumu na obrazie oraz symulacji odbić w obiektach na scenie. Nawet w przypadku trudnych scen, z dużą ilością źródeł światła oraz obiektów wersja rozszerzona w dalszym ciągu prezentuje zalety na pozostałych modelach. Jako jedyna zdołała ona poprawnie za symulować wpływ światła odbijanego od lustra znajdującego się za kamerą na oświetlenie sceny. Niestety, porównując wygenerowane obrazy z testów najbardziej zbliżonych czasowo, nie oferuje w tym wypadku przewagi pod względem ilości szumu.

Wersja rozszerzona nie jest ostatecznym rozwiązaniem wszystkich problemów w algorytmie path-tracingu. Istnieją sceny, w których konfiguracja obiektów i źródeł światła sprawi, że model rozszerzony będzie znacznie gorszy od wersji klasycznej algorytmu. Jednak w szczególności dla scen z mniejszą ilością źródeł światła, algorytm ten oferuje lepsze wyniki wizualne oraz ułatwia symulacje zjawisk fizycznych związanych ze światłem. Model ten może posłużyć jako baza do stworzenia nowych, bądź optymalizacji istniejących, obiektywnych rendererów. Pomimo niedoskonałości modyfikacje przedstawione w tej pracy mogą posłużyć jako baza do rozwoju nowych rozwiązań w dziedzinie generowania grafiki.

8.2. Dodatkowe badania

Model hybrydowy powstał jako jedna z prób optymalizacji modelu rozszerzonego poprzez ograniczanie sprawdzania oświetlenia tylko do pierwszych odbić każdego promienia. Doliczanie dodatkowego światła nie przyniosło oczekiwanego efektu. Jak widzimy na porównaniach szumu ten model osiąga takie same wyniki jak wersja klasyczny przy takiej samej ilości próbek. Ponadto w przypadku sceny "Cornel Box" widzimy, że model ten może prowadzić do nadmiernej ekspozycji niektórych elementów sceny. Największą zaletą tego modelu jest zwiększenie rozpiętości tonalnej końcowych obrazów. Pomaga to z wyciągnięciem szczegółów w obrazie, szczególnie na granicach pomiędzy obiektami oraz na wyeksponowaniu granicy pomiędzy elementem oświetlonym, a cieniem. Model ten najlepiej wypadł w przypadku sceny "Labirynt" gdzie pomógł on z wyciągnięciem szczegółów sceny, nie mając tak negatywnego wpływu na przejścia między kolorami jak model rozszerzony. Był on jednak na tyle wolniejszy od wersji klasycznej, że normalizując pod względem czasu ukończenia renderu musimy wybrać test o mniejszej ilości próbek. Powoduje to, że końcowy obraz jest bardziej zaszumiony. W stosunku do wersji rozszerzonej nie udało się zasymulować poprawnie efektu światła odbitego od lustra znajdującego się za kamerą.

Niestety autor nie widzi możliwości sensownych poprawek, które uzasadniły by dalszy rozwój tej wersji. Jeżeli zależy nam na jakości w zdecydowanej większości przypadków lepiej będzie wykorzystać model rozszerzony. Natomiast jeżeli chcemy uzyskać jak najszybszy render wersja klasyczna pozostaje najlepsza.

8.3. Dalszy rozwój

Istnieje kilka elementów, z których autor nie jest zadowolony bądź po prostu ich zabrakło ze względu na ograniczenia czasowe. Przede wszystkim brakuje elementu, który reguluje moc źródła światła w zależności od odległości. Obecnie jeżeli punkt sceny posiada nieprzerwaną linię łączącą go z źródłem światła zostanie on oświetlony tak samo mocno niezależnie od odległości. Nie jest to realistyczne zachowanie. Ponadto system ekstremów nie sprawdza się tak dobrze jak autor na liczył. Szczególnie przy obiektach znajdujących się bardzo daleko i bardzo blisko. Dla obiektów znajdujących się bardzo blisko źródła światła tworzy to efekt skupienia się światła przy ekstremum. Wygląda to nienaturalnie jakby źródło nie emitowało światła równomiernie. Przy dalekich źródłach światła sprawdzanie wszystkich ekstremów nie ma sensu. Zdaniem autora wprowadzenie ograniczenia na używanie ekstremów wyeliminowałoby wiele niepotrzebnych sprawdzeń oraz pozwoliło by na uzyskanie lepszego wizualnie efektu. Należałoby także zmienić system wcześniejszego przerywania działania algorytmu. Obecny warunek, który wymaga trafienia dwóch obiektów rozpraszających światło pod rząd nie jest wystarczający. Jeżeli uzależnić by wcześniejsze przerwanie od odbicia się od oświetlanego elementu sceny pozwoliło by ograniczyć potrzebną głębokość odbić i skupić się tylko na ścieżkach w nieoświetlonych elementach sceny. Ważnym elementem było by tutaj wprowadzenie algorytmu osłabiającego moc oświetlenia wraz z głębokością odbicia. Wersja CUDA-3.6 pokazała, że bardzo łatwo jest sprawić, żeby cienie były zbyt jasne w efekcie pogarszając końcowy obraz.

Istnieje wiele innych modyfikacji i ulepszeń, które można by wprowadzić do testowanej aplikacji. Wiele z nich zostało pominiętych ze względu na ograniczenia czasowe. Kod aplikacji jest dostępny na platformie GitHub autora. Jeżeli ktoś jest zainteresowany dalszym rozwojem tej aplikacji autor zaprasza do odwiedzenia swoje repozytorium dostępnego pod adresem <https://github.com/AdamStudies-PWR/Improved-Path-Tracer>.

Literatura

- [1] I. A. Astuti, I. H. Purwanto, T. Hidayat, D. A. Satria, R. Purnama, i in. Comparison of time, size and quality of 3d object rendering using render engine eevee and cycles in blender. *2022 5th International Conference of Computer and Informatics Engineering (IC2IE)*, strony 54–59. IEEE, 2022.
- [2] D. Azinovic, T.-M. Li, A. Kaplanyan, M. Nießner. Inverse path tracing for joint material and lighting estimation. *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, strony 2442–2451, 2019.
- [3] K. Beason. smallpt: Global illumination in 99 lines of c++, 2014. <https://www.kevinbeason.com/smallpt/>[Dostęp dnia 10 listopada 2023].
- [4] D. D. Cline. smallpt: Global illumination in 99 lines of c++ a ray tracer by kevin beason. Raport instytutowy, Oklahoma State University, 2013. <https://drive.google.com/file/d/0B8g97JkuSSBwUENiWTJXeGtTOHFmSm51UC01YWtCZw/view?resourcekey=0-BSMES1GmPEtIIcv6EgBTjw>[Dostęp dnia 10 listopada 2023].
- [5] P. Debevec. Rendering synthetic objects into real scenes: Bridging traditional and image-based graphics with global illumination and high dynamic range photography. *Acm siggraph 2008 classes*, strony 1–10, 2008.
- [6] J. T. Kajiya. The rendering equation. *Proceedings of the 13th annual conference on Computer graphics and interactive techniques*, strony 143–150, 1986.
- [7] E. Lafortune, Y. Willems. Bi-directional path tracing. *Proceedings of Third International Conference on Computational Graphics and Visualization Techniques (Compugraphics)*, 93, 01 1998.
- [8] mathworld.wolfram.com. Point-line distance–3-dimensional. <https://mathworld.wolfram.com/Point-LineDistance3-Dimensional.html>[Dostęp dnia 11 maja 2024].
- [9] Y. Ouyang, S. Liu, M. Kettunen, M. Pharr, J. Pantaleoni. Restir gi: Path resampling for real-time path tracing. *Computer Graphics Forum*, wolumen 40, strony 17–29. Wiley Online Library, 2021.
- [10] A. Pajot, L. Barthe, M. Paulin, P. Poulin. Combinatorial bidirectional path-tracing for efficient hybrid cpu/gpu rendering. *Computer Graphics Forum*, wolumen 30, strony 315–324. Wiley Online Library, 2011.
- [11] S. Sengupta, J. Gu, K. Kim, G. Liu, D. W. Jacobs, J. Kautz. Neural inverse rendering of an indoor scene from a single image. *Proceedings of the IEEE/CVF International Conference on Computer Vision*, strony 8598–8607, 2019.
- [12] O. Villar. *Learning Blender*. Addison-Wesley Professional, 2021.

-
- [13] Z. Wang, J. Philion, S. Fidler, J. Kautz. Learning indoor inverse rendering with 3d spatially-varying lighting. *Proceedings of the IEEE/CVF International Conference on Computer Vision*, strony 12538–12547, 2021.
 - [14] L. Wu, R. Zhu, M. B. Yaldiz, Y. Zhu, H. Cai, J. Matai, F. Porikli, T.-M. Li, M. Chandraker, R. Ramamoorthi. Factorized inverse path tracing for efficient and accurate material-lighting estimation. *Proceedings of the IEEE/CVF International Conference on Computer Vision*, strony 3848–3858, 2023.
 - [15] R. Zhu, Z. Li, J. Matai, F. Porikli, M. Chandraker. Irisformer: Dense vision transformers for single-image inverse rendering in indoor scenes. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, strony 2822–2831, 2022.

Spis rysunków

1.1.	Diagram przedstawiający uproszczony schemat algorytmu klasycznego.	9
1.2.	Przykład działania modelu klasycznego.	9
1.3.	Diagram przedstawiający uproszczony schemat algorytmu rozszerzonego	11
1.4.	Przykład działania modelu rozszerzonego.	12
1.5.	Diagram przedstawiający uproszczony schemat algorytmu hybrydowego.	13
1.6.	Przykład działania modelu hybrydowego.	13
2.1.	Scena Cornell Box wygenerowana przez smallpt, 5000 próbek, głębokość 10	15
3.1.	Scena Cornell Box wygenerowana przez <i>tracer(cuda-3)</i> , 5000 próbek, głębokość 10	25
3.2.	Scena Lustra wygenerowana przez <i>tracer(cuda-3)</i> , 5000 próbek, głębokość 10 . . .	26
3.3.	Scena Labirynt wygenerowana przez <i>tracer(cuda-3)</i> , 5000 próbek, głębokość 10 . .	27
4.1.	Problem znikającego światła widoczny na scenie "Labirynt"i "Lustra"(40 próbek, głębokość 10).	28
4.2.	Scena "Lustra"przed i po rozwiązaniu problemu (40 próbek, głębokość 10).	29
4.3.	Scena "Labirynt"przed i po rozwiązaniu problemu (40 próbek, głębokość 10). . . .	29
4.4.	Wczesna wersja sceny labirynt, widoczny problem z wielkością renderownych kul (10 próbek, głębokość 4).	29
4.5.	Diagram przedstawiający koncept pola widzenia.	30
4.6.	Stopień oświetlenia sceny "Cornell Box"przez źródło światła.	36
4.7.	Stopień oświetlenia sceny "Lustra"przez źródło światła.	36
4.8.	Stopień oświetlenia sceny "Cornell Box"przez źródło światła.	36
5.1.	Czas renderu sceny "Cornell Box"pomiedzy platformą I i II, głębokość 10, próbki 40 - 400.	42
5.2.	Czas renderu sceny "Cornell Box"pomiedzy platformą I i II, głębokość 10, próbki 40 - 400.	42
5.3.	Czas renderu sceny "Lustra"pomiedzy platformą I i II, głębokość 10, próbki 40 - 400.	43
5.4.	Czas renderu sceny "Lustra"pomiedzy platformą I i II, głębokość 10, próbki 1000 - 10000.	43
5.5.	Czas renderu sceny "Labirynt"pomiedzy platformą I i II, głębokość 10, próbki 40 - 400.	44
5.6.	Czas renderu sceny "Labirynt"pomiedzy platformą I i II, głębokość 10, próbki 1000 - 10000.	44
5.7.	Porównanie render sceny "Cronell Box". Od lewej: platforma I, platforma II. Przedział 40 - 400, głębokość 10.	71
5.8.	Porównanie render sceny "Cronell Box". Od lewej: platforma I, platforma II. Przedział 1000 - 10000, głębokość 10.	71
5.9.	Porównanie render sceny "Lustra". Od lewej: platforma I, platforma II. Przedział 40 - 400, głębokość 10.	72

5.10. Porównanie render sceny "Lustra". Od lewej: platforma I, platforma II. Przedział 1000 - 10000, głębokość 10.	72
5.11. Porównanie render sceny "Labirynt". Od lewej: platforma I, platforma II. Przedział 40 - 400, głębokość 10.	72
5.12. Porównanie render sceny "Labirynt". Od lewej: platforma I, platforma II. Przedział 1000 - 10000, głębokość 10.	72
6.1. Porównanie jakości rendererów dla sceny "Cornell Box" pomiędzy wersją CUDA-3.5 i CUDA-3.6. Zbliżenie na lustrzaną kulę, 40 próbek, głębokość 10.	88
6.2. Porównanie jakości rendererów dla sceny "Cornell Box" pomiędzy wersją CUDA-3.5 i CUDA-3.6. Zbliżenie na szklaną kulę, 40 próbek, głębokość 10.	88
6.3. Porównanie jakości rendererów dla sceny "Cornell Box" pomiędzy wersją CUDA-3.5 i CUDA-3.6. Zbliżenie na lustrzaną kulę, 400 próbek, głębokość 10.	89
6.4. Porównanie jakości rendererów dla sceny "Cornell Box" pomiędzy wersją CUDA-3.5 i CUDA-3.6. Zbliżenie na szklaną kulę, 400 próbek, głębokość 10.	89
6.5. Porównanie jakości rendererów dla sceny "Cornell Box" pomiędzy wersją CUDA-3.5 i CUDA-3.6. Zbliżenie na lustrzaną kulę, 5000 próbek, głębokość 10.	89
6.6. Porównanie jakości rendererów dla sceny "Cornell Box" pomiędzy wersją CUDA-3.5 i CUDA-3.6. Zbliżenie na szklaną kulę, 5000 próbek, głębokość 10.	89
6.7. Porównanie jakości rendererów dla sceny "Lustra" pomiędzy wersją CUDA-3.5 i CUDA-3.6. Zbliżenie na szybę, 40 próbek, głębokość 10.	91
6.8. Porównanie jakości rendererów dla sceny "Lustra" pomiędzy wersją CUDA-3.5 i CUDA-3.6. Zbliżenie na lustro, 40 próbek, głębokość 10.	91
6.9. Porównanie jakości rendererów dla sceny "Lustra" pomiędzy wersją CUDA-3.5 i CUDA-3.6. Zbliżenie na szybę, 40 próbek, głębokość 10.	92
6.10. Porównanie jakości rendererów dla sceny "Lustra" pomiędzy wersją CUDA-3.5 i CUDA-3.6. Zbliżenie na lustro, 40 próbek, głębokość 10.	92
6.11. Porównanie jakości rendererów dla sceny "Lustra" pomiędzy wersją CUDA-3.5 i CUDA-3.6. Zbliżenie na szybę, 40 próbek, głębokość 10.	92
6.12. Porównanie jakości rendererów dla sceny "Lustra" pomiędzy wersją CUDA-3.5 i CUDA-3.6. Zbliżenie na lustro, 40 próbek, głębokość 10.	92
6.13. Porównanie jakości rendererów dla sceny "Labirynt" pomiędzy wersją CUDA-3.5 i CUDA-3.6. Zbliżenie na obszar centralny, 40 próbek, głębokość 10.	94
6.14. Porównanie jakości rendererów dla sceny "Labirynt" pomiędzy wersją CUDA-3.5 i CUDA-3.6. Zbliżenie na obszar wschodni, 40 próbek, głębokość 10.	95
6.15. Porównanie jakości rendererów dla sceny "Labirynt" pomiędzy wersją CUDA-3.5 i CUDA-3.6. Zbliżenie na korytarz południowo-wschodni, 40 próbek, głębokość 10.	95
6.16. Porównanie jakości rendererów dla sceny "Labirynt" pomiędzy wersją CUDA-3.5 i CUDA-3.6. Zbliżenie na obszar centralny, 1000 próbek, głębokość 10.	96
6.17. Porównanie jakości rendererów dla sceny "Labirynt" pomiędzy wersją CUDA-3.5 i CUDA-3.6. Zbliżenie na obszar wschodni, 1000 próbek, głębokość 10.	96
6.18. Porównanie jakości rendererów dla sceny "Labirynt" pomiędzy wersją CUDA-3.5 i CUDA-3.6. Zbliżenie na korytarz południowo-wschodni, 1000 próbek, głębokość 10.	97
7.1. Porównanie sceny "Cornell Box", 40 próbek, głębokość 10.	101
7.2. Porównanie szumu na scenie "Cornell Box", 40 próbek, głębokość 10.	101
7.3. Zbliżenie na lustrzaną kulę, scena "Cornell Box", 40 próbek, głębokość 10.	102
7.4. Porównanie sceny "Cornell Box", 400 próbek, głębokość 10.	102
7.5. Porównanie szumu na scenie "Cornell Box", 400 próbek, głębokość 10.	103
7.6. Zbliżenie na lustrzaną kulę, scena "Cornell Box", 400 próbek, głębokość 10.	103

7.7. Porównanie sceny "Cornell Box", 10000 próbek, głębokość 10.	104
7.8. Porównanie szumu na scenie "Cornell Box", 10000 próbek, głębokość 10.	104
7.9. Zbliżenie na lustrzaną kulę, scena "Cornell Box", 10000 próbek, głębokość 10. . .	104
7.10. Porównanie sceny "Lustra", 80 80 40 próbek, głębokość 10.	107
7.11. Porównanie szumu na scenie "Lustra", 80 80 40 próbek, głębokość 10.	107
7.12. Zbliżenie na kulę widoczną po drugiej stronie tafli szkła, scena "Lustra", 80 80 40 próbek, głębokość 10.	108
7.13. Porównanie sceny "Lustra", 1000 1000 400 próbek, głębokość 10.	108
7.14. Porównanie szumu na scenie "Lustra", 1000 1000 400 próbek, głębokość 10.	109
7.15. Zbliżenie na kulę widoczną po drugiej stronie tafli szkła, scena "Lustra", 1000 1000 400 próbek, głębokość 10.	109
7.16. Porównanie sceny "Lustra", 10000 10000 2000 próbek, głębokość 10.	110
7.17. Porównanie szumu na scenie "Lustra", 10000 10000 2000 próbek, głębokość 10. . .	110
7.18. Zbliżenie na kulę widoczną po drugiej stronie tafli szkła, scena "Lustra", 10000 10000 2000 próbek, głębokość 10.	110
7.19. Porównanie sceny "Labirynt", 400 200 40 próbek, głębokość 10.	113
7.20. Porównanie szumu na scenie "Lustra", 400 200 40 próbek, głębokość 10.	114
7.21. Zjawisko mieszania się światła z różnych źródeł światła na scenie "Labirynt", 400 200 40 próbek, głębokość 10.	114
7.22. Zbliżenie na wschodnią część sceny "Labirynt", 400 200 40 próbek, głębokość 10. .	115
7.23. Porównanie sceny "Labirynt", 2000 1000 200 próbek, głębokość 10.	115
7.24. Porównanie szumu na scenie "Lustra", 2000 1000 200 próbek, głębokość 10.	116
7.25. Zjawisko mieszania się światła z różnych źródeł światła na scenie "Labirynt", 2000 1000 40 próbek, głębokość 10.	116
7.26. Zbliżenie na wschodnią część sceny "Labirynt", 2000 1000 200 próbek, głębokość 10.	116
7.27. Porównanie sceny "Labirynt", 1000 5000 1000 próbek, głębokość 10.	117
7.28. Porównanie szumu na scenie "Lustra", 1000 5000 1000 próbek, głębokość 10. . . .	118
7.29. Zjawisko mieszania się światła z różnych źródeł światła na scenie "Labirynt", 10000 5000 1000 próbek, głębokość 10.	118
7.30. Zbliżenie na wschodnią część sceny "Labirynt", 10000 5000 1000 próbek, głębo- kość 10.	118

Spis tabel

3.1. Specyfikacje platformy testowej I (wydajna)	21
3.2. Specyfikacje platformy testowej I	21
3.3. Wersje oprogramowania systemu operacyjnego	22
3.4. Wersje kompilatorów i oprogramowania wspomagającego	22
3.5. Domyślne parametry testowe	22
5.1. Czas potrzebny na wygenerowanie obrazu, wersja CPU, platforma I, głębokość 10, próbki 40 - 400	39
5.2. Czas potrzebny na wygenerowanie obrazu, wersja CPU, platforma I, głębokość 10, próbki 1000 - 10000	39
5.3. Czas renderu względem sceny "Lustra", platforma I, głębokość 10.	40
5.4. Zużycie pamięci systemowej, wersja CPU, platforma I, dane w MiB	40
5.5. Czas potrzebny na wygenerowanie obrazu, wersja CPU, platforma II, głębokość 10, próbki 40 - 400	40
5.6. Czas potrzebny na wygenerowanie obrazu, wersja CPU platforma II, głębokość 10, próbki 1000 - 10000	41
5.7. Czas renderu względem sceny "Lustra", platforma II, głębokość 10.	41
5.8. Zużycie pamięci systemowej, wersja CPU, platforma II, dane w MiB	41
5.9. Porównanie czasu renderu sceny "Cornell Box" pomiędzy platformami, głębokość 10.	42
5.10. Porównanie czasu renderu sceny "Lustra" pomiędzy platformami, głębokość 10.	43
5.11. Porównanie czasu renderu sceny "Labirynt" pomiędzy platformami, głębokość 10.	44
5.12. Czas potrzebny na wygenerowanie sceny Cornell Box, platforma I, głębokość 10, próbki 40 - 400.	45
5.13. Czas potrzebny na wygenerowanie sceny Cornell Box, platforma I, głębokość 10, próbki 1000 - 10000.	45
5.14. Porównanie czasu renderu sceny "Cornell Box" pomiędzy wersją CPU, a CUDA-1, platforma I, głębokość 10.	45
5.15. Zużycie pamięci systemowej dla sceny "Cornell Box", platforma I, głębokość 10, dane w MiB.	46
5.16. Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 40 - 400	46
5.17. Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 1000 - 10000	46
5.18. Porównanie czasu renderu sceny "Cornell Box" pomiędzy wersją CPU, a CUDA-1, platforma I, głębokość 10.	46
5.19. Zużycie pamięci systemowej dla sceny "Cornell Box", platforma II, głębokość 10	46
5.20. Czas potrzebny na wygenerowanie sceny "Lustra", platforma II, głębokość 10, próbki 40 - 400.	47
5.21. Czas potrzebny na wygenerowanie sceny "Lustra", platforma II, głębokość 10, próbki 1000 - 10000.	47

5.22. Porównanie czasu renderu sceny "Lustra" pomiędzy wersją CPU, a CUDA-1, platforma I, głębokość 10.	47
5.23. Zużycie pamięci systemowej dla sceny "Labirynt", platforma II, głębokość 10, dane w MiB.	47
5.24. Czas potrzebny na wygenerowanie sceny "Lustra", platforma II, głębokość 10, próbki 40 - 400.	48
5.25. Czas potrzebny na wygenerowanie sceny "Lustra", platforma II, głębokość 10, próbki 40 - 400.	48
5.26. Porównanie czasu renderu sceny "Lustra" pomiędzy wersją CPU, a CUDA-1, platforma II, głębokość 10.	48
5.27. Zużycie pamięci systemowej dla sceny "Lustra", platforma II, głębokość 10, dane w MiB.	48
5.28. Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 40 - 400.	48
5.29. Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 1000 - 10000.	49
5.30. Porównanie czasu renderu sceny "Labirynt" pomiędzy wersją CPU, a CUDA-1, platforma I, głębokość 10.	49
5.31. Zużycie pamięci systemowej dla sceny "Labirynt", platforma I, głębokość 10, dane w MiB.	49
5.32. Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 40 - 400.	49
5.33. Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 1000 - 10000.	49
5.34. Porównanie czasu renderu sceny "Labirynt" pomiędzy wersją CPU, a CUDA-1, platforma II, głębokość 10.	50
5.35. Zużycie pamięci systemowej dla sceny "Labirynt", platforma I, głębokość 10, dane w MiB.	50
5.36. Zużycie pamięci GPU, wersja CUDA-1, głębokość 10, dane w MiB.	50
5.37. Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma I, głębokość 10, próbki 40 - 400.	51
5.38. Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma I, głębokość 10, próbki 1000 - 10000.	51
5.39. Porównanie czasu renderu sceny "Cornell Box" pomiędzy wersją CPU, a CUDA-2, platforma I, głębokość 10.	51
5.40. Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 40 - 400.	51
5.41. Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 1000 - 10000.	52
5.42. Porównanie czasu renderu sceny "Cornell Box" pomiędzy wersją CPU, a CUDA-II, platforma II, głębokość 10.	52
5.43. Czas potrzebny na wygenerowanie sceny "Lustra", platforma I, głębokość 10, próbki 40 - 400.	52
5.44. Czas potrzebny na wygenerowanie sceny "Lustra", platforma I, głębokość 10, próbki 1000 - 10000.	52
5.45. Porównanie czasu renderu sceny "Lustra" pomiędzy wersją CPU, a CUDA-2, platforma I, głębokość 10.	52
5.46. Czas potrzebny na wygenerowanie sceny "Lustra", platforma II, głębokość 10, próbki 40 - 400.	53

5.47. Czas potrzebny na wygenerowanie sceny "Lustra", platforma II, głębokość 10, próbki 1000 - 10000.	53
5.48. Porównanie czasu renderu sceny "Lustra" pomiędzy wersją CPU, a CUDA-2, platforma Z, głębokość 10.	53
5.49. Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 40 - 400.	53
5.50. Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 1000 - 10000.	53
5.51. Porównanie czasu renderu sceny "Labirynt" pomiędzy wersją CPU, a CUDA-2, platforma I, głębokość 10.	54
5.52. Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 40 - 400.	54
5.53. Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 1000 - 10000.	54
5.54. Porównanie czasu renderu sceny "X" pomiędzy wersją CPU, a CUDA-Y, platforma Z, głębokość 10.	54
5.55. Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma I, głębokość 10, próbki 40 - 400.	55
5.56. Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma I, głębokość 10, próbki 1000 - 10000.	55
5.57. Porównanie czasu renderu sceny "Cornell Box" pomiędzy wersją CPU, a CUDA-3, platforma I, głębokość 10.	55
5.58. Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 40 - 400.	55
5.59. Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 1000 - 10000.	56
5.60. Porównanie czasu renderu sceny "Cornell Box" pomiędzy wersją CPU, a CUDA-3, platforma II, głębokość 10.	56
5.61. Czas potrzebny na wygenerowanie sceny "Lustra", platforma I, głębokość 10, próbki 40 - 400.	56
5.62. Czas potrzebny na wygenerowanie sceny "Lustra", platforma I, głębokość 10, próbki 1000 - 10000.	56
5.63. Porównanie czasu renderu sceny "Lustra" pomiędzy wersją CPU, a CUDA-3, platforma I, głębokość 10.	56
5.64. Czas potrzebny na wygenerowanie sceny "Lustra", platforma II, głębokość 10, próbki 40 - 400.	57
5.65. Czas potrzebny na wygenerowanie sceny "Lustra", platforma II, głębokość 10, próbki 1000 - 10000.	57
5.66. Porównanie czasu renderu sceny "Lustra" pomiędzy wersją CPU, a CUDA-3, platforma II, głębokość 10.	57
5.67. Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 40 - 400.	57
5.68. Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 1000 - 10000.	58
5.69. Porównanie czasu renderu sceny "Labirynt" pomiędzy wersją CPU, a CUDA-3, platforma I, głębokość 10.	58
5.70. Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 40 - 400.	58
5.71. Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 1000 - 10000.	58

5.72. Porównanie czasu renderu sceny "Labirynt" pomiędzy wersją CPU, a CUDA-3, platforma II, głębokość 10.	58
5.73. Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma I, głębokość 10, próbki 40 - 400.	59
5.74. Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma I, głębokość 10, próbki 1000 - 10000.	59
5.75. Porównanie czasu renderu sceny "Cornell Box" pomiędzy wersją CPU, a CUDA-4, platforma I, głębokość 10.	59
5.76. Zużycie pamięci systemowej dla sceny "Cornell Box", platforma I, głębokość 10, dane w MiB.	59
5.77. Zużycie pamięci GPU dla sceny "Cornell Box", platforma I, głębokość 10, dane w MiB	60
5.78. Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 40 - 400.	60
5.79. Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 1000 - 10000.	60
5.80. Porównanie czasu renderu sceny "Cornell Box" pomiędzy wersją CPU, a CUDA-4, platforma II, głębokość 10.	60
5.81. Zużycie pamięci systemowej dla sceny "Cornell Box", platforma II, głębokość 10, dane w MiB.	60
5.82. Zużycie pamięci GPU dla sceny "X", platforma Y, głębokość 10, dane w MiB	61
5.83. Czas potrzebny na wygenerowanie sceny "Lustra", platforma I, głębokość 10, próbki 40 - 400.	61
5.84. Czas potrzebny na wygenerowanie sceny "Lustra", platforma I, głębokość 10, próbki 1000 - 10000.	61
5.85. Porównanie czasu renderu sceny "Lustra" pomiędzy wersją CPU, a CUDA-4, platforma I, głębokość 10.	61
5.86. Zużycie pamięci systemowej dla sceny "Lustra", platforma I, głębokość 10, dane w MiB.	61
5.87. Zużycie pamięci GPU dla sceny "Lustra", platforma I, głębokość 10, dane w MiB . .	62
5.88. Czas potrzebny na wygenerowanie sceny "Lustra", platforma II, głębokość 10, próbki 40 - 400.	62
5.89. Czas potrzebny na wygenerowanie sceny "Lustra", platforma II, głębokość 10, próbki 1000 - 10000.	62
5.90. Porównanie czasu renderu sceny "Lustra" pomiędzy wersją CPU, a CUDA-4, platforma II, głębokość 10.	62
5.91. Zużycie pamięci systemowej dla sceny "Lustra", platforma II, głębokość 10, dane w MiB.	62
5.92. Zużycie pamięci GPU dla sceny "Lustra", platforma II, głębokość 10, dane w MiB .	63
5.93. Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 40 - 400.	63
5.94. Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 1000 - 10000.	63
5.95. Porównanie czasu renderu sceny "Labirynt" pomiędzy wersją CPU, a CUDA-4, platforma I, głębokość 10.	63
5.96. Zużycie pamięci systemowej dla sceny "Labirynt", platforma I, głębokość 10, dane w MiB.	63
5.97. Zużycie pamięci GPU dla sceny "Labirynt", platforma I, głębokość 10, dane w MiB	64
5.98. Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 40 - 400.	64

5.99. Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 1000 - 10000.	64
5.100 Porównanie czasu renderu sceny "Labirynt" pomiędzy wersją CPU, a CUDA-4, platforma II, głębokość 10.	64
5.101 Zużycie pamięci systemowej dla sceny "Labirynt", platforma II, głębokość 10, dane w MiB.	64
5.102 Zużycie pamięci GPU dla sceny "Labirynt", platforma I, głębokość 10, dane w MiB	65
5.103 Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma I, głębokość 10, próbki 40 - 400.	65
5.104 Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma I, głębokość 10, próbki 1000 - 10000.	65
5.105 Porównanie czasu renderu sceny "Cornell Box" pomiędzy wersją CPU, a CUDA-5, platforma I, głębokość 10.	65
5.106 Zużycie pamięci systemowej dla sceny "Cornell Box", platforma I, głębokość 10, dane w MiB.	66
5.107 Zużycie pamięci GPU dla sceny "Cornell Box", platforma I, głębokość 10, dane w MiB	66
5.108 Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 40 - 400.	66
5.109 Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 1000 - 10000.	66
5.110 Porównanie czasu renderu sceny "Cornell Box" pomiędzy wersją CPU, a CUDA-5, platforma II, głębokość 10.	66
5.111 Zużycie pamięci systemowej dla sceny "Cornell Box", platforma II, głębokość 10, dane w MiB.	67
5.112 Zużycie pamięci GPU dla sceny "Cornell Box", platforma II, głębokość 10, dane w MiB	67
5.113 Czas potrzebny na wygenerowanie sceny "Lustra", platforma I, głębokość 10, próbki 40 - 400.	67
5.114 Czas potrzebny na wygenerowanie sceny "Lustra", platforma I, głębokość 10, próbki 1000 - 10000.	67
5.115 Porównanie czasu renderu sceny "Lustra" pomiędzy wersją CPU, a CUDA-5, platforma I, głębokość 10.	67
5.116 Zużycie pamięci systemowej dla sceny "Lustra", platforma I, głębokość 10, dane w MiB.	68
5.117 Zużycie pamięci GPU dla sceny "Lustra", platforma I, głębokość 10, dane w MiB .	68
5.118 Czas potrzebny na wygenerowanie sceny "Lustra", platforma II, głębokość 10, próbki 40 - 400.	68
5.119 Czas potrzebny na wygenerowanie sceny "Lustra", platforma II, głębokość 10, próbki 1000 - 10000.	68
5.120 Porównanie czasu renderu sceny "Lustra" pomiędzy wersją CPU, a CUDA-5, platforma II, głębokość 10.	68
5.121 Zużycie pamięci systemowej dla sceny "Lustra", platforma II, głębokość 10, dane w MiB.	69
5.122 Zużycie pamięci GPU dla sceny "Lustra", platforma II, głębokość 10, dane w MiB .	69
5.123 Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 40 - 400.	69
5.124 Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 1000 - 10000.	69

5.125	Porównanie czasu renderu sceny "Labirynt" pomiędzy wersją CPU, a CUDA-5, platforma I, głębokość 10.	69
5.126	Zużycie pamięci systemowej dla sceny "Labirynt", platforma I, głębokość 10, dane w MiB.	69
5.127	Zużycie pamięci GPU dla sceny "Labirynt", platforma I, głębokość 10, dane w MiB	70
5.128	Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 40 - 400.	70
5.129	Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 1000 - 10000.	70
5.130	Porównanie czasu renderu sceny "Labirynt" pomiędzy wersją CPU, a CUDA-5, platforma II, głębokość 10.	70
5.131	Zużycie pamięci systemowej dla sceny "Labirynt", platforma II, głębokość 10, dane w MiB.	70
5.132	Zużycie pamięci GPU dla sceny "Labirynt", platforma II, głębokość 10, dane w MiB	70
6.1.	Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma I, głębokość 10, próbki 40 - 400.	74
6.2.	Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma I, głębokość 10, próbki 1000 - 10000.	75
6.3.	Porównanie czasu renderu sceny "Cornell Box" pomiędzy wersją CPU, a CUDA-3.1, platforma I, głębokość 10.	75
6.4.	Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 40 - 400.	75
6.5.	Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 1000 - 10000.	75
6.6.	Porównanie czasu renderu sceny "Cornell Box" pomiędzy wersją CPU, a CUDA-3.1, platforma II, głębokość 10.	75
6.7.	Czas potrzebny na wygenerowanie sceny "Lustra", platforma I, głębokość 10, próbki 40 - 400.	76
6.8.	Czas potrzebny na wygenerowanie sceny "Lustra", platforma I, głębokość 10, próbki 1000 - 10000.	76
6.9.	Porównanie czasu renderu sceny "Lustra" pomiędzy wersją CPU, a CUDA-3.1, platforma I, głębokość 10.	76
6.10.	Czas potrzebny na wygenerowanie sceny "Lustra", platforma II, głębokość 10, próbki 40 - 400.	76
6.11.	Czas potrzebny na wygenerowanie sceny "Lustra", platforma II, głębokość 10, próbki 1000 - 10000.	77
6.12.	Porównanie czasu renderu sceny "Lustra" pomiędzy wersją CPU, a CUDA-3.1, platforma II, głębokość 10.	77
6.13.	Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 40 - 400.	77
6.14.	Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 1000 - 10000.	77
6.15.	Porównanie czasu renderu sceny "Labirynt" pomiędzy wersją CPU, a CUDA-3.1, platforma I, głębokość 10.	77
6.16.	Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 40 - 400.	78
6.17.	Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 1000 - 10000.	78

6.18. Porównanie czasu renderu sceny "Labirynt" pomiędzy wersją CPU, a CUDA-3.1, platforma II, głębokość 10.	78
6.19. Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma I, głębokość 10, próbki 40 - 400.	79
6.20. Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma I, głębokość 10, próbki 1000 - 10000.	79
6.21. Porównanie czasu renderu sceny "Cornell Box" pomiędzy wersją CPU, a CUDA-3.2, platforma I, głębokość 10.	79
6.22. Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 40 - 400.	79
6.23. Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 1000 - 10000.	79
6.24. Porównanie czasu renderu sceny "Cornell Box" pomiędzy wersją CPU, a CUDA-3.2, platforma II, głębokość 10.	80
6.25. Czas potrzebny na wygenerowanie sceny "Lustra", platforma I, głębokość 10, próbki 40 - 400.	80
6.26. Czas potrzebny na wygenerowanie sceny "Lustra", platforma I, głębokość 10, próbki 1000 - 10000.	80
6.27. Porównanie czasu renderu sceny "Lustra" pomiędzy wersją CPU, a CUDA-3.2, platforma I, głębokość 10.	80
6.28. Zużycie pamięci systemowej dla sceny "Lustra", platforma I, głębokość 10, dane w MiB.	80
6.29. Czas potrzebny na wygenerowanie sceny "Lustra", platforma I, głębokość 10, próbki 40 - 400.	81
6.30. Czas potrzebny na wygenerowanie sceny "Lustra", platforma I, głębokość 10, próbki 1000 - 10000.	81
6.31. Porównanie czasu renderu sceny "Lustra" pomiędzy wersją CPU, a CUDA-3.2, platforma I, głębokość 10.	81
6.32. Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 40 - 400.	81
6.33. Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 1000 - 10000.	81
6.34. Porównanie czasu renderu sceny "Labirynt" pomiędzy wersją CPU, a CUDA-3.2, platforma I, głębokość 10.	82
6.35. Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 40 - 400.	82
6.36. Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 1000 - 10000.	82
6.37. Porównanie czasu renderu sceny "Labirynt" pomiędzy wersją CPU, a CUDA-3.2, platforma II, głębokość 10.	82
6.38. Porównanie średniej zmiany wyniku dla wersji CUDA-3.1 i CUDA-3.2 względem wersji CPU, głębokość 10.	83
6.39. Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma I, głębokość 10, próbki 40 - 400.	83
6.40. Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma I, głębokość 10, próbki 1000 - 10000.	84
6.41. Porównanie czasu renderu sceny "Cornell Box" pomiędzy wersją CUDA-3.3, a CUDA-3.4, platforma I, głębokość 10.	84
6.42. Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 40 - 400.	84

6.43. Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 1000 - 10000.	84
6.44. Porównanie czasu renderu sceny "Cornell Box" pomiędzy wersją CUDA-3.3, a CUDA-3.4, platforma II, głębokość 10.	84
6.45. Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 40 - 400.	85
6.46. Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 1000 - 10000.	85
6.47. Porównanie czasu renderu sceny "Labirynt" pomiędzy wersją CUDA-3.3, a CUDA-3.4, platforma I, głębokość 10.	85
6.48. Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 40 - 400.	85
6.49. Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 1000 - 10000.	85
6.50. Porównanie czasu renderu sceny "Labirynt" pomiędzy wersją CUDA-3.3, a CUDA-3.4, platforma II, głębokość 10.	86
6.51. Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 40 - 400.	86
6.52. Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 1000 - 10000.	86
6.53. Porównanie czasu renderu sceny "Labirynt" pomiędzy wersją CUDA-3.3, a CUDA-3.4, platforma I, głębokość 10.	86
6.54. Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 40 - 400.	87
6.55. Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 1000 - 10000.	87
6.56. Porównanie czasu renderu sceny "Labirynt" pomiędzy wersją CUDA-3.3, a CUDA-3.4, platforma II, głębokość 10.	87
6.57. Średni wynik sceny CUDA-3.4 względem sceny CUDA-3.3.	87
6.58. Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma I, głębokość 10, próbki 40 - 400.	90
6.59. Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma I, głębokość 10, próbki 1000 - 10000.	90
6.60. Porównanie czasu renderu sceny "Cornell Box" pomiędzy wersją CUDA-3.5, a CUDA-3.6, platforma I, głębokość 10.	90
6.61. Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 40 - 400.	90
6.62. Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 1000 - 10000.	90
6.63. Porównanie czasu renderu sceny "Cornell Box" pomiędzy wersją CUDA-3.5, a CUDA-3.6, platforma II, głębokość 10.	91
6.64. Czas potrzebny na wygenerowanie sceny "Lustra", platforma I, głębokość 10, próbki 40 - 400.	93
6.65. Czas potrzebny na wygenerowanie sceny "Lustra", platforma I, głębokość 10, próbki 1000 - 10000.	93
6.66. Porównanie czasu renderu sceny "Lustra" pomiędzy wersją CUDA-3.5, a CUDA-3.6, platforma I, głębokość 10.	93
6.67. Czas potrzebny na wygenerowanie sceny "Lustra", platforma II, głębokość 10, próbki 40 - 400.	93

6.68. Czas potrzebny na wygenerowanie sceny "Lustra", platforma II, głębokość 10, próbki 1000 - 10000.	93
6.69. Porównanie czasu renderu sceny "Lustra" pomiędzy wersją CUDA-3.5, a CUDA-3.6, platforma II, głębokość 10.	94
6.70. Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 40 - 400.	97
6.71. Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 1000 - 10000.	97
6.72. Porównanie czasu renderu sceny "Labirynt" pomiędzy wersją CUDA-3.5, a CUDA-3.6, platforma I, głębokość 10.	97
6.73. Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 40 - 400.	97
6.74. Porównanie czasu renderu sceny "Labirynt" pomiędzy wersją CUDA-3.5, a CUDA-3.6, platforma II, głębokość 10.	98
6.75. Średni wynik sceny CUDA-3.4 względem sceny CUDA-3.3.	98
7.1. Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma I, głębokość 10, próbki 40 - 400.	99
7.2. Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma I, głębokość 10, próbki 1000 - 10000.	100
7.3. Porównanie czasu renderów sceny "Cornell Box" pomiędzy modelami oświetlenia. Model hybrydowy jako 100%. Platforma I, głębokość 10.	100
7.4. Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 40 - 400.	100
7.5. Czas potrzebny na wygenerowanie sceny "Cornell Box", platforma II, głębokość 10, próbki 1000 - 10000.	100
7.6. Porównanie czasu renderów sceny "Cornell Box" pomiędzy modelami oświetlenia. Model hybrydowy jako 100%. Platforma II, głębokość 10.	100
7.7. Czas potrzebny na wygenerowanie sceny "Lustra", platforma I, głębokość 10, próbki 40 - 400.	105
7.8. Czas potrzebny na wygenerowanie sceny "Lustra", platforma I, głębokość 10, próbki 1000 - 10000.	105
7.9. Porównanie czasu renderów sceny "Lustra" pomiędzy modelami oświetlenia. Model hybrydowy jako 100%. Platforma I, głębokość 10.	105
7.10. Czas potrzebny na wygenerowanie sceny "Lustra", platforma II, głębokość 10, próbki 40 - 400.	106
7.11. Czas potrzebny na wygenerowanie sceny "Lustra", platforma II, głębokość 10, próbki 1000 - 10000.	106
7.12. Porównanie czasu renderów sceny "Lustra" pomiędzy modelami oświetlenia. Model hybrydowy jako 100%. Platforma II, głębokość 10.	106
7.13. Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 40 - 400.	111
7.14. Czas potrzebny na wygenerowanie sceny "Labirynt", platforma I, głębokość 10, próbki 1000 - 10000.	111
7.15. Porównanie czasu renderów sceny "Labirynt" pomiędzy modelami oświetlenia. Model hybrydowy jako 100%. Platforma I, głębokość 10.	111
7.16. Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 40 - 400.	112
7.17. Czas potrzebny na wygenerowanie sceny "Labirynt", platforma II, głębokość 10, próbki 1000 - 10000.	112

7.18. Porównanie czasu renderów sceny "Labirynt" pomiędzy modelami oświetlenia. Model hybrydowy jako 100%. Platforma II, głębokość 10.	112
--	-----

Spis listingów

2.1.	Przykładowy plik JSON sceny.	17
2.2.	JSON obiektu płaszczyzny.	18
2.3.	Komenda do uruchomienia programu <i>tracer</i>	19
2.4.	Komenda do wyświetlenia pomocy programu <i>tracer</i>	20
2.5.	Dane zapisywane w pliku <i>benchmark.txt</i>	20
3.1.	Uruchomienie skryptu <i>test_automation.py</i> z własnymi argumentami.	22
3.2.	Plik <i>benchmark.txt</i>	23
3.3.	Funkcja pomiaru czasu	23
3.4.	Przygotowanie i egzekucja testu w skrypcie <i>test_automation.py</i>	24
3.5.	Pomiar zużycia pamięci systemowej	25
3.6.	Parametru dla <i>nvidia-smi</i>	25
4.1.	Linijka powodująca błąd w systemie oświetlenie płaszczyzny	29
4.2.	Linijka powodująca błąd w systemie emisji promieni.	30
4.3.	Poprawiony kod obliczający zakrzywienie promienia spojrzenia.	31
4.4.	Tworzenie współdzielonego obiektu <i>tracer</i>	32
4.5.	Wyznaczenie odległości do płaszczyzny przed optymalizacją.	33
4.6.	Wyznaczenie odległości do płaszczyzny po optymalizacji.	33
4.7.	Funkcja obliczająca odległość punktu do boku prostokąta przed optymalizacją.	34
4.8.	Funkcja obliczająca odległość punktu do boku prostokąta po optymalizacji.	35
4.9.	Funkcja obliczająca odległość punktu do boku prostokąta po optymalizacji.	37
4.10.	Warunek wcześniejszego przerwania algorytmu śledzenia promienia dla obiektów rozproszających światło (Wersja <i>CUDA-3.6</i>).	37
A.1.	Kompilacja programu <i>tracer</i>	139
A.2.	Uruchomienie programu <i>tracer</i>	139

Dodatek A

Instrukcja wdrożeniowa

A.1. Wymagania:

A.1.1. System operacyjny

Aplikacja przetestowana na systemie Kubuntu w wersji 24.04. Autor zaleca wykorzystanie tego systemu operacyjnego w celu zapewnienia kompatybilności. Inne wersje systemu Linux także powinny działać bez problemów.

Uruchomienie poniższej aplikacji powinno być także możliwe pod system Windows. Taka konfiguracja nie była jednak testowana i autor nie może zagwarantować kompatybilności.

A.1.2. Sprzęt

- Dowolny procesor architektury x86_64,
- 1 GB pamięci RAM,
- 60 MiB przestrzeni dyskowej,
- Połączenie z Internetem (potrzebne w trakcie pierwszej kompilacji),
- Dla wersji GPU: Karta graficzna wspierająca CUDA w co najmniej wersji 12.4.

A.1.3. Podstawowe narzędzia kompilacyjne

- g++ - wersja 13.2.0 lub nowsza,
- gcc - wersja 13.2.0 lub nowsza,
- make - wersja 4.3 lub nowsza,
- cmake - wersja 3.28.3 lub nowsza,
- nvcc - wersja 12.4 lub nowsza.

A.1.4. Zewnętrzne biblioteki

- nlohmann json - przetestowane z wersją 3.11.2,
- ImageMagick 7 - przetestowane z wersją 7.1.1-24. **Uwaga:** Wersja 6 nie posiada funkcjonalności wykorzystywanych przez program *tracer*. Ponieważ wersja 7 w momencie pisania tego dokumentu nie jest dostępna w repozytoriach niektórych dystrybucji systemu Linux, autor poleca następujące narzędzie do instalacji najnowszej wersji: ,
- sterownik CUDA w wersji 12.4 lub nowszej. Instrukcje instalacji (dla systemu Linux) dostępne na stronie producenta: .

A.2. Kompilacja

Do skompilowania programu należy użyć komendy:

Listing A.1: Kompilacja programu *tracer*.

```
make compile-release
```

A.3. Uruchomienie programu

Do uruchomienia programu należy użyć komendy:

Listing A.2: Uruchomienie programu *tracer*.

```
.\tracer -d=10 -s=40 scene.json
```

- `[-d/-depth]` (Opcjonalne - domyślnie 10) - maksymalna dozwolona ilość odbić promienia na scenie,
- `[-s/-samples]` (Opcjonalne - domyślnie 40) - ilość próbek na piksel obrazu,
- `scene.json` - ścieżka do pliku JSON zawierającego dane sceny.

A.4. Plik Json

Uwaga: Obiekt typu Wektor - obiekt zawierający trzy zmiennoprzecinkowe liczby *xx*, *yy* oraz *zz*.

Wymagane pola:

- *height* - liczba całkowita - rozdzielczość pionowa renderowanego obrazu,
- *width* - liczba całkowita - rozdzielczość pozioma renderowanego obrazu,
- *camera* - kompleksowy obiekt - obiekt definiujący parametry kamery,
- *objects* - lista obiektów znajdujących się na scenie.

A.4.1. Obiekt kamera

- *direction* - Wektor - Wektor normalny zgodny z kierunkiem spojrzenia z kamery,
- *orientation* - Wektor - Wektor normalny wskazujący górę kamery,
- *position* - Wektor - Współrzędne środka kamery w układzie XYZ.

A.4.2. Obiekt Płaszczyzny

- *type* - string - "plane",
- *reflection* - enum - Diffuse (0), Specular (1) lub Refractive (2),
- *position* - Wektor - Współrzędne środka obiektu w układzie XYZ,
- *north* - Wektor - Wektor długości połowy boku skierowany w górę płaszczyzny,
- *east* - Wektor - Wektor długości połowy boku skierowany na prawo płaszczyzny,
- *color* - Wektor - Kolor obiektu określony gdzie wartości oznaczają nasycenie koloru kolejno: czerwonego, zielonego i niebieskiego. Wartość te są w zakresie od 0 do 1,
- *emission* - Wektor - Moc emisji danej płaszczyzny. Używane jeżeli dana płaszczyzna jest źródłem światła.

A.4.3. Obiekt Kuli

- *type* - string - "sphere",
- *reflection* - enum - Diffuse (0), Specular (1) lub Refractive (2),
- *radius* - liczna zmiennoprzecinkowa Długość promienia kuli,
- *position* - Wektor - Współrzędne środka obiektu w układzie XYZ,
- *color* - Wektor - Kolor obiektu określony gdzie wartości oznaczają nasycenie koloru kolejno: czerwonego, zielonego i niebieskiego. Wartość te są w zakresie od 0 do 1.
- *emission* - Wektor - Moc emisji danej płaszczyzny. Używane jeżeli dana kula jest źródłem światła.

Dodatek B

Opis załączonej płyty CD/DVD

- Skompresowany folder *.zip* z folderami zawierającymi kod źródłowy wszystkich wersji programu,
- Skompresowany folder *.zip* zawierający pliki Json scen testowych,
- Skompresowany folder *.zip* zawierający renderery wykonane w trakcie testów aplikacji. Są one posortowane w folderach zgodnie z modelami, dla których zostały wykonane
- Plik PDF będący cyfrową kopią tego dokumentu.

W celu zapewnienia poprawnego działania programu w wersji GPU wersja CUDA użyta do kompilacji musi być dokładnie zgodna z wersją użytą do uruchomienia programu. W związku z tym pre-kompilowane wersje aplikacji nie zostały załączone na płycie CD. Instrukcja jak skompilować program zostały załączone w Dodatku A.